

UNIVERSITAS GUNADARMA

FAKULTAS Ilmu Komputer dan Teknologi Informasi



SKRIPSI

**APLIKASI AUTOML BERBASIS WEBSITE MENGGUNAKAN METODE
DSR DAN WATERFALL**

Disusun Oleh:

Nama : I Gusti Putu Yoga Wiratama
NPM : 10122609
Program Studi : Sistem Informasi
Pembimbing : Dr. rer. nat. I Made Wiryana, S.Si., S.Kom., M.SC.

Diajukan Guna Melengkapi Sebagian Syarat Dalam Mencapai

Gelar Sarjana Strata Satu (S1)

JAKARTA

2026

PERNYATAAN ORIGINALITAS DAN PUBLIKASI

Saya yang bertanda tangan di bawah ini:

Nama : I Gusti Putu Yoga Wiratama

NPM : 10122609

Judul Skripsi :

Tanggal Sidang :

Tanggal Lulus :

Menyatakan bahwa tulisan ini adalah merupakan hasil karya saya sendiri dan dapat dipublikasikan sepenuhnya oleh Universitas Gunadarma. Segala kutipan dalam bentuk apapun telah mengikuti kaidah dan etika yang berlaku. Semua hak cipta dari logo serta produk yang disebut dalam buku ini adalah milik masing-masing pemegang haknya, kecuali disebutkan lain. Mengenai isi dan tulisan merupakan tanggung jawab Penulis, bukan Universitas Gunadarma.

Demikianlah pernyataan ini dibuat dengan sebenarnya dan dengan penuh kesadaran.

, 8 026

(Gusti Putu Yoga Wiratama)

LEMBAR PENGESAHAN

Judul Skripsi : APLIKASI AUTOML BERBASIS WEBSITE
MENGGUNAKAN METODE DSR DAN WATERFALL

Nama : Gusti Putu Yoga Wiratama

NPM : 0122609

Tanggal Lulus :

PANITIA UJIAN

No.	Nama	Kedudukan
1.	Dr. Ravi Ahmad Salim, SSi.	Ketua
2.	Prof. Dr., Wahyudi Priyono Suwarso.	Sekretaris
3.	Dr. I Komang Sugiartha, SKom., MMSI.	Anggota
4.	Dr. Ida Astuti, SKom., MMSI.	Anggota
5.	Dr. Miftahul Jannah, SKom., MMSI.	Anggota

Menyetujui,

Pembimbing

Bagian Sidang Ujian

(Dr. rer. nat. I Made Wiryana, S.Si., S.Kom., M.SC.)

(Dr. Edi Sukirman, SSi., MM., M.I.Ko

ABSTRAK

Gusti Putu Yoga Wiratama. 0122609

APLIKASI AUTOML BERBASIS WEBSITE MENGGUNAKAN METODE DSR DAN WATERFALL.

Skripsi. Program studi Informasi, Fakultas Kompute dan Teknologi Informasi, Universitas Gunadarma, 026.

Kata Kunci: Machine Learning, FastAPI, Large Language Model, Ollama, Pelindungan Data Pribadi.

(XV+ 77+ lampiran)

Perkembangan teknologi kecerdasan buatan mendorong kebutuhan akan alat analisis data yang dapat dimanfaatkan oleh peneliti dari berbagai latar belakang keilmuan secara mandiri. Penelitian ini bertujuan untuk mengembangkan komponen back end platform Automated Machine Learning (AutoML) berbasis web yang mengintegrasikan tiga kapabilitas utama: otomatisasi pipeline machine learning untuk data tabular, gambar, dan teks; analisis teks kualitatif berbasis Large Language Model (LLM) lokal melalui Ollama; serta mekanisme pengelolaan persetujuan data pribadi sesuai Undang-Undang Nomor 27 Tahun 2022 tentang Pelindungan Data Pribadi. Penelitian ini menggunakan Design Science Research (DSR) sebagai metode penelitian dengan metode pengembangan Waterfall, menghasilkan artefak berupa sistem back end berbasis FastAPI dengan arsitektur berlapis yang terdiri dari delapan modul terintegrasi. Pengujian dilakukan menggunakan metode white box testing terhadap seluruh fungsi utama sistem. Hasil pengujian menunjukkan bahwa seluruh modul berhasil diverifikasi sesuai rancangan, mencakup alur autentikasi berbasis JWT, deteksi peran dataset secara otomatis, enkripsi NIK menggunakan algoritma Fernet, mekanisme penguncian dataset berdasarkan status persetujuan subjek, eksekusi preprocessing dan pelatihan model secara asinkron, serta analisis LLM dengan evaluasi kualitas menggunakan metrik ROUGE. Platform ini diharapkan dapat memfasilitasi peneliti di bidang ilmu sosial dan psikologi dalam memanfaatkan teknologi machine learning dan kecerdasan buatan secara mandiri, efisien, dan bertanggung jawab secara etis maupun hukum.

Daftar Pustaka (011-2026)

ABSTRACT

Gusti Putu Yoga Wiratama. 0122609

IMPLEMENTATION OF A CHATBOT BASED ON RETRIEVAL-AUGMENTED GENERATION (RAG) LOCALLY USING THE OLLAMA PLATFORM.

Undergraduate Thesis. Study Program System, Faculty of Industrial Technology, Gunadarma University, 026.

Keywords: Machine Learning, FastAPI, Large Language Model, Ollama, Personal Data Protection

(XV+ 77+ attachment)

The advancement of artificial intelligence technology has driven the need for data analysis tools that can be used independently by researchers from various academic backgrounds. This study aims to develop the back-end component of a web-based Automated Machine Learning (AutoML) platform that integrates three main capabilities: automation of the machine learning pipeline for tabular, image, and text data; qualitative text analysis based on a locally-run Large Language Model (LLM) via Ollama; and a personal data consent management mechanism in accordance with Indonesian Law Number 27 of 2022 on Personal Data Protection. This study employs Design Science Research (DSR) as the research methodology with the Waterfall development method, producing an artifact in the form of a FastAPI-based back-end system with a layered architecture consisting of eight integrated modules. Testing was conducted using the white box testing method on all major system functions. The test results demonstrate that all modules were successfully verified in accordance with the design, covering JWT-based authentication flow, automatic dataset role detection, NIK encryption using the Fernet algorithm, dataset locking mechanism based on subject consent status, asynchronous preprocessing and model training execution, and LLM analysis with quality evaluation using ROUGE metrics. This platform is expected to facilitate researchers in the social sciences and psychology in utilizing machine learning and artificial intelligence technology independently, efficiently, and in an ethically and legally responsible manner.

References (011-2026)

KATA PENGANTAR

Dengan mengucapkan segala puji dan syukur atas kehadiran Allah SWT yang telah memberikan rahmat dan karunia-Nya sehingga penulis dapat menyelesaikan penulisan skripsi ini pada waktu yang telah ditentukan.

Adapun tujuan dari penulisan skripsi ini adalah sebagai salah satu syarat untuk menyelesaikan Program Sarjana (S1) Jurusan Informatika, Universitas Gunadarma. Adapun judul penulisan skripsi ini adalah “IMPLEMENTASI CHATBOT BERBASIS *RETRIEVAL-AUGMENTED GENERATION* (RAG) SECARA LOKAL MENGGUNAKAN PLATFORM OLLAMA”.

Walaupun banyak kesulitan yang penulis harus hadapi ketika menyusun penulisan ini, namun berkat bantuan dan dorongan dari berbagai pihak, akhirnya penulisan skripsi ini dapat diselesaikan dengan baik. Untuk itu penulis tidak lupa mengucapkan terima kasih kepada:

1. Prof. DR. Hj. E.S. Margianti, SE, MM., selaku Rektor Universitas Gunadarma.
2. Prof. Dr. Ing. Adang Suhendra, S.Si., S.Kom., M.Sc., selaku Dekan Fakultas Teknologi Industri Universitas Gunadarma.
3. Prof. Dr. Lintang Yuniar Banowosari, S.Kom., M.Sc., selaku Ketua Program Studi Informatika Universitas Gunadarmas.
4. Dr. Edi Sukirman, S.Si., MM., M.I.Kom., selaku Bagian Sidang Ujian Universitas Gunadarma.
5. . rer. nat. I Made Wiryana, S.Si., S.Kom., M.SC., selaku dosen Pembimbing penulis yang selalu memberikan bimbingan, motivasi, arahan serta dorongan kepada penulis sehingga penulisan ini dapat diselesaikan.

Sebagai manusia biasa yang tak luput dari kesalahan, maka penulis meminta maaf atas segala kekurangan dan keterbatasan dalam penyusunan skripsi ini.

Akhir kata, hanya kepada Tuhan jugalah segalanya dikembalikan dan penulis sadari bahwa penulisan ini masih jauh dari sempurna, disebabkan karena berbagai keterbatasan yang penulis miliki. Untuk itu penulis mengharapkan kritik dan saran yang bersifat membangun untuk menjadi perbaikan di masa yang akan datang.

Semoga apa yang ada pada penulisan skripsi ini dapat bermanfaat bagi kita semua. Amin.

, 8 026

Gusti Putu Yoga Wiratama

DAFTAR ISI

HALAMAN JUDUL	i
PERNYATAAN ORISINALITAS DAN PUBLIKASI	ii
LEMBAR PENGESAHAN	iii
ABSTRAK	iv
ABSTRACT	v
KATA PENGANTAR	vi
DAFTAR ISI	viii
DAFTAR TABEL	xiii
DAFTAR GAMBAR	xiv
DAFTAR LAMPIRAN	xvi
1 PENDAHULUAN	1
1.1 Latar Belakang	1
1.2 Rumusan Masalah	3
1.3 Ruang Lingkup	3
1.4 Tujuan Penelitian	4
1.5 Sistematika Penulisan	4

2 TINJAUAN PUSTAKA	6
2.1 <i>Design Science Research (DSR)</i>	6
2.2 Metode Pengembangan Waterfall	8
2.3 Privacy by Design	10
2.4 Aplikasi Berbasis Website	11
2.5 Python	12
2.5.1 Karakteristik dan Keunggulan Python	13
2.5.2 Python dalam Machine Learning dan Data Science	14
2.6 FastAPI	14
2.6.1 Arsitektur Asinkron ASGI dan WSGI	15
2.6.2 Fitur -Fitur Utama FastAPI	16
2.6.3 Validasi data dengan Pydantic	17
2.6.4 FastAPI Sebagai Jembatan Machine Learning dan Frontend	17
2.7 ReactJS	18
2.7.1 Arsitektur Berbasis Komponen	19
2.7.2 Virtual DOM	19
2.7.3 State, Props, dan React Hooks	20
2.7.4 Single Page Application	21
2.8 RestAPI	21
2.9 Autentikasi dan Otorisasi	22
2.9.1 Konsep Otentikasi dan Otorisasi	23
2.9.2 JSON Web Token (JWT)	23
2.9.3 Perbandingan JWT dengan Session-Based Authentication	24
2.9.4 Keamanan dan Mitigasi Risiko JWT	24
2.10SQLite	25
2.11Machine Learning	26

2.12	Automated Machine Learning	26
2.13	Large Language Model dan Ollama	27
3	METODE PENELITIAN	29
3.1	Pendekatan Penelitian	29
3.2	Metode Pengembangan Sistem	30
3.3	Rancangan Backend	32
3.3.1	Arsitektur Backend	32
3.3.2	Struktur Modul dan Endpoint	33
3.3.3	Skema Basis Data	36
3.3.4	Mekanisme Keamanan	46
3.4	Diagram IDEF0	46
3.4.1	Diagram A0 Mengelola Platform AutoML	46
3.4.2	Diagram A1-A6	48
3.4.3	Diagram Dekomposisi A1: Autentikasi & Manajemen Role	49
3.4.4	Diagram Dekomposisi A2: Upload Dataset & Privacy Consent	50
3.4.5	Diagram Dekomposisi A3: Preprocessing Data	51
3.4.6	Diagram Dekomposisi A4: Training & Eksperimen ML	52
3.4.7	Diagram Dekomposisi A5: Analisis LLM	53
3.4.8	Diagram Dekomposisi A6: Evaluasi Model	54
3.5	Diagram BPMN	55
3.5.1	BPMN Upload Dataset dan Privacy Consent	55
3.5.2	BPMN Preprocessing Dataset	56
3.5.3	BPMN Training Model	57
3.5.4	BPMN Analisis LLM	59
3.5.5	BPMN Hasil Eksperimen	60

3.5.6	BPMN Hasil Analisis LLM	61
3.6	Struktur Rancangan Implementasi	62
3.7	Pemilihan Teknologi	66
4	HASIL DAN PEMBAHASAN	68
4.1	Implementasi Modul Autentikasi & Manajemen Role	68
4.1.1	Fungsi login()	68
4.1.2	Fungsi get_current_user()	69
4.1.3	Fungsi require_role()	71
4.1.4	Fungsi create_user()	72
4.2	Implementasi Modul Dataset	73
4.2.1	Fungsi _detect_dataset_role()	73
4.2.2	Fungsi upload_tabular()	75
4.2.3	Fungsi save_image_zip()	77
4.2.4	Fungsi get_dataset_quality()	79
4.3	Implementasi Modul Privacy Consent	80
4.3.1	Fungsi encrypt_nik()	81
4.3.2	Fungsi upload_consent_pdf()	82
4.3.3	Fungsi submit_consent()	83
4.4	Implementasi Modul Preprocessing	86
4.4.1	Fungsi create_preprocessing_job()	86
4.4.2	Fungsi run_preprocessing_job()	88
4.4.3	Fungsi start_preprocessing() pada Router . .	90
4.5	Implementasi Modul Eksperimen & Training	92
4.5.1	Fungsi create_experiment()	92
4.5.2	Fungsi run_training_job()	94
4.5.3	Fungsi download_model()	98
4.6	Implementasi Modul Analisis LLM	99
4.6.1	Fungsi _build_prompt()	99

4.6.2 Fungsi _run_analysis()	101
4.6.3 Fungsi _calculate_rouge()	104
4.6.4 Fungsi generate_analysis() pada Router	105
4.7 Pengujian	108
4.7.1 Pengujian Modul Autentikasi dan Manajemen Role	108
4.7.2 Pengujian Modul Dataset	110
4.7.3 Pengujian Modul Privacy Consent	114
4.7.4 Pengujian Modul Preprocessing	117
4.7.5 Pengujian Modul Eksperimen dan Training	119
4.7.6 Pengujian Modul Analisis LLM	122
5 PENUTUP	126
5.1 Kesimpulan	126
5.2 Saran	127
DAFTAR PUSTAKA	134
LAMPIRAN	L-1

DAFTAR TABEL

3.1	Tabel Data Users	37
3.2	Tabel Data Datasets	38
3.3	Tabel Data Preprocessings	39
3.4	Tabel Data Eksperimen	41
3.5	Tabel Data privacy_consent	42
3.6	Tabel Data privacy_concent_subjects	43
3.7	Tabel Data Metadata	44
3.8	Tabel Data llm_analyses	45
3.9	Teknologi yang Digunakan	67
4.1	Pengujian Modul Autentikasi	108
4.2	Pengujian Modul Dataset	110
4.3	Pengujian Modul Privacy Concent	114
4.4	Pengujian Modul Preprocessing	117
4.5	Pengujian Modul Eksperimen dan Training	119
4.6	Pengujian Modul Analisis LLM	122

DAFTAR GAMBAR

2.1	Design Science Research	7
2.2	Struktur Metode Pengembangan Waterfall	9
3.1	Diagram IDEF0 Level 1	47
3.2	Diagram IDEF0 A1-A6	48
3.3	Diagram A1: Autentikasi & Manajemen Role	49
3.4	Diagram Dekomposisi A2: Upload Dataset & Privacy Consent	50
3.5	Diagram Dekomposisi A3: Preprocessing Data	51
3.6	Diagram Dekomposisi A4: Training & Eksperimen ML	52
3.7	Diagram Dekomposisi A5: Analisis LLM	53
3.8	Diagram Dekomposisi A6: Evaluasi Model	54
3.9	BPMN Upload Dataset dan Privacy Consent	55
3.10	BPMN Preprocessing Dataset	56
3.11	BPMN Training Model	58
3.12	BPMN Analisis LLM	59
3.13	BPMN Hasil Eksperimen	60
3.14	BPMN Hasil Analisis LLM	61
3.15	Struktur Rancangan Folder Aplikasi Backend	63
4.1	Code Fungsi Login	69
4.2	Code Fungsi get_current_user()	70
4.3	Code Fungsi require_role()	71
4.4	Code Fungsi create_user()	72
4.5	Code Fungsi _detect_dataset_role()	74
4.6	Code Fungsi upload_tabular()	75
4.7	Code Fungsi save_image_zip()	77
4.8	Code Fungsi get_dataset_quality()	79
4.9	Code Fungsi encrypt_nik()	81
4.10	Code Fungsi upload_consent_pdf()	82

4.11 Code Fungsi submit_consent()	84
4.12 Code Fungsi create_preprocessing_job()	87
4.13 Code Fungsi run_preprocessing_job()	89
4.14 Code Fungsi start_preprocessing() pada Router	91
4.15 Code Fungsi create_experiment()	93
4.16 Code Fungsi run_training_job()	96
4.17 Code Fungsi download_model()	98
4.18 Code Fungsi _build_prompt()	100
4.19 Code Fungsi _run_analysis()	102
4.20 Code Fungsi _calculate_rouge()	104
4.21 Code Fungsi generate_analysis() pada Router	106

DAFTAR LAMPIRAN

Lampiran 1 <i>Listing Program</i>	L-1
Lampiran 2 <i>Output Program</i>	L-1

1. PENDAHULUAN

1.1 Latar Belakang

Perkembangan pesat teknologi kecerdasan buatan dan machine learning telah membuka peluang besar bagi berbagai bidang ilmu untuk memanfaatkan data dalam pengambilan keputusan yang lebih akurat dan berbasis bukti. Di berbagai domain seperti kesehatan, pendidikan, dan ilmu sosial, pertumbuhan data berbasis teks dan data terstruktur yang dihasilkan dari aktivitas penelitian terus meningkat secara eksponensial, melampaui kapasitas metode analisis konvensional yang bersifat manual, memakan waktu, dan rentan terhadap subjektivitas (Barros dkk., 2025). Kondisi ini mendorong kebutuhan yang mendesak akan sistem yang mampu membantu peneliti dari berbagai latar belakang keilmuan dalam mengolah, menganalisis, dan menghasilkan temuan dari data yang mereka kumpulkan secara lebih efisien.

Namun demikian, penerapan machine learning dalam penelitian ilmiah kerap menghadapi hambatan yang signifikan, terutama bagi peneliti yang tidak memiliki latar belakang teknis di bidang pemrograman atau ilmu komputer. Penerapan machine learning oleh kalangan non-teknis masih sangat terbatas akibat kompleksitas alur pengembangan konvensional yang mensyaratkan keahlian software engineering dan data science yang mendalam (Giraldo & Laso, 2025). Kesenjangan ini menyebabkan banyak peneliti, khususnya di bidang ilmu sosial, psikologi, dan humaniora, tidak dapat memanfaatkan potensi penuh teknologi machine learning meskipun data yang mereka miliki sesungguhnya sangat potensial untuk dianalisis lebih lanjut menggunakan pendekatan berbasis kecerdasan buatan.

Sebagai respons terhadap tantangan tersebut, pendekatan Automated Machine Learning (AutoML) hadir untuk mengotomatisasi seluruh tahap-

an pemodelan machine learning sehingga peneliti dapat membangun model prediktif yang andal tanpa perlu menulis kode secara manual. Platform-platform AutoML berbasis low-code dan no-code telah muncul sebagai solusi yang menjanjikan untuk mendemokratisasi akses terhadap machine learning dengan mengabstraksikan banyak tugas teknis yang terlibat dalam pipeline pengembangan (Giraldo & Laso, 2025). Selain otomatisasi pemodelan, kebutuhan penelitian modern juga mencakup kemampuan menganalisis data kualitatif yang kompleks seperti transkrip wawancara mendalam yang secara tradisional membutuhkan waktu dan tenaga analis yang sangat besar. Large Language Model (LLM) telah muncul sebagai alat yang transformatif dan mampu mengotomatisasi serta meningkatkan proses analisis kualitatif yang sebelumnya menuntut keterlibatan manusia secara ekstensif (Barros dkk., 2025).

Di sisi lain, data yang digunakan dalam penelitian ilmiah di bidang psikologi dan ilmu sosial seringkali memuat informasi pribadi subjek penelitian yang bersifat sensitif. Undang-Undang Nomor 27 Tahun 2022 tentang Perlindungan Data Pribadi (UU PDP) mengamanatkan bahwa pemrosesan data pribadi harus didasari persetujuan eksplisit dari subjek data yang disampaikan secara tertulis atau terekam. Kerentanan fundamental dalam banyak sistem yang mengelola data penelitian bermuara pada lemahnya mekanisme pengelolaan persetujuan dan perlindungan data pribadi sejak tahap perancangan, yang tidak hanya merupakan pelanggaran etika keprofesian, tetapi juga pelanggaran terhadap regulasi yang berlaku (Nurmiati & Kusuma, 2026). Kepatuhan terhadap ketentuan hukum perlindungan data bukan sekadar kewajiban administratif, melainkan keharusan arsitektural yang wajib diimplementasikan sejak fase awal pengembangan sistem informasi (Nurmiati & Kusuma, 2026).

Berdasarkan latar belakang tersebut, penelitian ini bertujuan untuk mengembangkan backend sebuah platform Automated Machine Learning berbasis web yang mengintegrasikan tiga kapabilitas utama yaitu otomatisasi pipeline machine learning untuk berbagai jenis data sehingga peneliti dari latar belakang non-teknis dapat memanfaatkan teknologi machine learning secara mandiri, analisis teks kualitatif berbasis Large Language Model lokal sehingga proses analisis data wawancara yang sebelumnya dilakukan secara manual dapat dilakukan secara otomatis dan efisien serta sistem pengelolaan persetujuan data yang memastikan setiap pemrosesan data pribadi subjek

penelitian dilakukan sesuai dengan amanat regulasi perlindungan data yang berlaku. Dengan adanya platform ini, diharapkan para peneliti dari berbagai bidang keilmuan dapat memanfaatkan teknologi kecerdasan buatan secara mandiri, efisien, dan bertanggung jawab secara etis maupun hukum.

1.2 Rumusan Masalah

Berdasarkan latar belakang yang telah dipaparkan, maka rumusan masalah dalam penelitian ini adalah sebagai berikut:

1. Bagaimana merancang dan membangun komponen backend platform Automated Machine Learning berbasis web yang mampu mengelola seluruh tahapan pipeline machine learning secara modular dan dapat diakses tanpa keahlian pemrograman mendalam?
2. Bagaimana mengintegrasikan fitur analisis teks berbasis Large Language Model lokal melalui Ollama ke dalam back end platform sehingga proses analisis data kualitatif dapat dilakukan secara otomatis dan efisien?
3. Bagaimana mengimplementasikan mekanisme pengelolaan persetujuan data pribadi (privacy consent management) pada sistem back end yang memenuhi ketentuan Undang-Undang Nomor 27 Tahun 2022 tentang Pelindungan Data Pribadi?

1.3 Ruang Lingkup

Ruang lingkup penelitian mencakup:

1. Penelitian ini berfokus pada pengembangan komponen back end platform AutoML, mencakup perancangan arsitektur sistem, endpoint API, skema basis data, mekanisme keamanan, preprocessing data, pelatihan model, analisis LLM, dan privacy consent. Komponen front end dan ML service dikembangkan oleh anggota tim yang berbeda sesuai pembagian peran dalam proyek penelitian ini.
2. Algoritma machine learning yang diimplementasikan adalah algoritma berbasis Scikit-learn, meliputi Decision Tree, Random Forest, Support

Vector Machine, K-Nearest Neighbor, Logistic Regression, dan Linear Regression, yang dipilih karena kesesuaiannya dengan karakteristik data.

3. Platform ini dirancang untuk mendukung tiga tipe data yang paling umum digunakan dalam penelitian, yaitu data tabular (CSV), data gambar (ZIP), dan data teks (TXT).
4. Fitur analisis LLM diimplementasikan menggunakan model yang berjalan secara lokal melalui Ollama.

1.4 Tujuan Penelitian

Berdasarkan rumusan masalah yang telah ditetapkan, tujuan penelitian ini adalah sebagai berikut:

1. Merancang dan membangun komponen back end platform Automated Machine Learning berbasis web menggunakan framework FastAPI yang mampu mengelola seluruh tahapan pipeline machine learning secara modular untuk data tabular, gambar, dan teks.
2. Mengintegrasikan fitur analisis teks berbasis Large Language Model lokal melalui Ollama ke dalam sistem back end platform sebagai sarana otomatisasi analisis data kualitatif dengan evaluasi kualitas hasil menggunakan metrik ROUGE.
3. Mengimplementasikan mekanisme privacy consent management pada sistem back end yang menerapkan persetujuan granular, enkripsi data identitas subjek, dan mekanisme penguncian dataset sesuai amanat Undang-Undang Nomor 27 Tahun 2022 tentang Pelindungan Data Pribadi.

1.5 Sistematika Penulisan

Penelitian ini disusun dalam lima bab dengan sistematika sebagai berikut:

Bab I Pendahuluan membahas gambaran umum penelitian yang mencakup latar belakang yang menjelaskan konteks dan urgensi penelitian, rumusan masalah yang diidentifikasi, batasan masalah yang menetapkan ruang lingkup penelitian, tujuan yang ingin dicapai, metode penelitian yang digunakan, serta sistematika penulisan laporan penelitian.

Bab II Tinjauan Pustaka membahas landasan teori yang menjadi fondasi ilmiah penelitian ini, mencakup konsep aplikasi berbasis web, machine learning beserta paradigma dan algoritma-algoritmanya, Automated Machine Learning, evaluasi model, teknologi-teknologi yang digunakan dalam pengembangan sistem (Python, Scikit-learn, FastAPI, ReactJS, REST API, SQLite), autentikasi dan otorisasi, pemrosesan asinkron, perlindungan data dan privasi, analisis teks berbasis Large Language Model dengan Ollama, serta metode penelitian DSR dan metode pengembangan Waterfall.

Bab III Metodologi Penelitian membahas metode dan perancangan sistem yang digunakan dalam penelitian, mencakup penerapan metode DSR dan Waterfall, rancangan arsitektur back end, struktur modul dan endpoint API, skema basis data, mekanisme keamanan sistem, serta pemodelan sistem menggunakan diagram IDEF0 dan BPMN.

Bab IV Hasil dan Pembahasan membahas hasil implementasi dan pengujian sistem yang telah dibangun, mencakup implementasi seluruh modul back end beserta hasil pengujian white box testing pada setiap fungsi utama sistem untuk memverifikasi kesesuaian perilaku sistem dengan rancangan yang telah ditetapkan.

Bab V Penutup membahas kesimpulan yang ditarik dari hasil penelitian berdasarkan tujuan yang telah ditetapkan, serta saran-saran untuk pengembangan penelitian selanjutnya.

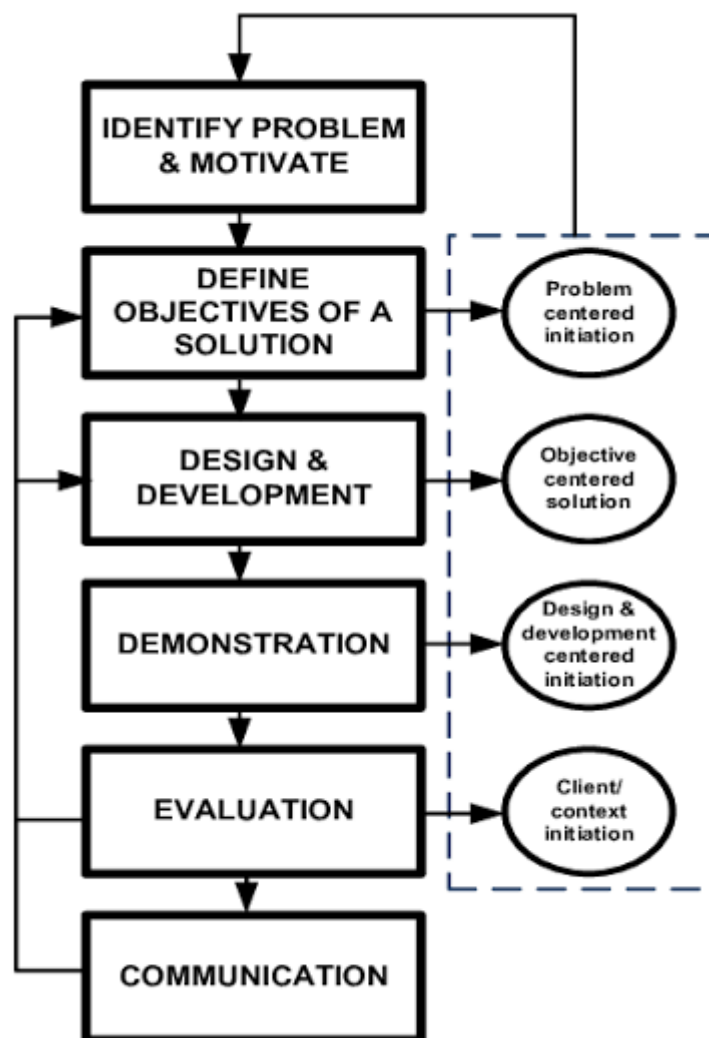
2. TINJAUAN PUSTAKA

2.1 *Design Science Research (DSR)*

Design Science Research (DSR) adalah paradigma penelitian dalam bidang Sistem Informasi yang berfokus pada penciptaan dan evaluasi artefak teknologi inovatif sebagai solusi atas permasalahan yang relevan di dunia nyata. DSR merupakan metodologi penelitian yang penting dalam disiplin ilmu yang berorientasi pada penciptaan artefak yang berhasil, dan metodologi ini menggabungkan prinsip-prinsip, praktik-praktik, serta prosedur-prosedur yang diperlukan untuk melaksanakan penelitian jenis ini secara terstruktur (Peppers dkk., 2007). Paradigma ini berbeda secara mendasar dari behavioral science yang bertujuan memahami dan menjelaskan realitas yang ada, DSR justru mengupayakan penciptaan kembali realitas tersebut melalui pengembangan sebuah artefak Sistem Informasi yang dirancang untuk mengubah atau memperbaiki keadaan yang ada, kemudian mengevaluasinya untuk membuktikan kontribusinya dalam menyelesaikan persoalan yang diformulasikan sebelumnya (Inan, 2023).

Peppers dkk. (2007) memperkenalkan Design Science Research Methodology (DSRM) sebagai model proses nominal yang terdiri dari enam tahapan utama yang dilaksanakan secara berurutan, yaitu:

1. Problem identification and motivation (identifikasi masalah dan motivasi), yaitu mendefinisikan masalah penelitian secara spesifik dan memotivasi pentingnya solusi yang akan dibangun.
2. Definition of the objectives for a solution (penetapan tujuan solusi), yaitu menyimpulkan tujuan dari solusi yang akan dikembangkan berdasarkan definisi masalah yang telah ditetapkan.
3. Design and development (desain dan pengembangan), yaitu meran-



Gambar 2.1: Design Science Research

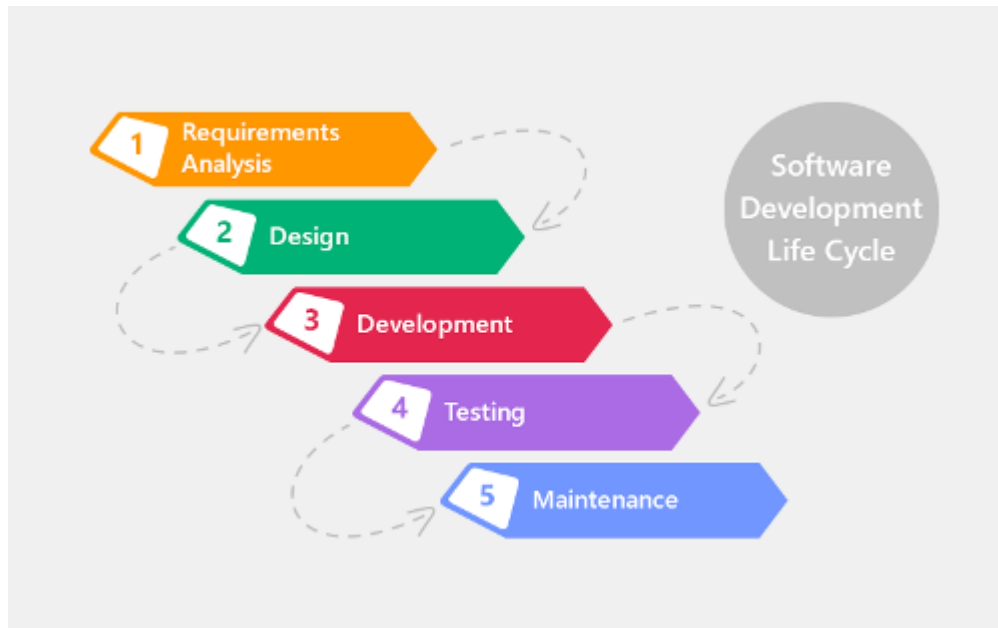
cang dan membangun artefak solusi yang dimaksud, termasuk menentukan fungsionalitas, arsitektur, dan komponen-komponen yang diperlukan.

4. *Demonstration* (demonstrasi), yaitu mendemonstrasikan penggunaan artefak untuk menyelesaikan permasalahan yang telah didefinisikan melalui eksperimen, simulasi, studi kasus, atau cara lain yang sesuai.
5. *Evaluation* (evaluasi), yaitu menilai seberapa baik artefak yang dikembangkan mampu mendukung penyelesaian masalah dibandingkan dengan tujuan yang telah ditetapkan sebelumnya.
6. *Communication* (komunikasi), yaitu mengomunikasikan masalah dan nilai penting artefak kepada komunitas peneliti dan praktisi.

2.2 Metode Pengembangan Waterfall

Metode Waterfall atau Waterfall Model adalah salah satu model pengembangan perangkat lunak yang paling klasik dan paling lama digunakan dalam rekayasa perangkat lunak. Model ini pertama kali diperkenalkan oleh Winston Royce pada tahun 1970 sebagai sebuah model sederhana yang menggambarkan alur pengembangan sistem secara linear keluaran dari tahapan sebelumnya menjadi masukan bagi tahapan berikutnya, sehingga menyerupai aliran air terjun yang mengalir ke bawah secara berurutan (Khan dkk., 2024). Metode Waterfall adalah proses pengembangan perangkat lunak yang bersifat sekuensial, di mana kemajuan pengembangan dipandang terus mengalir ke bawah melalui tahapan-tahapan perencanaan, pemodelan, implementasi, dan pengujian secara berurutan (Tjahjanto dkk., 2022).

Model Waterfall sangat cocok diterapkan pada proyek pengembangan sistem yang kebutuhan-kebutuhannya dapat diidentifikasi secara lengkap sejak awal, memiliki spesifikasi yang jelas dan tidak berubah-ubah, serta bertujuan membangun sistem dari awal dengan mengumpulkan seluruh kebutuhan sistem terlebih dahulu sebelum proses pembangunan dimulai (Tjahjanto dkk., 2022). Keunggulan utama model ini terletak pada kesederhanaan dan kemudahannya untuk dipahami, tahapannya yang terdefinisi dengan jelas, kemudahannya dalam mengatur tugas-tugas pengembangan, serta dokumentasi



Gambar 2.2: Struktur Metode Pengembangan Waterfall

proses dan hasil yang tersusun dengan baik (Khan dkk., 2024). Tahapan-tahapan dalam metode Waterfall adalah sebagai berikut:

1. Requirements Definition (Definisi Kebutuhan), yaitu tahapan untuk menganalisis dan mendokumentasikan seluruh kebutuhan pengguna terhadap sistem yang akan dibangun. Pada tahapan ini, pengembang berkomunikasi langsung dengan pengguna untuk memahami fungsi-fungsi, antarmuka, dan spesifikasi yang diharapkan dari sistem. Hasil dari tahapan ini adalah dokumen spesifikasi kebutuhan yang menjadi acuan bagi seluruh tahapan pengembangan berikutnya.
2. System and Software Design (Desain Sistem dan Perangkat Lunak), yaitu tahapan untuk menerjemahkan kebutuhan yang telah didefinisikan sebelumnya menjadi sebuah rancangan sistem yang dapat diimplementasikan. Proses desain mencakup perancangan arsitektur sistem, struktur data, antarmuka pengguna (user interface), serta rincian algoritma dan prosedur. Hasil dari tahapan ini adalah dokumen desain sistem yang menjadi panduan bagi para pengembang dalam menulis kode.
3. Implementation and Unit Testing (Implementasi dan Pengujian Unit), yaitu tahapan di mana rancangan yang telah dibuat pada tahapan sebelumnya diimplementasikan ke dalam kode program. Setiap unit atau

modul yang selesai dikembangkan kemudian diuji secara individual untuk memastikan fungsinya bekerja sesuai dengan yang diharapkan.

4. Integration and System Testing (Integrasi dan Pengujian Sistem), yaitu tahapan pengujian terhadap seluruh fitur dan fungsi sistem secara menyeluruh setelah semua modul diintegrasikan menjadi satu kesatuan. Tahapan ini bertujuan untuk memastikan bahwa sistem secara keseluruhan berjalan dengan benar sesuai spesifikasi dan memenuhi kebutuhan pengguna yang telah ditetapkan.
5. Operation and Maintenance (Operasi dan Pemeliharaan), yaitu tahapan akhir di mana sistem yang telah lulus pengujian dioperasikan oleh pengguna di lingkungan produksi yang sesungguhnya. Pemeliharaan dilakukan untuk memperbaiki bug yang teridentifikasi pasca-implementasi, melakukan penyesuaian fitur apabila diperlukan, serta memperbarui sistem agar tetap relevan dengan kebutuhan pengguna yang berkembang seiring waktu (Khan dkk., 2024).

2.3 Privacy by Design

Privacy by Design adalah pendekatan perancangan sistem yang menempatkan perlindungan data pribadi sebagai bagian bawaan (default) sejak fase konseptual, bukan sebagai fitur tambahan yang disisipkan belakangan. Kerentanan fundamental pada sistem yang mengelola data pelanggan umumnya bermuara pada dua hal, yaitu kelemahan enkripsi di sisi backend dan manipulasi antarmuka akuisisi persetujuan (dark patterns) di sisi frontend; untuk memitigasi hal ini, pengembang sistem perlu mengadopsi arsitektur Privacy by Design melalui mekanisme pengaburan data (data masking), pemisahan basis data, dan perancangan antarmuka opt-in yang transparan (Nurmiati & Kusuma, 2026). Sejalan dengan hal tersebut, perancangan pilihan privasi (privacy choice) yang bermakna bagi pengguna memerlukan kerangka konseptual yang memperlakukan persetujuan sebagai sebuah proses yang berpusat pada pengguna (user-centered process), bukan sekadar formalitas administratif (Feng, Yao, & Sadeh, 2021).

Dari sisi regulasi, Undang-Undang Nomor 27 Tahun 2022 tentang Pelindungan Data Pribadi mensyaratkan bahwa pemrosesan data pribadi berda-

sarkan persetujuan harus dilakukan melalui persetujuan yang eksplisit (Pasal 20 ayat 2 huruf a), disampaikan secara tertulis atau terekam, baik secara elektronik maupun nonelektronik (Pasal 22 ayat 1–2). Undang-undang ini juga mewajibkan pengendali data pribadi untuk melindungi keamanan dan menjaga kerahasiaan data pribadi yang diprosesnya (Pasal 35–36). Ketentuan yang sejalan juga terdapat dalam General Data Protection Regulation (GDPR) Uni Eropa, di mana pengendali data harus mampu membuktikan bahwa subjek data telah memberikan persetujuannya (Article 7), serta menerapkan pseudonimisasi dan enkripsi data pribadi sebagai salah satu langkah teknis keamanan yang tepat (Article 32).

Berdasarkan landasan tersebut, implementasi Privacy by Design pada platform ini diwujudkan melalui dua mekanisme utama: pertama, persetujuan granular yang memisahkan izin penyimpanan data (consent to store) dari izin pemrosesan data (consent to process), disertai bukti persetujuan tertulis dalam bentuk dokumen yang diunggah oleh pengguna sebagai pemenuhan syarat persetujuan terekam sebagaimana diamanatkan UU PDP; kedua, enkripsi data sensitif seperti Nomor Induk Kependudukan (NIK) menggunakan algoritma kriptografi simetris sebelum disimpan ke dalam basis data, sehingga data tetap terlindungi meskipun terjadi akses yang tidak sah terhadap basis data.

2.4 Aplikasi Berbasis Website

Aplikasi berbasis web atau web application merupakan perangkat lunak yang dapat diakses melalui peramban internet tanpa memerlukan proses instalasi pada perangkat pengguna. Aplikasi web memungkinkan pengguna menjalankan berbagai fungsi yang kompleks langsung melalui peramban dan dapat diakses dari mana saja melalui koneksi internet (Aghili dkk, 2024). Berbeda dengan aplikasi desktop konvensional yang mengharuskan pengguna mengunduh dan memasang program secara lokal, aplikasi web cukup dibuka melalui alamat URL sehingga lebih mudah diakses dari berbagai platform maupun sistem operasi yang berbeda. Karakteristik ini menjadikan aplikasi web sebagai sarana yang efektif untuk menyebarkan hasil penelitian dan menyajikan keluaran ilmiah dengan cara yang lebih mudah dipahami (Saia dkk, 2022).

Secara arsitektur, aplikasi web umumnya dibangun menggunakan model client-server, yaitu sebuah model perangkat lunak di mana klien meminta sumber daya melalui jaringan, kemudian server menerima permintaan tersebut, memprosesnya, dan memberikan respons yang sesuai (Nyabuto, 2024). Pada model ini, banyak pengguna dapat secara bersamaan mengakses dan menggunakan sumber daya yang sama. Komunikasi antara klien dan server dilakukan melalui protokol standar seperti HTTP/HTTPS yang bekerja secara request-response. Pada perkembangannya, aplikasi web modern umumnya mengadopsi arsitektur tiga lapis atau dikenal dengan three-tier architecture yang membagi sistem menjadi lapisan presentasi atau presentation layer, lapisan logika aplikasi atau application layer, dan lapisan data atau data layer. Pemisahan tanggung jawab ini membuat sistem lebih terstruktur, mudah dikembangkan, serta lebih mudah dipelihara seiring bertambahnya skala aplikasi.

Keunggulan utama dari pendekatan berbasis web adalah aksesibilitas dan kemudahan pembaruan sistem. Pembaruan fitur atau perbaikan sistem cukup dilakukan satu kali di sisi server dan secara otomatis berlaku bagi seluruh pengguna, tanpa perlu mendistribusikan ulang instalasi ke setiap perangkat. Karakteristik inilah yang menjadikan aplikasi web sebagai medium yang tepat untuk platform AutoML seperti yang dikembangkan dalam penelitian ini, di mana pengguna dari berbagai latar belakang teknis dapat mengakses layanan pelatihan model machine learning hanya melalui peramban, tanpa perlu mengkonfigurasi lingkungan pemrograman apa pun di perangkat mereka.

2.5 Python

Python adalah bahasa pemrograman komputer bertujuan umum yang bersifat high-level, interpreted, berorientasi objek, serta mendukung banyak paradigma pemrograman sekaligus, termasuk pemrograman prosedural, fungsional, dan berorientasi objek. Python pertama kali dikembangkan oleh Guido van Rossum dan dirilis secara publik pada tahun 1991 dengan filosofi desain yang mengutamakan keterbacaan kode dan kesederhanaan sintaks, sehingga kode yang ditulis dalam Python cenderung lebih ringkas dan mudah dipahami dibandingkan bahasa pemrograman lain seperti C++

atau Java. Python merupakan salah satu bahasa pemrograman yang paling banyak digunakan saat ini dan telah menjadi bahasa utama untuk pengembangan aplikasi dalam domain data science, komputasi ilmiah, analitik data, dan machine learning (Castro dkk, 2023).

Salah satu keunggulan terbesar Python terletak pada ekosistem pustakanya yang sangat kaya dan terus berkembang secara aktif. Python menawarkan berbagai pustaka dan framework yang kuat, seperti NumPy, Pandas, dan Matplotlib, yang memudahkan pengolahan, manipulasi, analisis, dan visualisasi data secara efisien, sekaligus mendukung pemodelan statistik dan machine learning yang dapat digunakan untuk menganalisis hubungan antar variabel dalam konteks penelitian ilmiah. Sebagai contoh, Python beserta pustaka-pustaka tersebut telah diterapkan dalam penelitian eksplorasi hubungan antara kualitas kopi dan lokasi pertanian menggunakan metode exploratory data analysis atau EDA (Lo dkk., 2023). Selain itu, Python juga banyak digunakan untuk membangun berbagai jenis perangkat lunak yang beragam, mulai dari situs web, aplikasi desktop, hingga sistem otomatisasi.

2.5.1 Karakteristik dan Keunggulan Python

Python memiliki sejumlah karakteristik fundamental yang membedakannya dari bahasa pemrograman lain dan menjadikannya pilihan utama di dunia sains data dan machine learning. Pertama, Python bersifat interpreted, artinya kode Python dieksekusi baris per baris secara langsung oleh interpreter tanpa perlu melalui proses kompilasi terlebih dahulu seperti pada bahasa C atau Java. Hal ini mempercepat siklus pengembangan karena developer dapat langsung melihat hasil eksekusi kode secara real-time. Kedua, Python bersifat dinamis (dynamically typed), artinya tipe data dari suatu variabel tidak perlu dideklarasikan secara eksplisit dan ditentukan secara otomatis oleh interpreter pada saat program berjalan, sehingga penulisan kode menjadi lebih fleksibel dan lebih cepat.

Ketiga, Python memiliki sintaks yang sederhana dan mudah dibaca, yang memungkinkan pengembang untuk lebih fokus pada logika pemecahan masalah daripada terjebak dalam kompleksitas sintaks bahasa pemrograman itu sendiri. Keempat, Python didukung oleh komunitas yang sangat besar dan aktif di seluruh dunia, yang secara konsisten menghasilkan ribuan pustaka open-source yang dapat dimanfaatkan secara bebas. Python telah menja-

di bahasa utama untuk pengembangan aplikasi dalam domain data science dan machine learning, meskipun para ilmuwan data tidak selalu merupakan pemrogram berpengalaman, Python tetap memungkinkan mereka untuk mengimplementasikan algoritma-algoritma kompleks dengan cepat dan efisien (Castro, dkk., 2023).

2.5.2 Python dalam Machine Learning dan Data Science

Python telah menjadi lingua franca atau bahasa pemersatu dalam dunia machine learning dan data science. Hal ini tidak terlepas dari tersedianya ekosistem pustaka ilmiah yang sangat lengkap dan terintegrasi dengan baik satu sama lain. Tiga pustaka yang paling mendasar dan paling banyak digunakan adalah NumPy untuk komputasi numerik berbasis array, Pandas untuk manipulasi dan analisis data tabular, serta Scikit-learn untuk implementasi algoritma machine learning. Ketiga pustaka ini secara konsisten menjadi rujukan utama dalam praktik statistik dan komputasi numerik untuk data science dan machine learning.

Selain ketiga pustaka tersebut, ekosistem Python untuk machine learning juga mencakup pustaka-pustaka deep learning seperti TensorFlow dan PyTorch untuk membangun jaringan saraf tiruan berskala besar, Matplotlib dan Seaborn untuk visualisasi data yang informatif, serta SciPy untuk berbagai keperluan komputasi ilmiah tingkat lanjut. Keberadaan ekosistem yang saling terintegrasi ini menjadikan Python sebagai platform yang mandiri dan lengkap untuk menyelesaikan seluruh siklus hidup sebuah proyek machine learning, mulai dari eksplorasi data awal, pra-pemrosesan, pelatihan model, evaluasi, hingga penyajian hasil secara visual. Pemodelan machine learning menggunakan berbagai teknik dalam algoritma klasifikasi dengan framework yang dimiliki oleh bahasa pemrograman Python telah terbukti mampu menghasilkan model prediktif yang akurat dalam berbagai domain aplikasi dunia nyata (Wardhana dkk., 2023).

2.6 FastAPI

FastAPI adalah framework web Python modern berperforma tinggi yang dirancang khusus untuk membangun antarmuka pemrograman aplikasi berbasis HTTP secara cepat, efisien, dan mudah. FastAPI merupakan framework

web modern yang berkinerja tinggi untuk membangun API dengan Python 3.7 ke atas, yang didasarkan pada type hints Python standar, dengan kinerja yang sangat tinggi dan setara dengan NodeJS dan Go, menjadikannya salah satu framework Python tercepat yang tersedia saat ini (FastAPI, 2024). FastAPI pertama kali dikembangkan oleh Sebastián Ramírez dan dirilis secara publik pada Desember 2018. Meskipun tergolong framework yang relatif baru, FastAPI mengalami pertumbuhan adopsi yang sangat pesat dan pada tahun 2024 telah melampaui Flask dalam hal popularitas di kalangan pengembang Python, dengan jumlah bintang di GitHub mencapai lebih dari 78.000. FastAPI dibangun di atas dua fondasi teknologi utama, yaitu Starlette sebagai toolkit ASGI atau Asynchronous Server Gateway Interface yang menangani seluruh aspek komunikasi web, dan Pydantic sebagai pustaka validasi data yang memanfaatkan anotasi tipe Python untuk memvalidasi, menserialisasi, dan mendokumentasikan data secara otomatis (FastAPI, 2024).

2.6.1 Arsitektur Asinkron ASGI dan WSGI

Salah satu keunggulan paling signifikan dari FastAPI dibandingkan framework Python tradisional adalah dukungannya yang penuh terhadap pemrograman asinkron melalui standar ASGI atau Asynchronous Server Gateway Interface. Terdapat dua standar antarmuka server web yang umum digunakan dalam ekosistem Python yang perlu dipahami perbedaannya secara mendasar.

WSGI atau Web Server Gateway Interface adalah standar lama yang digunakan oleh Flask dan Django versi konvensional. WSGI bersifat sinkronus, artinya setiap permintaan yang masuk harus diselesaikan sepenuhnya sebelum server dapat menangani permintaan berikutnya. Flask dirancang untuk web secara general purpose, sedangkan FastAPI dirancang khusus sebagai web service (Azhari, 2022). Model satu-thread-per-request yang dimiliki WSGI menjadikannya tidak efisien dalam menangani beban kerja yang bersifat I/O-bound, yaitu operasi yang banyak melibatkan waktu tunggu seperti akses basis data atau pemrosesan data dalam jumlah besar.

ASGI sebagai penerus WSGI mendukung pemrosesan request secara asinkron sejati, di mana server tidak perlu menunggu satu operasi selesai sebelum memulai operasi berikutnya. Dengan fondasinya pada Starlette, FastAPI mewarisi performa yang sangat tinggi, termasuk dukungan Web-

Socket, background tasks dalam proses, serta streaming response, sehingga menjadikannya ideal untuk aplikasi berperforma tinggi dan penanganan data secara real-time dibandingkan Flask yang lebih cocok untuk solusi yang lebih sederhana dan fleksibel (FastAPI, 2024). Dengan memanfaatkan syntax `async/await` yang disediakan Python 3.7+, FastAPI mampu menangani ribuan koneksi secara bersamaan dalam satu thread tunggal, menghasilkan throughput yang sangat tinggi dengan konsumsi sumber daya yang efisien.

2.6.2 Fitur -Fitur Utama FastAPI

FastAPI memiliki sejumlah fitur unggulan yang membedakannya dari framework web Python lainnya dan menjadikannya pilihan yang sangat produktif bagi para pengembang.

Pertama, FastAPI memanfaatkan type hints Python secara ekstensif. Dengan mendeklarasikan tipe data dari parameter fungsi, FastAPI secara otomatis melakukan validasi data masukan, konversi tipe, dan serialisasi data keluaran tanpa kode tambahan. Jika klien mengirimkan data dengan tipe yang tidak sesuai, FastAPI secara otomatis mengembalikan respons error yang informatif dengan kode status HTTP yang tepat.

Kedua, FastAPI menghasilkan dokumentasi API interaktif secara otomatis. Berdasarkan deklarasi type hints dan model Pydantic yang digunakan dalam kode, FastAPI secara otomatis menghasilkan dua versi dokumentasi interaktif yang dapat langsung diakses melalui peramban, yaitu Swagger UI yang tersedia di endpoint `/docs` dan ReDoc yang tersedia di endpoint `/redoc`. Kedua dokumentasi ini sepenuhnya mengikuti standar terbuka OpenAPI dan JSON Schema, sehingga dapat digunakan oleh klien dari berbagai bahasa pemrograman, termasuk untuk pembuatan kode klien atau client code generation secara otomatis. FastAPI merupakan framework yang memanfaatkan sepenuhnya type hints dan pemrograman asinkron Python, menyediakan validasi data otomatis, parsing request, dan pembuatan dokumentasi melalui standar OpenAPI (FastAPI, 2024).

Ketiga, FastAPI memiliki sistem dependency injection yang elegan dan mudah digunakan. Sistem ini memungkinkan pengembang untuk mendeklarasikan dependensi seperti koneksi basis data, autentikasi pengguna, atau konfigurasi aplikasi secara deklaratif, yang kemudian disuntikkan secara otomatis ke dalam fungsi handler yang membutuhkannya tanpa perlu dipanggil

secara manual di setiap fungsi, bahkan mendukung dependensi berjenjang yang membentuk sebuah hierarki (FastAPI, 2024).

2.6.3 Validasi data dengan Pydantic

Pydantic adalah pustaka Python yang menjadi fondasi utama sistem validasi data dalam FastAPI. Pydantic memungkinkan pengembang untuk mendefinisikan model data sebagai kelas Python biasa menggunakan anotasi tipe, yang kemudian secara otomatis berfungsi sebagai skema validasi yang ketat. Ketika sebuah request masuk ke endpoint FastAPI, data dalam request body secara otomatis divalidasi, dikonversi ke tipe yang sesuai, dan dideserialisasi menjadi objek Pydantic yang type-safe sebelum diteruskan ke fungsi handler. Pydantic mendukung berbagai jenis validasi, mulai dari validasi tipe data dasar seperti string, integer, float, dan boolean, hingga validasi yang lebih kompleks seperti validasi format email, URL, rentang nilai numerik, panjang string, serta validasi kondisional yang bergantung pada nilai field lainnya. Kemampuan validasi yang komprehensif ini memastikan bahwa API yang dibangun dengan FastAPI selalu menerima dan memproses data yang bersih dan sesuai format, sehingga meminimalkan risiko error yang disebabkan oleh data yang tidak valid.

2.6.4 FastAPI Sebagai Jembatan Machine Learning dan Frontend

Dalam arsitektur sistem machine learning berbasis web, FastAPI memainkan peran strategis sebagai lapisan jembatan yang menghubungkan model machine learning yang berjalan di backend dengan antarmuka pengguna yang berjalan di frontend. Model Scikit-learn yang telah dilatih dapat disimpan dalam format pickle atau joblib, kemudian dimuat kembali oleh aplikasi FastAPI saat server dijalankan. Ketika pengguna mengunggah dataset melalui antarmuka web, request tersebut diterima oleh endpoint FastAPI yang kemudian meneruskannya ke fungsi pemrosesan dan pelatihan model. Penelitian yang mengintegrasikan model machine learning ke dalam aplikasi berbasis web melalui API yang dibuat menggunakan framework FastAPI telah membuktikan bahwa pendekatan ini menghasilkan sistem yang dapat diakses secara luas dengan tingkat keberhasilan pengujian yang tinggi (Ali-

mi dkk, 2024). FastAPI juga mendukung endpoint untuk pengunggahan file melalui tipe data UploadFile, yang memungkinkan pengguna mengirimkan file CSV langsung ke server untuk diproses oleh pipeline machine learning secara otomatis. Seluruh proses ini dapat dieksekusi secara asinkron sehingga server tetap responsif terhadap request lain yang datang secara bersamaan.

2.7 ReactJS

React.js adalah pustaka atau library JavaScript open-source yang dikembangkan oleh Facebook dan dirilis secara publik pada tahun 2013, yang dirancang khusus untuk membangun antarmuka pengguna secara interaktif, dinamis, dan responsif pada aplikasi web. React.js merupakan library JavaScript berbasis komponen yang tergolong sebagai salah satu library frontend tercepat dan termudah digunakan dalam pengembangan aplikasi web (Maratkar & Adkar, 2021). Berbeda dengan framework web konvensional yang merender ulang seluruh halaman setiap kali terjadi perubahan data, React.js memanfaatkan konsep Virtual DOM sehingga hanya memperbarui bagian-bagian antarmuka yang benar-benar berubah, tanpa harus memuat ulang seluruh halaman, sehingga menghasilkan pengalaman pengguna yang jauh lebih mulus dan responsif. React.js tidak bersifat framework penuh, melainkan sebuah pustaka yang berfokus secara spesifik pada lapisan tampilan atau view layer, sehingga mudah dikombinasikan dengan pustaka atau framework lain sesuai kebutuhan pengembang.

Sejak pertama kali dirilis, React.js mengalami pertumbuhan adopsi yang sangat pesat di kalangan pengembang frontend di seluruh dunia. Berdasarkan survei Stack Overflow Developer Survey, React.js secara konsisten menempati posisi sebagai framework atau pustaka web yang paling banyak digunakan selama beberapa tahun berturut-turut. Penerapan React.js dalam pengembangan antarmuka frontend sebuah aplikasi telah terbukti mampu menghasilkan tampilan yang responsif dan efisien, di mana pengembang dapat memecah tampilan antarmuka menjadi beberapa komponen yang berbeda-beda dan dapat digunakan kembali (reusable) (Juan & Budi, 2023). Popularitas ini didorong oleh filosofi desain React.js yang sederhana namun sangat kuat, ekosistem pustaka pendukung yang kaya, serta komunitas pengembang yang aktif dan terus berkontribusi.

2.7.1 Arsitektur Berbasis Komponen

Konsep paling fundamental dalam React.js adalah arsitektur berbasis komponen. Dalam paradigma ini, antarmuka pengguna dibangun dari sekumpulan unit-unit mandiri yang disebut komponen, di mana setiap komponen merupakan potongan kode yang dapat berdiri sendiri, memiliki logikanya sendiri, dan bertanggung jawab untuk merender bagian tertentu dari antarmuka pengguna. React.js menggunakan komponen yang dapat digunakan kembali dan dapat diproduksi secara terpisah menjadi beberapa komponen, yang membantu pengembang mengetahui letak kesalahan dengan lebih mudah sehingga proses debugging menjadi lebih cepat (Jonathan & Supriyadi, 2023). Setiap komponen dalam React.js dapat diibaratkan sebagai sebuah blok bangunan yang dapat disusun, digabungkan, dan digunakan kembali secara bebas untuk membangun antarmuka yang kompleks dari bagian-bagian yang lebih sederhana.

Komponen React.js secara umum terbagi menjadi dua jenis berdasarkan cara mendefinisikannya. Pertama adalah komponen berbasis kelas, yaitu komponen yang didefinisikan menggunakan sintaks kelas JavaScript ES6 dan mewarisi kelas `React.Component`. Kedua adalah komponen fungsional, yaitu komponen yang didefinisikan sebagai fungsi JavaScript biasa yang menerima props sebagai argumen dan mengembalikan elemen React sebagai keluaran. Sejak diperkenalkannya React Hooks pada versi 16.8, komponen fungsional menjadi pendekatan yang paling direkomendasikan karena lebih ringkas, lebih mudah diuji, dan mendukung manajemen state serta efek samping secara langsung tanpa perlu menggunakan kelas.

2.7.2 Virtual DOM

Virtual DOM atau Virtual Document Object Model adalah salah satu inovasi teknis paling penting yang diperkenalkan oleh React.js dan menjadi fondasi dari performanya yang tinggi. Untuk memahami Virtual DOM, perlu dipahami terlebih dahulu cara kerja DOM yang sesungguhnya. DOM adalah representasi terstruktur dari dokumen HTML dalam bentuk pohon yang dapat dimanipulasi oleh JavaScript. Setiap kali terjadi perubahan pada DOM, peramban harus menghitung ulang tata letak dan merender ulang elemen-elemen yang terdampak, yang merupakan operasi yang sangat mahal secara

komputasi apabila dilakukan terlalu sering.

Virtual DOM adalah representasi ringan dari DOM yang sesungguhnya yang disimpan sepenuhnya di dalam memori JavaScript. Setiap kali terjadi perubahan pada state atau props sebuah komponen, React.js terlebih dahulu memperbarui Virtual DOM, bukan DOM yang sesungguhnya. Setelah itu, React.js akan melakukan perbandingan antara Virtual DOM yang baru dengan Virtual DOM yang lama melalui proses yang disebut reconciliation, dan jika terdapat perbedaan, maka React.js akan melakukan proses rendering ulang hanya pada bagian-bagian yang benar-benar berubah. Pendekatan ini secara dramatis mengurangi jumlah operasi DOM yang mahal, sehingga menghasilkan antarmuka pengguna yang terasa sangat cepat dan responsif meskipun data yang ditampilkan berubah secara terus-menerus.

2.7.3 State, Props, dan React Hooks

Dua mekanisme utama yang digunakan React.js untuk mengelola dan mengalirkan data di dalam aplikasi adalah state dan props. Props adalah mekanisme untuk mengalirkan data dari komponen induk (parent component) ke komponen anak. Props bersifat read-only, artinya komponen yang menerima props tidak diperbolehkan untuk mengubah nilai props tersebut secara langsung, sedangkan state adalah sebuah objek data internal komponen yang bersifat read-write, artinya pengembang dapat mengubah nilainya sesuai kebutuhan.

State adalah mekanisme untuk menyimpan data internal sebuah komponen yang dapat berubah seiring waktu. Ketika nilai state berubah, React.js secara otomatis merender ulang komponen tersebut beserta seluruh komponen turunannya yang bergantung pada state tersebut. Pada komponen fungsional modern, pengelolaan state dilakukan menggunakan React Hooks, yaitu serangkaian fungsi khusus yang memungkinkan komponen fungsional untuk menggunakan fitur-fitur React yang sebelumnya hanya tersedia pada komponen berbasis kelas. Hook yang paling mendasar adalah useState untuk mengelola state lokal komponen, useEffect untuk menangani efek samping seperti pemanggilan API atau langganan data, useContext untuk mengakses context global, serta useReducer untuk mengelola state yang kompleks dengan pola yang menyerupai Redux.

2.7.4 Single Page Application

React.js adalah teknologi utama yang digunakan untuk membangun SPA atau Single Page Application, yaitu jenis aplikasi web yang hanya memuat satu dokumen HTML tunggal dan secara dinamis memperbarui konten halaman tersebut melalui JavaScript tanpa perlu melakukan navigasi ke halaman baru. Penggunaan metode SPA pada pengembangan aplikasi web terbukti dapat menawarkan kemudahan bagi pengembang serta menghemat waktu dalam proses pengembangan, sebagaimana ditunjukkan dalam penelitian implementasi React.js pada pengembangan aplikasi web (Jonathan & Supriyadi, 2023). Pendekatan SPA menghasilkan pengalaman pengguna yang jauh lebih mulus dan menyerupai pengalaman menggunakan aplikasi desktop native, karena transisi antar halaman atau bagian aplikasi terjadi secara instan tanpa jeda pemuatan halaman yang terasa oleh pengguna.

Untuk mendukung navigasi antar halaman dalam arsitektur SPA, React.js menggunakan pustaka React Router, yaitu library routing standar untuk ekosistem React yang memungkinkan pengembang mendefinisikan rute-rute URL dan menghubungkannya dengan komponen yang sesuai. React Router memungkinkan navigasi tampilan di antara berbagai komponen di React, memungkinkan perubahan pada URL, serta menjaga antarmuka pengguna tetap sinkron terhadap URL secara real-time (Juan & Budi, 2023). Dengan React Router, pengguna tetap dapat menggunakan tombol back dan forward pada peramban, mem-bookmark halaman tertentu, dan berbagi tautan langsung ke bagian tertentu aplikasi, seolah-olah mereka sedang menjelajahi aplikasi multi-halaman konvensional.

2.8 RestAPI

REST API atau Representational State Transfer Application Programming Interface adalah sebuah antarmuka pemrograman aplikasi yang mengikuti prinsip-prinsip arsitektur REST dalam komunikasi antar sistem melalui jaringan internet. Konsep REST pertama kali diperkenalkan oleh Roy Fielding dalam disertasi doktoralnya pada tahun 2000 di University of California, Irvine, sebagai sebuah gaya arsitektur untuk membangun sistem terdistribusi yang bersifat hypermedia. Dalam gaya arsitektur REST, data dan fungsionalitas dianggap sebagai sumber daya yang dapat diakses dan dimanipulasi

menggunakan seperangkat aturan yang seragam dan terdefinisi dengan baik. REST API telah menjadi standar *de facto* dalam pengembangan layanan web modern dan digunakan secara masif oleh hampir seluruh perusahaan teknologi besar di dunia untuk mengekspos layanan mereka kepada pengembang eksternal.

RESTful API merupakan gaya arsitektur web service yang memanfaatkan protokol HTTP dan format data seperti JSON atau XML, di mana JSON sendiri merupakan standar pertukaran data pada web yang mendukung struktur data seperti object dan arrays sehingga sangat mudah untuk menulis maupun membaca datanya (Muharom., 2025). Keunggulan utama REST API dibandingkan pendekatan web service lama seperti SOAP atau Simple Object Access Protocol terletak pada kesederhanaannya: REST lebih lugas, lebih fleksibel, lebih mudah dipelajari dengan kurva pembelajaran yang lebih kecil, serta lebih efisien karena tidak menggunakan format XML yang berat dalam pengiriman data. Karakteristik-karakteristik inilah yang menjadikan REST API sebagai pilihan utama para pengembang modern dalam membangun komunikasi antar komponen sistem yang terdistribusi.

2.9 Autentikasi dan Otorisasi

Otentikasi dan otorisasi merupakan dua aspek keamanan fundamental yang wajib diimplementasikan pada setiap sistem berbasis API, khususnya yang mengelola data pengguna yang bersifat sensitif. Otentikasi dan otorisasi pengguna merupakan aspek penting dalam setiap aplikasi yang mengelola data pengguna yang sensitif, dan JSON Web Token (JWT) telah menjadi metode otentikasi yang umum digunakan untuk mengamankan data yang dikirim melalui jaringan (Nashikhuddin dkk., 2023). Dalam konteks pengembangan backend pada platform AutoML yang dibangun dalam penelitian ini, mekanisme otentikasi dan otorisasi berperan penting untuk memastikan bahwa hanya pengguna yang telah terverifikasi identitasnya yang dapat mengakses fitur-fitur seperti pembuatan experiment, pelatihan model, maupun pengunduhan hasil model.

2.9.1 Konsep Otentikasi dan Otorisasi

Meskipun sering disebut bersamaan, otentikasi dan otorisasi merupakan dua proses yang berbeda secara konseptual. Otentikasi adalah proses verifikasi identitas pengguna, yaitu memastikan bahwa pengguna benar-benar orang yang ia klaim, umumnya dilakukan melalui kombinasi username dan password, atau mekanisme lain seperti token digital. Otorisasi, di sisi lain, adalah proses penentuan hak akses, yaitu memastikan bahwa pengguna yang telah terotentikasi memiliki izin untuk melakukan tindakan atau mengakses sumber daya tertentu. Dengan kata lain, otentikasi menjawab pertanyaan "siapa Anda?", sedangkan otorisasi menjawab pertanyaan "apa yang boleh Anda lakukan?". Kedua proses ini biasanya diimplementasikan secara berurutan: sistem terlebih dahulu mengotentikasi pengguna, kemudian memeriksa otorisasinya sebelum memberikan akses ke sumber daya yang diminta.

2.9.2 JSON Web Token (JWT)

JSON Web Token (JWT) adalah standar terbuka (RFC 7519) yang mendefinisikan format token ringkas dan mandiri (self-contained) untuk menyampaikan informasi secara aman antara dua pihak dalam bentuk objek JSON. Sebuah JWT terdiri dari tiga bagian yang dipisahkan oleh tanda titik, yaitu header yang berisi jenis token dan algoritma hashing yang digunakan, payload yang berisi klaim seperti identitas pengguna dan waktu kedaluwarsa token, serta signature yang digunakan untuk memverifikasi keaslian token dan memastikan bahwa isi token tidak berubah sejak diterbitkan.

Implementasi JSON Web Token (JWT) dalam API RESTful terbukti menjadi metode yang efektif untuk otentikasi dan otorisasi pengguna, karena penggunaannya memastikan bahwa hanya pengguna yang telah terotentikasi yang mendapatkan akses ke data sensitif). JWT berperan sebagai mekanisme untuk mengenkripsi dan menyampaikan informasi otentikasi secara aman antara pengguna dan server, tanpa server perlu menyimpan status sesi pengguna secara eksplisit. Dari sisi performa, hasil pengujian menunjukkan bahwa proses otentikasi menggunakan JWT dapat berlangsung dengan cepat, dengan rata-rata waktu respons otentikasi sebesar 101,792 milidetik dari 30 kali percobaan (Nashikhuddin dkk., 2023).

2.9.3 Perbandingan JWT dengan Session-Based Authentication

Selain JWT, terdapat pendekatan otentikasi lain yang umum digunakan, yaitu session-based authentication, di mana status otentikasi pengguna disimpan di sisi server (bersifat stateful), berbeda dengan JWT yang bersifat stateless karena seluruh informasi otentikasi dibawa langsung di dalam token. Hasil pengujian beban bertingkat hingga 10.000 request menunjukkan bahwa JWT memiliki performa yang lebih baik dalam hal kecepatan respons dan efisiensi sumber daya, dengan throughput yang lebih tinggi serta konsumsi CPU dan memori yang lebih rendah dibandingkan metode session (Cahyadi & Akbi, 2026). Pada beban 10.000 request, JWT mencatatkan rata-rata waktu respons 160 ms dan throughput 525 request/detik, sedangkan metode session-based mencatatkan rata-rata waktu respons 240 ms dan throughput 480 request/detik.

Meskipun unggul dari sisi performa, JWT memerlukan manajemen token yang ketat karena rentan terhadap serangan Cross-Site Scripting (XSS) apabila disimpan secara tidak aman di sisi klien. Sebaliknya, session-based authentication lebih aman terhadap XSS, namun menunjukkan performa yang lebih rendah dan konsumsi sumber daya yang lebih tinggi, serta memiliki kerentanan tersendiri terhadap serangan seperti Cross-Site Request Forgery (CSRF) dan session hijacking (Cahyadi & Akbi, 2026).

2.9.4 Keamanan dan Mitigasi Risiko JWT

Berdasarkan hasil pengujian keamanan yang dilakukan, JWT unggul dalam hal efisiensi dan skalabilitas, dengan waktu respons yang lebih cepat, throughput yang lebih tinggi, dan konsumsi sumber daya yang lebih rendah dibandingkan metode session. Namun demikian, JWT memerlukan mekanisme keamanan tambahan, seperti manajemen refresh token dan mitigasi terhadap pencurian token (token theft), agar tetap aman digunakan dalam produksi (Cahyadi & Akbi, 2026). Oleh karena itu, implementasi JWT pada backend platform AutoML dalam penelitian ini turut mempertimbangkan penggunaan mekanisme access token dengan masa berlaku (expiration time) yang singkat, disertai refresh token untuk memperbarui sesi pengguna tanpa perlu melakukan proses login ulang secara berkala.

2.10 SQLite

SQLite adalah sebuah pustaka perangkat lunak yang mengimplementasikan mesin basis data relasional yang bersifat self-contained, serverless, zero-configuration, dan transactional, yang berbeda dari sistem manajemen basis data relasional pada umumnya karena memiliki kode pustaka yang relatif kecil, mudah digunakan tanpa membutuhkan penginstalasian dan konfigurasi yang rumit. SQLite pertama kali dikembangkan oleh D. Richard Hipp pada tahun 2000 dan dirilis di bawah lisensi public domain, yang berarti siapa pun dapat menggunakannya secara bebas untuk keperluan apa pun, baik komersial maupun nonkomersial, tanpa biaya dan tanpa batasan. Dewasa ini, SQLite dianggap sebagai mesin basis data yang paling banyak digunakan di dunia, dengan jumlah instalasi aktif yang diperkirakan mencapai miliaran, karena SQLite tertanam secara bawaan di setiap perangkat Android, iOS, macOS, Windows 10, dan hampir semua peramban web modern.

Karakteristik yang paling mendasar dan paling membedakan SQLite dari sistem manajemen basis data lainnya adalah sifatnya yang serverless dan embedded. Tidak seperti sistem basis data konvensional seperti MySQL, PostgreSQL, atau Oracle yang beroperasi sebagai proses server independen yang terpisah dari aplikasi dan memerlukan komunikasi melalui protokol jaringan, SQLite tidak memerlukan proses server yang terpisah sama sekali. Seluruh basis data SQLite, termasuk data, indeks, skema, dan metadata, disimpan dalam satu berkas tunggal di sistem berkas perangkat. SQLite merupakan embedded database yang sangat populer dan banyak digunakan untuk penyimpanan data lokal pada berbagai platform, di mana keunggulan utamanya terletak pada reliability, robustness, keamanan, kecepatan, efisiensi, dan fleksibilitasnya yang unik dibandingkan basis data embedded lainnya (Bharadwaj dkk., 2019). Berkas basis data SQLite bersifat cross-platform, artinya sebuah berkas basis data yang dibuat pada satu sistem operasi dapat dibaca dan digunakan langsung pada sistem operasi yang berbeda tanpa perlu konversi atau modifikasi apa pun.

2.11 Machine Learning

Machine learning atau pembelajaran mesin merupakan salah satu cabang paling fundamental dari kecerdasan buatan atau artificial intelligence yang saat ini berkembang pesat dan diterapkan di berbagai bidang kehidupan. Secara konseptual, machine learning merupakan studi ilmiah mengenai algoritma dan model statistik yang digunakan oleh sistem komputer untuk menyelesaikan tugas tertentu tanpa diprogram secara eksplisit (Mahesh, 2020). Machine learning juga dapat dipahami sebagai suatu pendekatan dalam kecerdasan buatan yang berguna untuk meniru sikap manusia dalam pengambilan keputusan berdasarkan pengalaman data yang dimiliki (Fahmi, 2023).

Keunggulan utama dari pendekatan machine learning terletak pada kemampuannya untuk belajar secara otomatis. Setelah sebuah algoritma berhasil mempelajari pola dari data, sistem dapat menjalankan tugasnya secara mandiri tanpa intervensi manusia lebih lanjut. Secara prinsip, sistem machine learning bekerja dengan menerima data masukan, mempelajari pola darinya, kemudian menghasilkan keluaran berdasarkan data tersebut; ketika diberikan data baru, mesin akan memberikan keluaran berdasarkan pengalaman dan pola yang telah dipelajarinya sebelumnya (Suyal & Goyal, 2022). Algoritma-algoritma machine learning ini dimanfaatkan untuk berbagai keperluan seperti data mining, pemrosesan citra atau image processing, hingga analitik prediktif atau predictive analytics, yang penerapannya telah menjadi bagian dari aktivitas penggunaan teknologi sehari-hari (Mahesh, 2020).

Secara umum, terdapat tiga paradigma utama dalam machine learning berdasarkan cara sistem belajar dari data, yaitu supervised learning, unsupervised learning, dan reinforcement learning. Ketiga paradigma ini memiliki perbedaan mendasar dalam pendekatan pembelajaran, jenis data yang dibutuhkan, serta tujuan yang ingin dicapai, dan perbedaan ini memiliki dampak yang signifikan terhadap aplikasi praktis serta hasil yang diharapkan.

2.12 Automated Machine Learning

Automated Machine Learning atau yang lebih dikenal dengan singkatan AutoML merupakan sebuah pendekatan dalam pengembangan sistem kecerdasan buatan yang bertujuan untuk mengotomatisasi seluruh atau sebagian

besar tahapan dalam proses pembangunan model machine learning, mulai dari persiapan data hingga evaluasi model akhir. AutoML hadir sebagai respons atas tantangan yang dihadapi dalam penerapan machine learning secara luas, di mana membangun aplikasi machine learning yang berkinerja tinggi selama ini membutuhkan tim ahli yang sangat terlatih, terdiri dari data scientist yang memiliki pengetahuan mendalam tentang algoritma dan statistika, serta pakar domain yang memahami karakteristik masalah secara spesifik (Zöller & Huber, 2021). Ketergantungan yang tinggi terhadap keahlian manusia ini menjadikan proses pengembangan model machine learning sebagai sebuah pekerjaan yang panjang, mahal, dan sulit untuk diulang secara konsisten.

AutoML muncul sebagai solusi atas permasalahan tersebut dengan cara mengurangi ketergantungan terhadap data scientist dan memungkinkan para pakar domain untuk membangun aplikasi machine learning secara mandiri tanpa harus menguasai statistika dan pemrograman machine learning secara mendalam (Zöller & Huber, 2021). Dengan kata lain, AutoML berupaya untuk mendemokratisasi akses terhadap teknologi machine learning, sehingga tidak hanya dapat dimanfaatkan oleh kalangan peneliti atau insinyur machine learning, tetapi juga oleh praktisi dari berbagai latar belakang profesi yang tidak memiliki keahlian teknis mendalam namun membutuhkan kemampuan analitik berbasis data dalam pekerjaan mereka sehari-hari.

2.13 Large Language Model dan Ollama

Large Language Model (LLM) adalah model bahasa berskala besar yang mampu memahami dan menghasilkan teks dalam bahasa alami untuk berbagai tugas, termasuk analisis dan sintesis teks. Pemanfaatan LLM melalui layanan cloud seperti ChatGPT, Google Gemini, atau Claude AI kerap menimbulkan kendala berupa latensi jaringan, risiko privasi data, dan biaya operasional yang berkelanjutan (Ramadhani & Nurhidayat, 2026; Riza dkk., 2025). Sebagai alternatif, Ollama hadir sebagai platform open-source yang memungkinkan LLM dijalankan secara lokal (on-premise) tanpa ketergantungan pada layanan eksternal maupun koneksi internet.

Hasil pengujian menunjukkan bahwa LLM lokal yang dijalankan melalui Ollama, seperti model Llama3.1 8B, mampu bersaing dengan model clo-

ud komersial dan unggul mutlak dalam aspek responsivitas, kendali privasi, dan efisiensi biaya, dengan biaya operasional yang dapat ditekan hingga nol rupiah dibandingkan model komersial yang memerlukan biaya langganan bulanan (Ramadhani & Nurhidayat, 2026). Dalam pengujian subjektif terhadap pengguna, aspek keamanan privasi data (privacy and data security) memperoleh penilaian tertinggi dengan rata-rata skor 4,6 dari 5,0, yang membuktikan bahwa arsitektur pemrosesan yang berjalan sepenuhnya secara lokal berhasil menjawab kekhawatiran pengguna terkait risiko kebocoran data pada platform AI berbasis cloud (Ramadhani & Nurhidayat, 2026). Sejalan dengan hal tersebut, rancangan agen AI lokal berbasis LLM yang beroperasi tanpa ketergantungan pada API eksternal terbukti tetap mampu menjaga privasi data serta cocok diterapkan pada organisasi dengan kebutuhan privasi tinggi dan sumber daya terbatas (Riza dkk., 2025). Ollama juga telah diintegrasikan pada berbagai kasus penggunaan lain, seperti sistem rekomendasi lokasi berbasis Business Intelligence, dengan hasil pengujian teknis menunjukkan akurasi rekomendasi sebesar 80%, waktu respon rata-rata 1,74 detik, dan tingkat konsistensi hasil sebesar 80% (Ahmad, Rahman, & Sutabri, 2025).

Untuk mengevaluasi kualitas keluaran teks yang dihasilkan oleh LLM dibandingkan dengan teks referensi, salah satu metrik yang umum digunakan adalah ROUGE (Recall-Oriented Understudy for Gisting Evaluation). Pada penelitian pengembangan chatbot berbasis dokumen menggunakan Ollama, hasil pengujian menggunakan metrik ROUGE-L menunjukkan skor sebesar 0,85, yang menandakan tingkat kesamaan yang tinggi antara jawaban yang dihasilkan model dengan referensi dokumen (Albert & Voutama, 2025). Skor ROUGE yang tinggi ini mengindikasikan bahwa LLM yang dijalankan secara lokal melalui Ollama tetap mampu menghasilkan keluaran teks yang relevan dan mendekati kualitas referensi, sehingga metrik ini layak digunakan sebagai salah satu tolok ukur objektif dalam menilai konsistensi hasil analisis yang dihasilkan oleh model terhadap pola penulisan pada data referensi.

3. METODE PENELITIAN

3.1 Pendekatan Penelitian

Penelitian ini menggunakan Design Science Research (DSR) sebagai metode penelitian. Pemilihan DSR sebagai metode penelitian didasarkan pada kesesuaiannya dengan tujuan penelitian ini, yaitu merancang dan membangun komponen back end dari platform AutoML sebagai solusi untuk memfasilitasi proses machine learning tanpa pemrograman yang disertai mekanisme perlindungan data pribadi. Sistem back end yang dihasilkan merupakan artefak berupa sebuah sistem yang dapat dioperasikan dan didemonstrasikan secara langsung. DSR memiliki enam tahapan yang membentuk siklus penelitian yaitu:

1. Identifikasi Masalah dan Motivasi (Problem Identification and Motivation): Pada tahap ini peneliti mengidentifikasi masalah spesifik yang menjadi dasar penelitian dan memotivasi pentingnya solusi yang akan dibangun. Dalam penelitian ini, masalah yang diidentifikasi adalah tidak tersedianya platform machine learning berbasis web yang modular, dapat diakses tanpa keahlian pemrograman mendalam, dan sekaligus memperhatikan aspek perlindungan data pribadi subjek penelitian sesuai Undang-Undang Nomor 27 Tahun 2022 tentang Pelindungan Data Pribadi.
2. Definisi Tujuan Solusi (Definition of the Objectives for a Solution): Pada tahap ini peneliti mendefinisikan tujuan dari artefak yang akan dibangun berdasarkan masalah yang telah diidentifikasi. Tujuan solusi dalam penelitian ini adalah membangun komponen back end platform AutoML yang mampu mengelola dataset, menjalankan preprocessing dan pelatihan model secara modular, mengintegrasikan analisis

teks berbasis LLM, serta menerapkan mekanisme privacy consent yang mengenkripsi data identitas subjek penelitian.

3. Perancangan dan Pengembangan (Design and Development): Pada tahap ini artefak dirancang dan dibangun. Perancangan mencakup arsitektur sistem, skema basis data, desain endpoint API, dan mekanisme keamanan. Pengembangan dilakukan menggunakan metode Waterfall yang diuraikan pada sub-bab berikutnya. Artefak yang dihasilkan adalah sistem back end berbasis FastAPI dengan delapan modul yang saling terintegrasi.
4. Demonstrasi (Demonstration): Pada tahap ini artefak yang telah dibangun didemonstrasikan kemampuannya dalam memecahkan masalah yang diidentifikasi. Demonstrasi dilakukan melalui pengoperasian sistem back end secara langsung menggunakan data uji pada setiap modul yang diimplementasikan.
5. Evaluasi (Evaluation): Pada tahap ini artefak dievaluasi seberapa baik ia memecahkan masalah. Evaluasi dilakukan melalui pengujian white box testing terhadap seluruh fungsi utama sistem untuk memverifikasi bahwa setiap modul bekerja sesuai dengan logika yang dirancang.
6. Komunikasi (Communication): Pada tahap ini hasil penelitian dikomunikasikan kepada komunitas ilmiah dan praktisi melalui penulisan laporan penelitian dalam bentuk skripsi.

3.2 Metode Pengembangan Sistem

Pengembangan komponen back end sistem AutoML Platform menggunakan metode Waterfall sebagai metode pengembangan sistem. Pemilihan metode Waterfall didasarkan pada beberapa pertimbangan. Pertama, kebutuhan sistem telah terdefinisi dengan jelas sejak awal melalui kesepakatan tim pengembang dan arahan dosen pembimbing sehingga perubahan kebutuhan di tengah pengembangan relatif minimal. Kedua, sistem dibangun oleh tim yang terdiri dari tiga anggota dengan pembagian role yang tegas (back end developer, front end developer, dan ML engineer) sehingga alur pengembangan yang linear memudahkan koordinasi antar anggota. Ketiga,

dokumentasi yang dihasilkan pada setiap tahapan Waterfall mendukung penulisan laporan penelitian secara sistematis. penelitian ini terdiri dari empat tahapan, yaitu:

1. Analisis Kebutuhan (Requirements Analysis): Tahap analisis kebutuhan merupakan tahap awal pengembangan yang bertujuan mengidentifikasi dan mendokumentasikan seluruh kebutuhan sistem secara lengkap. Pada tahap ini dilakukan identifikasi kebutuhan fungsional yaitu fitur-fitur yang harus dimiliki sistem, serta kebutuhan non-fungsional yaitu aspek kualitas sistem seperti keamanan, performa, dan kemudahan penggunaan. Kebutuhan fungsional back end yang diidentifikasi pada tahap ini mencakup: manajemen autentikasi dan otorisasi berbasis role, pengelolaan unggah dan penyimpanan tiga tipe dataset, implementasi mekanisme privacy consent sesuai UU PDP No. 27 Tahun 2022, eksekusi preprocessing dan pelatihan model secara asinkron, serta analisis teks menggunakan model bahasa besar lokal. Kebutuhan non-fungsional yang diidentifikasi mencakup: keamanan data melalui enkripsi token JWT dan enkripsi NIK, kemampuan menjalankan proses panjang tanpa memblokir respons API melalui background task, serta modularitas sistem agar setiap komponen dapat dikembangkan dan diuji secara independen.
2. Perancangan (System Design): Tahap perancangan mentransformasikan kebutuhan yang telah diidentifikasi menjadi rancangan teknis yang siap diimplementasikan. Pada tahap ini dihasilkan beberapa artefak perancangan yang menjadi acuan implementasi, yaitu rancangan arsitektur berlapis back end, diagram IDEF0 yang memodelkan fungsi-fungsi sistem beserta hubungan antar fungsi, diagram BPMN yang memodelkan alur proses bisnis setiap fitur utama, skema basis data beserta relasi antar tabel, struktur folder implementasi, serta justifikasi pemilihan teknologi yang digunakan.
3. Implementasi (Implementation): Tahap implementasi merupakan tahap penerjemahan rancangan teknis menjadi kode program yang dapat dieksekusi. Pada tahap ini seluruh modul back end dibangun berdasarkan rancangan yang telah disusun pada tahap sebelumnya. Implementasi dilakukan secara modular, setiap modul diselesaikan secara penuh

sebelum beralih ke modul berikutnya dengan urutan: modul autentikasi, modul dataset, modul privacy consent, modul preprocessing, modul eksperimen, dan modul analisis LLM. Urutan ini dipilih karena setiap modul bergantung pada modul sebelumnya untuk dapat berfungsi.

4. Pengujian (Testing): Tahap pengujian dilakukan setelah seluruh modul selesai diimplementasikan untuk memverifikasi bahwa sistem berjalan sesuai dengan kebutuhan dan rancangan yang telah ditetapkan. Pengujian dilakukan menggunakan metode white box testing yaitu pengujian dari sisi developer yang memverifikasi tidak hanya respons API yang dikembalikan kepada klien, tetapi juga kondisi internal sistem seperti record yang tersimpan di basis data, berkas fisik yang tersimpan di server, serta perubahan status entitas setelah operasi dijalankan.

3.3 Rancangan Backend

Rancangan back end mendeskripsikan seluruh aspek teknis dari komponen back end sistem AutoML Platform yang menjadi fokus pengembangan dalam penelitian ini. Rancangan ini mencakup arsitektur sistem yang menggambarkan bagaimana tanggung jawab dibagi antar lapisan, struktur modul dan endpoint API yang tersedia, skema basis data beserta relasi antar tabel, serta mekanisme keamanan yang diterapkan untuk melindungi data dan membatasi akses pengguna berdasarkan role masing-masing.

3.3.1 Arsitektur Backend

Komponen back end pada sistem AutoML Platform dibangun menggunakan arsitektur berlapis (layered architecture) yang membagi tanggung jawab sistem ke dalam beberapa lapisan yang saling terpisah namun saling terintegrasi. Pemisahan lapisan ini bertujuan agar setiap komponen memiliki tanggung jawab yang jelas, sehingga sistem lebih mudah dipelihara dan dikembangkan secara modular. Secara keseluruhan, arsitektur back end terdiri dari empat lapisan utama.

Lapisan pertama adalah Router Layer, yaitu lapisan paling luar yang bertugas menerima seluruh request HTTP dari front end, memetakan request tersebut ke handler yang sesuai berdasarkan method dan path, serta me-

mvalidasi otorisasi pengguna sebelum request diproses lebih lanjut. Lapisan ini diimplementasikan melalui delapan modul router yang masing-masing menangani domain fungsional yang berbeda.

Lapisan kedua adalah Service Layer, yaitu lapisan yang memuat seluruh logika bisnis sistem. Setiap operasi yang melibatkan pemrosesan data, aturan validasi, maupun pengambilan keputusan dikerjakan di lapisan ini, sehingga router tidak perlu mengetahui detail implementasi logika bisnis. Service layer juga bertanggung jawab memanggil fungsi-fungsi dari komponen ML Service yang berada di luar back end untuk menjalankan proses preprocessing dan training model secara asinkron melalui mekanisme background task.

Lapisan ketiga adalah Model Layer, yaitu lapisan yang mendefinisikan skema seluruh tabel dalam basis data menggunakan SQLAlchemy ORM. Lapisan ini menjadi satu-satunya titik interaksi antara kode Python dan basis data SQLite, sehingga perubahan skema cukup dilakukan pada lapisan ini tanpa harus mengubah lapisan lain.

Lapisan keempat adalah Utility Layer, yaitu lapisan pendukung yang menyediakan fungsi-fungsi lintas modul seperti pembuatan dan validasi token JWT, enkripsi dan dekripsi data sensitif, serta pengelolaan berkas yang diunggah ke server.

3.3.2 Struktur Modul dan Endpoint

Back end sistem AutoML Platform diorganisasikan ke dalam delapan modul router yang masing-masing merepresentasikan satu domain fungsional. Seluruh endpoint menggunakan base URL `/api/v2` dan menerapkan format respons yang seragam, yaitu `{"status": "success", "data": {...}}` untuk permintaan yang berhasil dan `{"status": "error", "message": "...", "detail": {...}}` untuk permintaan yang gagal.

- Modul Autentikasi (`/api/v2/auth`), Modul ini menangani proses masuk ke sistem dan pengambilan informasi pengguna yang sedang aktif. Endpoint `POST /auth/login` menerima email dan kata sandi, memverifikasi kecocokannya dengan data di basis data, lalu menghasilkan token JWT yang akan digunakan sebagai bukti autentikasi pada setiap request berikutnya. Endpoint `GET /auth/me` digunakan untuk mengambil informasi pengguna berdasarkan token yang aktif.

- Modul Admin (/api/v2/admin), Modul ini hanya dapat diakses oleh pengguna dengan role admin dan menyediakan operasi pengelolaan akun pengguna. Endpoint yang tersedia mencakup pembuatan akun baru (POST /admin/users), pengambilan daftar seluruh pengguna (GET /admin/users), pembaruan data pengguna termasuk perubahan role dan status aktif (PUT /admin/users/{user_id}), serta penghapusan akun (DELETE /admin/users/{user_id}).
- Modul Dataset (/api/v2/datasets), Modul ini menangani seluruh operasi yang berkaitan dengan dataset. Sistem mendukung tiga tipe dataset yaitu tabular (berkas CSV), gambar (berkas ZIP berisi folder folder kelas), dan teks (berkas TXT). Masing-masing tipe memiliki endpoint unggah tersendiri, yaitu POST /datasets/upload/tabular, POST /datasets/upload/image, dan POST /datasets/upload/text. Selain itu, modul ini menyediakan endpoint untuk mengambil daftar dataset (GET /datasets), melihat detail dan pratinjau dataset (GET /datasets/{dataset_id}), mengambil hasil analisis kualitas data (GET /datasets/{dataset_id}/quality), melihat pratinjau gambar per kelas untuk dataset bertipe gambar (GET /datasets/{dataset_id}/preview-images), melihat isi dokumen untuk dataset bertipe teks (GET /datasets/{dataset_id}/documents), mengubah status dataset (PATCH /datasets/{dataset_id}/status), serta menghapus dataset beserta berkas fisiknya (DELETE /datasets/{dataset_id}). Saat dataset tabular diunggah, sistem secara otomatis mendeteksi peran dataset (dataset role) berdasarkan nama-nama kolomnya. Dataset yang memiliki kolom verbatim, coding, dan analisis akan ditetapkan sebagai llm_reference; yang memiliki kolom verbatim dan coding ditetapkan sebagai llm_new; yang hanya memiliki kolom verbatim ditetapkan sebagai llm_raw; dan selain itu ditetapkan sebagai general. Penetapan peran ini dilakukan secara otomatis untuk menentukan alur pemrosesan yang sesuai bagi setiap dataset.
- Modul Privacy Consent (/api/v2/consent), Modul ini mengimplementasikan mekanisme persetujuan privasi yang mengacu pada Undang-Undang Nomor 27 Tahun 2022 tentang Pelindungan Data Pribadi. Sebelum dataset yang mengandung data subjek penelitian dapat diproses, pengguna dengan role data_engineer wajib mengunggah bukti per-

setujuan tertulis dari setiap subjek dalam format PDF dan mencatat status persetujuan masing-masing subjek untuk dua aspek, yaitu persetujuan penyimpanan data dan persetujuan pemrosesan data. NIK setiap subjek dienkripsi menggunakan algoritma Fernet sebelum disimpan ke basis data, sehingga data identitas tidak pernah tersimpan dalam bentuk teks biasa. Apabila terdapat satu subjek atau lebih yang tidak menyetujui pemrosesan data, status dataset akan dikunci (locked) dan tidak dapat digunakan untuk preprocessing maupun analisis lebih lanjut.

- Modul Preprocessing (`/api/v2/preprocessing`), Modul ini menangani proses transformasi dataset mentah menjadi data siap latih. Proses preprocessing dijalankan secara asinkron sebagai background task sehingga tidak memblokir respons API. Endpoint `POST /preprocessing` memulai proses preprocessing dan segera mengembalikan respons dengan ID pekerjaan, sementara `GET /preprocessing/{prep_id}` digunakan untuk memantau statusnya. Sistem mendukung tiga jenis konfigurasi preprocessing sesuai tipe dataset: tabular (penanganan nilai hilang, encoding, dan scaling), gambar (resize dan konversi warna), serta teks (pembersihan teks, penghapusan stopwords, dan representasi TF-IDF). Fungsi preprocessing aktual dipanggil dari komponen ML Service melalui import langsung ke modul Python yang telah disiapkan oleh ML Engineer.
- Modul Eksperimen (`/api/v2/experiments`), Modul ini mengelola seluruh siklus pelatihan model machine learning. Eksperimen hanya dapat dibuat dari data preprocessing yang telah berstatus completed. Sistem mendukung tiga tipe dataset dengan pilihan algoritma yang berbeda-beda: dataset tabular mendukung classification (Decision Tree, Random Forest, SVM, KNN, Logistic Regression) dan regression (Linear Regression, Decision Tree, Random Forest, SVR); dataset gambar dan teks hanya mendukung classification dengan algoritma SVM, Random Forest, dan KNN untuk gambar, serta Naive Bayes, Logistic Regression, SVM, dan Random Forest untuk teks. Sama seperti preprocessing, proses pelatihan dijalankan sebagai background task dan statusnya dapat dipantau melalui endpoint status.

- Modul Metadata (/api/v2/metadata), Modul ini mencatat metadata dari setiap entitas dalam sistem, mencakup dataset, hasil preprocessing, dan eksperimen. Metadata ditulis secara otomatis oleh service yang bersangkutan setiap kali terjadi operasi baru, dan dapat diambil kembali untuk keperluan dokumentasi dan penelusuran (lineage) data.
- Modul LLM Analysis (/api/v2/analysis), Modul ini menyediakan fitur analisis teks menggunakan model bahasa besar (Large Language Model) yang berjalan secara lokal melalui Ollama. Studi kasus yang diimplementasikan adalah analisis verbatim dan coding psikologi klinis, di mana sistem menggunakan dataset referensi yang sudah memiliki contoh analisis sebagai panduan gaya penulisan untuk menghasilkan analisis pada data baru. Sistem mendukung tiga model LLM yang dapat dipilih pengguna, yaitu mistral:7b, llama3.2:3b, dan qwen2.5:7b. Hasil analisis dari setiap model dievaluasi menggunakan skor ROUGE-1, ROUGE-2, dan ROUGE-L, serta dicatat waktu generate dan rata-rata panjang analisis yang dihasilkan, sehingga pengguna dapat membandingkan performa antar model.

3.3.3 Skema Basis Data

Basis data sistem AutoML Platform diimplementasikan menggunakan SQLite dengan tujuh table. Tabel users menyimpan data akun pengguna sistem. Kolom role menggunakan nilai enumerasi yang terdiri dari admin, data_engineer, data_scientist, ml_engineer, dan viewer, yang menentukan hak akses setiap pengguna terhadap masing-masing endpoint. Kolom password_hash menyimpan kata sandi dalam bentuk hash bcrypt dan tidak pernah menyimpan kata sandi dalam bentuk teks biasa. Kolom is_active memungkinkan admin menonaktifkan akun tanpa menghapus datanya dari sistem. Tabel data users ditunjukkan pada tabel 3.1 berikut:

Tabel 3.1: Tabel Data Users

Nama Kolom	Tipe Data	Deskripsi
id	INTEGER	Primary key, identitas unik pengguna, auto-increment
username	STRING	Nama pengguna, bersifat unik di seluruh sistem
email	STRING	Alamat email pengguna, bersifat unik, digunakan untuk login
password_hash	STRING	Kata sandi yang telah di-hash menggunakan bcrypt
role	STRING	Peran pengguna: admin, data_engineer, data_scientist, ml_engineer, atau viewer
is_active	INTEGER	Status aktif akun: 1 untuk aktif, 0 untuk dinonaktifkan
created_at	DATETIME	Waktu pembuatan akun, diisi otomatis oleh sistem

Tabel datasets menyimpan metadata setiap dataset yang diunggah. Kolom data_type menentukan tipe dataset (tabular, image, atau text), sedangkan kolom dataset_role menentukan peran dataset dalam sistem (general, llm_reference, llm_new, atau llm_raw). Kolom filepath menyimpan lokasi fisik berkas atau folder dataset di server. Kolom meta bertipe JSON menyimpan informasi ringkas yang berbeda-beda tergantung tipe dataset: untuk tabular berisi jumlah baris dan daftar kolom, untuk gambar berisi daftar kelas dan jumlah gambar per kelas, dan untuk teks berisi jumlah dokumen dan daftar label. Kolom status mencatat kondisi dataset saat ini dan dapat bernilai uploaded, preprocessed, atau locked. Tabel data datasets ditunjukkan pada tabel 3.2 berikut:

Tabel 3.2: Tabel Data Datasets

Nama Kolom	Tipe Data	Deskripsi
id	INTEGER	Primary key, identitas unik dataset, auto-increment
uploaded_by	INTEGER	Foreign key ke tabel users, pengguna yang mengunggah dataset
name	STRING	Nama dataset yang diberikan pengguna
data_type	STRING	Tipe dataset: tabular, image, atau text
filepath	STRING	Lokasi berkas atau folder dataset di server
original_filename	STRING	Nama berkas asli sebelum disimpan ke server
status	STRING	Status dataset: uploaded, preprocessed, atau locked
meta	JSON	Informasi ringkas dataset; untuk tabular berisi jumlah baris dan daftar kolom, untuk gambar berisi daftar kelas dan jumlah gambar per kelas, untuk teks berisi jumlah dokumen
dataset_role	STRING	Peran dataset: general, llm_reference, llm_new, atau llm_raw; dideteksi otomatis dari nama kolom saat upload
created_at	DATETIME	Waktu unggah dataset, diisi otomatis oleh sistem

Tabel preprocessings menyimpan setiap pekerjaan preprocessing yang dijalankan. Kolom config bertipe JSON menyimpan parameter konfigurasi yang dikirimkan pengguna dan berbeda isinya tergantung tipe data. Kolom output_path menyimpan lokasi hasil preprocessing yang akan digunakan sebagai masukan proses pelatihan. Kolom is_trained menandai apakah hasil preprocessing ini sudah pernah digunakan untuk melatih model. Tabel data preprocessings ditunjukkan pada tabel 3.3 berikut:

Tabel 3.3: Tabel Data Preprocessings

Nama Kolom	Tipe Data	Deskripsi
id	INTEGER	Primary key, identitas unik pekerjaan preprocessing, auto-increment
dataset_id	INTEGER	Foreign key ke tabel datasets, dataset yang diproses
processed_by	INTEGER	Foreign key ke tabel users, pengguna yang menjalankan preprocessing
name	STRING	Nama yang diberikan pengguna untuk pekerjaan preprocessing ini
config	JSON	Konfigurasi preprocessing; untuk tabular berisi target_column, feature_columns, missing_strategy, encoding, dan scaling; untuk gambar berisi image_size dan grayscale; untuk teks berisi labels, remove_stopwords, language, dan max_features
output_path	STRING	Lokasi hasil preprocessing di server yang akan digunakan sebagai masukan pelatihan
status	STRING	Status pekerjaan: pending, running, completed, atau failed
is_trained	INTEGER	Penanda apakah hasil preprocessing ini sudah digunakan untuk melatih model: 1 untuk sudah, 0 untuk belum
error_message	STRING	Pesan kesalahan jika proses gagal, bernilai NULL jika berhasil
created_at	DATETIME	Waktu pembuatan pekerjaan, diisi otomatis oleh sistem

Tabel experiments menyimpan setiap eksperimen pelatihan model. Kolom metrics bertipe JSON menyimpan hasil evaluasi model setelah pelatihan selesai, yang isinya berbeda tergantung tipe tugas: untuk classification beri-

si akurasi, presisi, recall, dan F1-score; untuk regression berisi MSE, RMSE, MAE, dan R^2 -score. Kolom `model_path` menyimpan lokasi berkas model terlatih dalam format `.pkl`. Tabel data eksperimen ditunjukkan pada tabel 3.4 berikut:

Tabel 3.4: Tabel Data Eksperimen

Nama Kolom	Tipe Data	Deskripsi
id	INTEGER	Primary key, identitas unik eksperimen, auto-increment
preprocessing_id	INTEGER	Foreign key ke tabel preprocessings, data hasil preprocessing yang digunakan
dataset_id	INTEGER	Foreign key ke tabel datasets, dataset asal eksperimen
created_by	INTEGER	Foreign key ke tabel users, pengguna yang membuat eksperimen
name	STRING	Nama eksperimen yang diberikan pengguna
data_type	STRING	Jenis tugas: classification atau regression; untuk dataset gambar dan teks selalu classification
algorithm	STRING	Nama algoritma yang digunakan, misalnya random_forest atau svm
hyperparameters	JSON	Parameter model yang dikonfigurasi pengguna; dapat berupa objek kosong jika menggunakan nilai default
task_type	STRING	Jenis tugas: classification atau regression; untuk dataset gambar dan teks selalu classification
status	STRING	Status eksperimen: queued, running, completed, atau failed
progress	INTEGER	Persentase kemajuan pelatihan dalam rentang 0–100
metrics	JSON	Hasil evaluasi model; untuk classification berisi akurasi, presisi, recall, dan F1-score; untuk regression berisi MSE, RMSE, MAE, dan R^2 ; bernilai NULL jika pelatihan belum selesai
model_path	STRING	Lokasi berkas model terlatih (.pkl) di server; bernilai NULL jika pelatihan belum selesai
error_message	STRING	Pesan kesalahan jika pelatihan gagal, bernilai NULL jika berhasil
created_at	DATETIME	Waktu pembuatan eksperimen, diisi otomatis oleh sistem

Tabel privacy_consent dan privacy_consent_subjects menyimpan data

persetujuan privasi. Setiap dataset dapat memiliki satu consent yang terdiri dari banyak subjek. Tabel `privacy_consent_subjects` menyimpan status persetujuan setiap subjek untuk dua aspek (penyimpanan dan pemrosesan), serta kolom `nik_encrypted` yang menyimpan NIK dalam bentuk terenkripsi. Tabel data `privacy_consents` ditunjukkan pada tabel 3.5 berikut:

Tabel 3.5: Tabel Data `privacy_consents`

Nama Kolom	Tipe Data	Deskripsi
<code>id</code>	INTEGER	Primary key, identitas unik catatan consent, auto-increment
<code>dataset_id</code>	INTEGER	Foreign key ke tabel <code>datasets</code> , dataset yang terkait dengan consent ini
<code>submitted_by</code>	INTEGER	Foreign key ke tabel <code>users</code> , pengguna yang mengajukan consent
<code>status_store</code>	STRING	Status agregat persetujuan penyimpanan data dari seluruh subjek: <code>approved</code> atau <code>rejected</code>
<code>status_process</code>	STRING	Status agregat persetujuan pemrosesan data dari seluruh subjek: <code>approved</code> atau <code>rejected</code>
<code>created_at</code>	DATETIME	Waktu pengajuan consent, diisi otomatis oleh sistem

Tabel `privacy_consent_subjects` ditunjukkan pada tabel 3.6 berikut:

Tabel 3.6: Tabel Data privacy_concent_subjects

Nama Kolom	Tipe Data	Deskripsi
id	INTEGER	Primary key, identitas unik data subjek, auto-increment
consent_id	INTEGER	Foreign key ke tabel privacy_consent, consent induk dari subjek ini
nik_encrypted	STRING	NIK subjek yang telah dienkripsi menggunakan algoritma Fernet sebelum disimpan
name	STRING	Nama lengkap subjek penelitian
agree_store	INTEGER	Persetujuan penyimpanan data dari subjek: 1 untuk setuju, 0 untuk tidak setuju
agree_process	INTEGER	Persetujuan pemrosesan data dari subjek: 1 untuk setuju, 0 untuk tidak setuju
pdf_path	STRING	Lokasi berkas PDF bukti tanda tangan persetujuan fisik di server; dapat bernilai NULL

Tabel metadata dan llm_analyses masing-masing menyimpan catatan metadata lintas entitas dan hasil analisis LLM beserta metrik evaluasinya. Tabel data metadata ditunjukkan pada tabel 3.7 berikut:

Tabel 3.7: Tabel Data Metadata

Nama Kolom	Tipe Data	Deskripsi
id	INTEGER	Primary key, identitas unik catatan metadata, auto-increment
entity_type	STRING	Jenis entitas yang dicatat: dataset, preprocessing, atau experiment
entity_id	INTEGER	ID entitas yang bersangkutan
dataset_id	INTEGER	Foreign key ke tabel datasets, diisi jika entitas terkait dengan dataset
preprocessing_id	INTEGER	Foreign key ke tabel preprocessings, diisi jika entitas terkait dengan preprocessing
experiment_id	INTEGER	Foreign key ke tabel experiments, diisi jika entitas terkait dengan eksperimen
name	STRING	Nama entitas yang dicatat
description	STRING	Deskripsi singkat entitas
source	STRING	Sumber asal data, misalnya nama berkas asli
data_type	STRING	Tipe data entitas yang dicatat
created_by	INTEGER	Foreign key ke tabel users, pengguna yang membuat entitas
properties	TEXT	Data properti tambahan dalam format JSON string
lineage	TEXT	Informasi silsilah data (data lineage) dalam format JSON string
created_at	DATETIME	Waktu pencatatan metadata, diisi otomatis oleh sistem

Tabel llm_analyses ditunjukkan pada tabel 3.8 berikut:

Tabel 3.8: Tabel Data llm_analyses

Nama Kolom	Tipe Data	Deskripsi
id	INTEGER	Primary key, identitas unik pekerjaan analisis LLM, auto-increment
dataset_reference_id	INTEGER	Foreign key ke tabel datasets, dataset referensi yang berisi contoh analisis
dataset_new_id	INTEGER	Foreign key ke tabel datasets, dataset baru yang akan dianalisis
created_by	INTEGER	Foreign key ke tabel users, pengguna yang memulai analisis
model_name	STRING	Nama model LLM yang digunakan: mistral:7b, llama3.2:3b, atau qwen2.5:7b
results	TEXT	Hasil analisis yang dihasilkan model dalam format JSON string; berisi daftar objek dengan verbatim, coding, dan analisis
rouge_scores	TEXT	Skor evaluasi ROUGE dalam format JSON string; berisi nilai ROUGE-1, ROUGE-2, dan ROUGE-L
generate_time	FLOAT	Waktu yang dibutuhkan model untuk menghasilkan seluruh analisis, dalam satuan detik
avg_length	FLOAT	Rata-rata panjang teks analisis yang dihasilkan, dalam satuan kata
status	STRING	Status pekerjaan: pending, running, completed, atau failed
error_message	STRING	Pesan kesalahan jika analisis gagal, bernilai NULL jika berhasil
created_at	DATETIME	Waktu pembuatan pekerjaan analisis, diisi otomatis oleh sistem

3.3.4 Mekanisme Keamanan

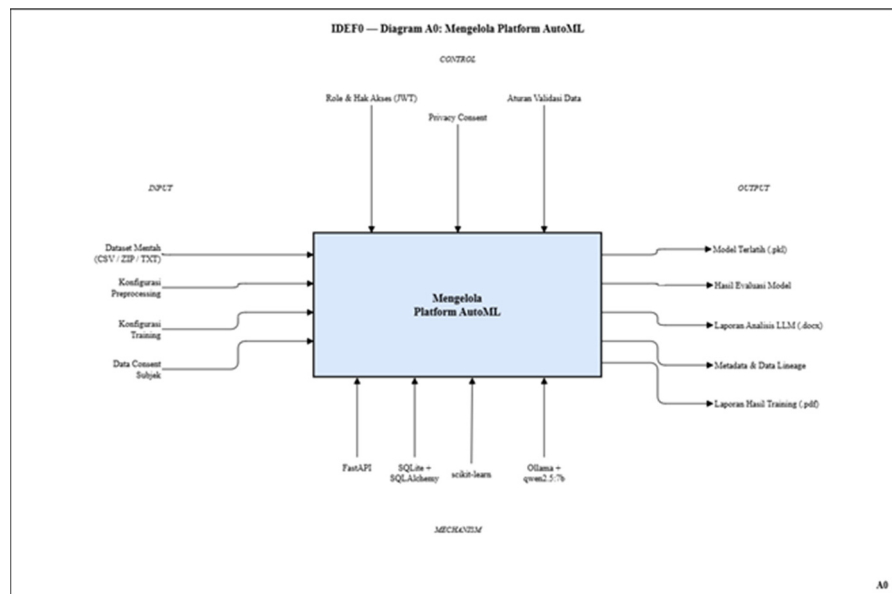
Sistem menerapkan beberapa mekanisme keamanan yang bekerja pada lapisan yang berbeda. Pertama, autentikasi berbasis token JWT (JSON Web Token) dengan algoritma HS256 diterapkan pada seluruh endpoint kecuali POST /auth/login. Token memiliki masa berlaku yang dikonfigurasi melalui variabel lingkungan dan harus disertakan pada setiap request melalui header Authorization: Bearer <token>. Kedua, otorisasi berbasis role diterapkan melalui dependency require_role() yang diperiksa pada setiap endpoint sebelum logika bisnis dijalankan. Ketiga, kata sandi pengguna selalu disimpan dalam bentuk hash menggunakan algoritma bcrypt dan tidak pernah dapat dikembalikan ke bentuk aslinya. Keempat, NIK subjek penelitian pada fitur privacy consent dienkrpsi menggunakan algoritma Fernet sebelum disimpan ke basis data. Kelima, seluruh konfigurasi sensitif seperti secret key, encryption key, dan database URL disimpan di berkas .env yang tidak diunggah ke repositori kode.

3.4 Diagram IDEF0

IDEF0 (Integration Definition for Function Modeling) merupakan metode pemodelan yang digunakan untuk menggambarkan fungsi-fungsi sistem beserta hubungan antar fungsi secara hierarkis. Setiap fungsi dalam IDEF0 digambarkan sebagai sebuah kotak dengan empat sisi yang merepresentasikan empat elemen, yaitu Input (sisi kiri) sebagai data yang dikonsumsi oleh fungsi, Output (sisi kanan) sebagai data yang dihasilkan oleh fungsi, Control (sisi atas) sebagai aturan atau kebijakan yang mengendalikan fungsi, dan Mechanism (sisi bawah) sebagai sumber daya yang digunakan untuk menjalankan fungsi. Diagram IDEF0 sistem back end AutoML Platform dimodelkan dalam tiga level hierarki, mulai dari A0 sebagai gambaran paling umum hingga level 3 sebagai dekomposisi paling rinci.

3.4.1 Diagram A0 Mengelola Platform AutoML

Diagram A0 pada Gambar 3.1 merupakan gambaran paling tinggi dari keseluruhan sistem back end AutoML Platform dalam satu fungsi tunggal bernama Mengelola Platform AutoML.



Gambar 3.1: Diagram IDEF0 Level 1

Input yang masuk ke sistem terdiri dari empat elemen, yaitu Dataset Mentah (CSV/ZIP/TXT) sebagai data yang akan diolah, Konfigurasi Preprocessing sebagai parameter transformasi data, Konfigurasi Training sebagai parameter pelatihan model, dan Data Consent Subjek sebagai data persetujuan privasi dari setiap subjek penelitian.

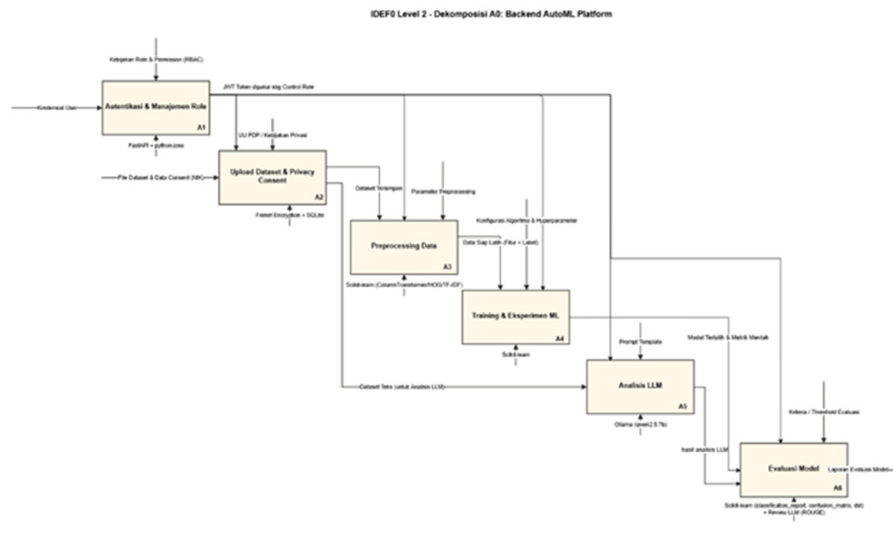
Control yang mengendalikan seluruh proses sistem terdiri dari tiga elemen, yaitu Role & Hak Akses (JWT) yang memastikan setiap operasi hanya dapat dilakukan oleh pengguna dengan role yang sesuai, Privacy Consent yang mengatur kebijakan penggunaan data subjek, dan Aturan Validasi Data yang menentukan format dan batasan data yang dapat diterima sistem.

Output yang dihasilkan sistem terdiri dari lima elemen, yaitu Model Terlatih (.pkl) sebagai hasil akhir proses pelatihan, Hasil Evaluasi Model berupa metrik performa, Laporan Analisis LLM (.docx) sebagai hasil analisis teks oleh model bahasa, Metadata & Data Lineage sebagai catatan dokumentasi seluruh proses, dan Laporan Hasil Training (.pdf).

Mechanism yang digunakan untuk menjalankan sistem terdiri dari empat komponen teknologi, yaitu FastAPI sebagai framework API server, SQLite + SQLAlchemy sebagai basis data dan ORM, scikit-learn sebagai library pelatihan model ML, serta Ollama + qwen2.5:7b sebagai engine model bahasa besar lokal.

3.4.2 Diagram A1-A6

Diagram A1 samapi A6 pada Gambar 3.2 merupakan dekomposisi level 2 dari fungsi A0 yang menguraikan sistem menjadi enam fungsi utama yang saling terhubung secara berurutan.

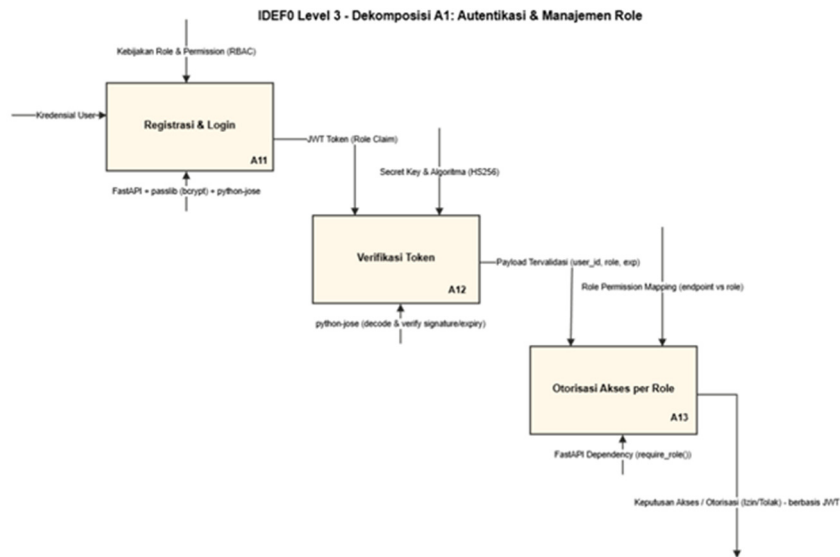


Gambar 3.2: Diagram IDEF0 A1-A6

Fungsi pertama A1 Autentikasi & Manajemen Role, menerima Kredensial User sebagai input dan dikendalikan oleh Kebijakan Role & Permission (RBAC). Fungsi ini dijalankan menggunakan FastAPI + python-jose dan menghasilkan JWT Token yang selanjutnya digunakan sebagai control oleh seluruh fungsi berikutnya untuk memastikan setiap operasi hanya dapat diakses oleh role yang berwenang.

Fungsi kedua A2 Upload Dataset & Privacy Consent, menerima File Dataset & Data Consent (NIK) sebagai input, dikendalikan oleh JWT Token dari A1 dan UU PDP/Kebijakan Privasi. Fungsi ini dijalankan menggunakan Fernet Encryption + SQLite dan menghasilkan dua output, yaitu Dataset Tersimpan yang diteruskan ke A3, serta Dataset Teks (untuk Analisis LLM) yang diteruskan langsung ke A5.

Fungsi ketiga A3 Preprocessing Data, menerima Dataset Tersimpan dari A2 dan Parameter Preprocessing sebagai input, dikendalikan oleh JWT Token dari A1. Fungsi ini dijalankan menggunakan Scikit-learn (ColumnTransformer/HOG/TF-IDF) dan menghasilkan Data Siap Latih (Fitur + Label) yang diteruskan ke A4.



Gambar 3.3: Diagram A1: Autentikasi & Manajemen Role

Fungsi keempat A4 Training & Eksperimen ML, menerima Data Siap Latih dari A3 dan Konfigurasi Algoritma & Hyperparameter sebagai input, dikendalikan oleh JWT Token dari A1. Fungsi ini dijalankan menggunakan Scikit-learn dan menghasilkan Model Terlatih & Metrik Mentah yang diteruskan ke A6.

Fungsi kelima A5 Analisis LLM, menerima Dataset Teks dari A2 dan Prompt Template sebagai input, dikendalikan oleh JWT Token dari A1. Fungsi ini dijalankan menggunakan Ollama (qwen2.5:7b) dan menghasilkan Hasil Analisis LLM yang diteruskan ke A6.

Fungsi keenam A6 Evaluasi Model, menerima Model Terlatih & Metrik Mentah dari A4 dan Hasil Analisis LLM dari A5 sebagai input, dikendalikan oleh JWT Token dari A1 dan Kriteria/Threshold Evaluasi. Fungsi ini dijalankan menggunakan Scikit-learn (classification_report, confusion_matrix) + Review LLM (ROUGE) dan menghasilkan Laporan Evaluasi Model sebagai output akhir sistem.

3.4.3 Diagram Dekomposisi A1: Autentikasi & Manajemen Role

Diagram Dekomposisi A1 pada Gambar 3.3 menguraikan fungsi A1 menjadi tiga sub-fungsi yang berjalan secara berurutan.

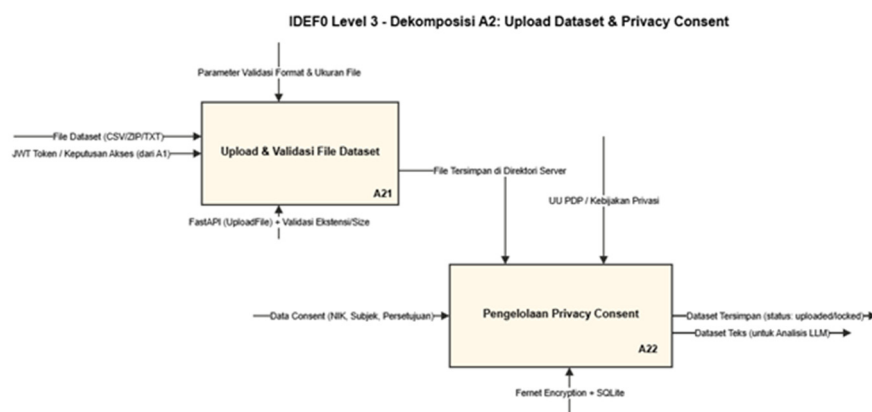
Sub-fungsi pertama A11 Registrasi & Login, menerima Kredensial User sebagai input dan dikendalikan oleh Kebijakan Role & Permission (RBAC). Fungsi ini dijalankan menggunakan FastAPI + passlib (bcrypt) + python-jose dan menghasilkan JWT Token (Role Claim) yang diteruskan ke sub-fungsi berikutnya.

Sub-fungsi kedua A12 Verifikasi Token, menerima JWT Token dari A11 sebagai input dan dikendalikan oleh Secret Key & Algoritma HS256. Fungsi ini dijalankan menggunakan python-jose (decode & verify signature/expiry) dan menghasilkan Payload Tervalidasi (user_id, role, exp) yang diteruskan ke A13.

Sub-fungsi ketiga A13 Otorisasi Akses per Role, menerima Payload Tervalidasi dari A12 sebagai input dan dikendalikan oleh Role Permission Mapping (endpoint vs role). Fungsi ini dijalankan menggunakan FastAPI Dependency (require_role()) dan menghasilkan Keputusan Akses/Otorisasi (Izin/Tolak) berbasis JWT sebagai output akhir yang digunakan oleh seluruh modul lain sebagai control.

3.4.4 Diagram Dekomposisi A2: Upload Dataset & Privacy Consent

Diagram Dekomposisi A2 pada Gambar 3.4 menguraikan fungsi A2 menjadi dua sub-fungsi.



Gambar 3.4: Diagram Dekomposisi A2: Upload Dataset & Privacy Consent

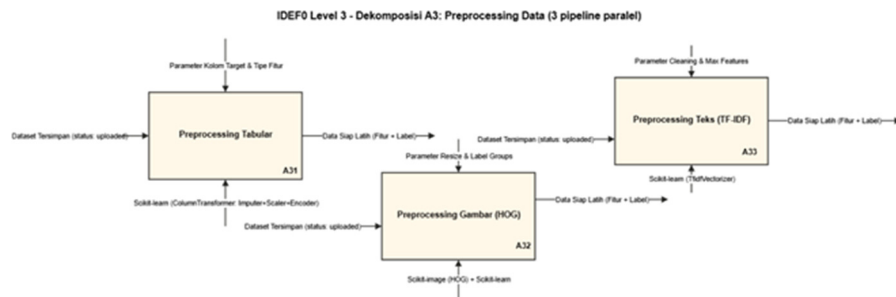
Sub-fungsi pertama A21 Upload & Validasi File Dataset, menerima File Dataset (CSV/ZIP/TXT) dan JWT Token/Keputusan Akses dari A1 sebagai

input, dikendalikan oleh Parameter Validasi Format & Ukuran File. Fungsi ini dijalankan menggunakan FastAPI (UploadFile) + Validasi Ekstensi/Size dan menghasilkan File Tersimpan di Direktori Server yang diteruskan ke sub-fungsi berikutnya.

Sub-fungsi kedua A22 Pengelolaan Privacy Consent, menerima File Tersimpan dari A21 dan Data Consent (NIK, Subjek, Persetujuan) sebagai input, dikendalikan oleh UU PDP/Kebijakan Privasi. Fungsi ini dijalankan menggunakan Fernet Encryption + SQLite dan menghasilkan dua output, yaitu Dataset Tersimpan dengan status uploaded atau locked bergantung pada hasil consent, serta Dataset Teks (untuk Analisis LLM) yang diteruskan langsung ke A5.

3.4.5 Diagram Dekomposisi A3: Preprocessing Data

Diagram Dekomposisi A3 pada Gambar 3.5 menguraikan fungsi A3 menjadi tiga sub-fungsi yang berjalan secara paralel sesuai tipe data yang diproses.



Gambar 3.5: Diagram Dekomposisi A3: Preprocessing Data

Sub-fungsi pertama A31 Preprocessing Tabular, menerima Dataset Tersimpan berstatus uploaded sebagai input dan dikendalikan oleh Parameter Kolom Target & Tipe Fitur. Fungsi ini dijalankan menggunakan Scikit-learn (ColumnTransformer: Imputer + Scaler + Encoder) untuk menangani nilai hilang, normalisasi, dan encoding variabel kategorikal, serta menghasilkan Data Siap Latih (Fitur + Label).

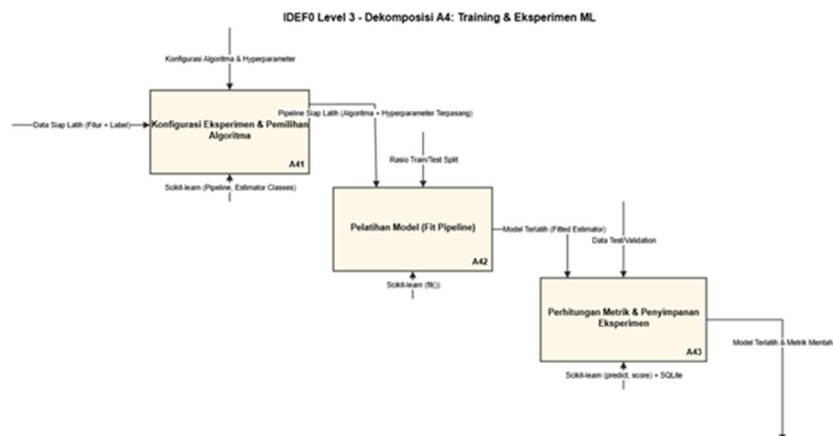
Sub-fungsi kedua A32 Preprocessing Gambar (HOG), menerima Dataset Tersimpan berstatus uploaded sebagai input dan dikendalikan oleh Parameter Resize & Label Groups. Fungsi ini dijalankan menggunakan Scikit-image

(HOG) + Scikit-learn untuk mengekstraksi fitur Histogram of Oriented Gradients dari setiap gambar, serta menghasilkan Data Siap Latih (Fitur + Label).

Sub-fungsi ketiga A33 Preprocessing Teks (TF-IDF), menerima Dataset Tersimpan berstatus uploaded sebagai input dan dikendalikan oleh Parameter Cleaning & Max Features. Fungsi ini dijalankan menggunakan Scikit-learn (TfidfVectorizer) untuk membersihkan teks, menghapus stopwords, dan merepresentasikan dokumen sebagai vektor TF-IDF, serta menghasilkan Data Siap Latih (Fitur + Label) yang diteruskan ke A4.

3.4.6 Diagram Dekomposisi A4: Training & Eksperimen ML

Diagram Dekomposisi A4 pada Gambar 3.6 menguraikan fungsi A4 menjadi tiga sub-fungsi yang berjalan secara berurutan.



Gambar 3.6: Diagram Dekomposisi A4: Training & Eksperimen ML

Sub-fungsi pertama A41 Konfigurasi Eksperimen & Pemilihan Algoritma, menerima Data Siap Latih (Fitur + Label) sebagai input dan dikendalikan oleh Konfigurasi Algoritma & Hyperparameter. Fungsi ini dijalankan menggunakan Scikit-learn (Pipeline, Estimator Classes) dan menghasilkan Pipeline Siap Latih (Algoritma + Hyperparameter Terpasang) yang diteruskan ke A42.

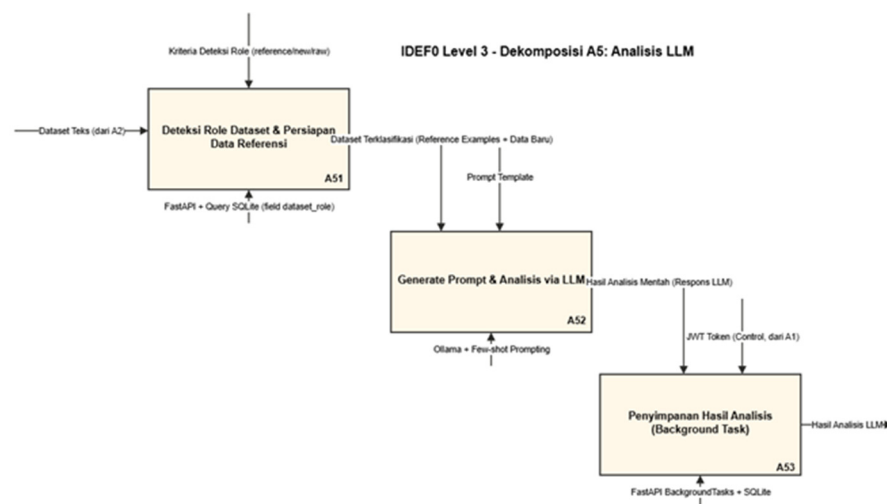
Sub-fungsi kedua A42 Pelatihan Model (Fit Pipeline), menerima Pipeline Siap Latih dari A41 sebagai input dan dikendalikan oleh Rasio Train/Test

Split. Fungsi ini dijalankan menggunakan Scikit-learn (fit()) dan menghasilkan Model Terlatih (Fitted Estimator) beserta Data Test/Validasi yang diteruskan ke A43.

Sub-fungsi ketiga A43 Perhitungan Metrik & Penyimpanan Eksperimen, menerima Model Terlatih dan Data Test/Validasi dari A42 sebagai input. Fungsi ini dijalankan menggunakan Scikit-learn (predict(), score()) + SQLite dan menghasilkan Model Terlatih & Metrik Mentah sebagai output yang diteruskan ke fungsi A6.

3.4.7 Diagram Dekomposisi A5: Analisis LLM

Diagram Dekomposisi A5 pada Gambar 3.7 menguraikan fungsi A5 menjadi tiga sub-fungsi yang berjalan secara berurutan.



Gambar 3.7: Diagram Dekomposisi A5: Analisis LLM

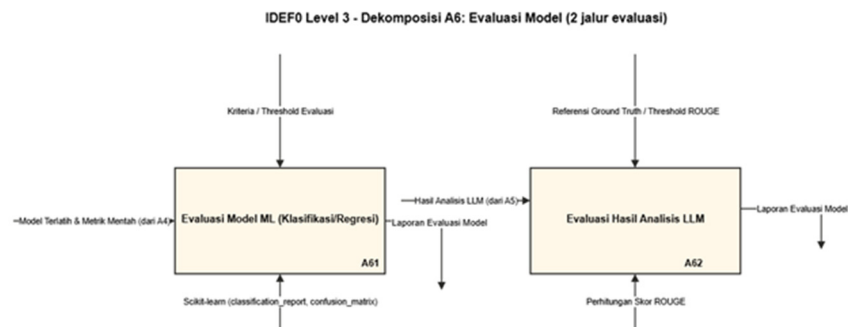
Sub-fungsi pertama A51 Deteksi Role Dataset & Persiapan Data Referensi, menerima Dataset Teks dari A2 sebagai input dan dikendalikan oleh Kriteria Deteksi Role (reference/new/raw). Fungsi ini dijalankan menggunakan FastAPI + Query SQLite (field dataset_role) dan menghasilkan Dataset Terklasifikasi (Reference Examples + Data Baru) yang diteruskan ke A52.

Sub-fungsi kedua A52 Generate Prompt & Analisis via LLM, menerima Dataset Terklasifikasi dari A51 sebagai input dan dikendalikan oleh Prompt Template. Fungsi ini dijalankan menggunakan Ollama + Few-shot Prompting dan menghasilkan Hasil Analisis Mentah (Respons LLM) yang diteruskan ke A53.

Sub-fungsi ketiga A53 Penyimpanan Hasil Analisis (Background Task), menerima Hasil Analisis Mentah dari A52 sebagai input dan dikendalikan oleh JWT Token dari A1. Fungsi ini dijalankan menggunakan FastAPI BackgroundTasks + SQLite dan menghasilkan Hasil Analisis LLM sebagai output akhir yang diteruskan ke fungsi A6.

3.4.8 Diagram Dekomposisi A6: Evaluasi Model

Diagram Dekomposisi A6 pada Gambar 3.8 menguraikan fungsi A6 menjadi dua sub-fungsi yang berjalan secara paralel sesuai jalur evaluasi yang ditempuh.



Gambar 3.8: Diagram Dekomposisi A6: Evaluasi Model

Sub-fungsi pertama A61 Evaluasi Model ML (Klasifikasi/Regresi), menerima Model Terlatih & Metrik Mentah dari A4 sebagai input dan dikendalikan oleh Kriteria/Threshold Evaluasi. Fungsi ini dijalankan menggunakan Scikit-learn (classification_report, confusion_matrix) dan menghasilkan Laporan Evaluasi Model berupa metrik akurasi, presisi, recall, F1-score, serta visualisasi confusion matrix.

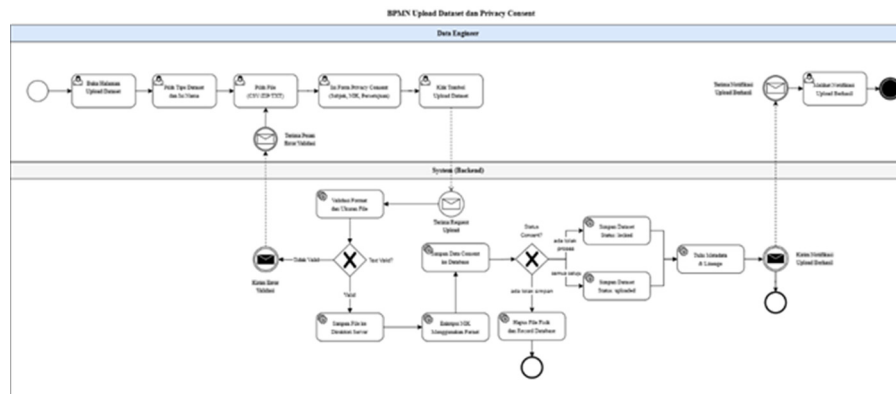
Sub-fungsi kedua A62 Evaluasi Hasil Analisis LLM, menerima Hasil Analisis LLM dari A5 sebagai input dan dikendalikan oleh Referensi Ground Truth/Threshold ROUGE. Fungsi ini dijalankan menggunakan Perhitungan Skor ROUGE dan menghasilkan Laporan Evaluasi Model berupa skor ROUGE-1, ROUGE-2, dan ROUGE-L yang mencerminkan kualitas teks analisis yang dihasilkan model bahasa dibandingkan dengan teks referensi.

3.5 Diagram BPMN

Business Process Model and Notation (BPMN) merupakan standar pemodelan proses bisnis yang menggambarkan alur kerja suatu sistem secara visual menggunakan notasi yang terstandarisasi. Dalam konteks penelitian ini, BPMN digunakan untuk memodelkan interaksi antara pengguna dan sistem back end secara kronologis, termasuk percabangan kondisi, penanganan kesalahan, dan komunikasi antar pool. Setiap diagram BPMN pada sistem AutoML Platform menggunakan swimlane untuk memisahkan tanggung jawab antara aktor pengguna dengan System (Backend), serta layanan eksternal seperti ML Service dan Ollama (LLM Service) apabila relevan. Sistem AutoML Platform dimodelkan ke dalam enam diagram BPMN yang masing-masing merepresentasikan satu alur proses utama.

3.5.1 BPMN Upload Dataset dan Privacy Consent

Diagram BPMN pada Gambar 3.9 menggambarkan alur proses unggah dataset dan pengelolaan privacy consent yang melibatkan dua pool, yaitu Data Engineer dan System (Backend).



Gambar 3.9: BPMN Upload Dataset dan Privacy Consent

Proses dimulai dari pool Data Engineer ketika pengguna membuka halaman Upload Dataset, kemudian memilih tipe dataset dan mengisi nama dataset, memilih file (CSV/ZIP/TXT), mengisi formulir Privacy Consent yang memuat data subjek, NIK, dan status persetujuan, lalu menekan tombol Upload Dataset.

Diagram BPMN pada Gambar 3.10 menggambarkan alur proses preprocessing dataset yang melibatkan tiga pool, yaitu Data Scientist, System (Backend), dan ML Service.



Proses dimulai dari pool Data Scientist ketika pengguna membuka halaman Dataset, memilih dataset dan mengklik tombol proses, melihat pratinjau dan kualitas data, lalu mengisi konfigurasi preprocessing sesuai tipe dataset yang dipilih.

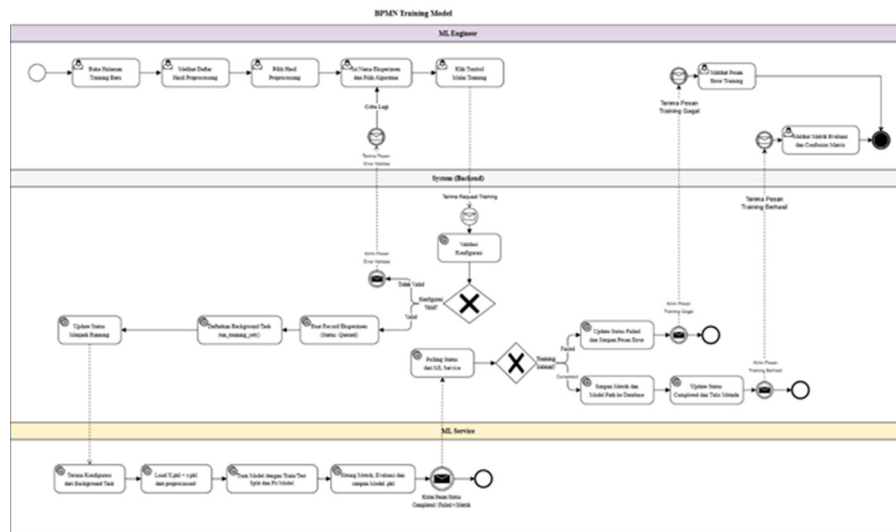
Pada pool System (Backend), sistem menerima request preprocessing dan menjalankan validasi konfigurasi. Apabila konfigurasi tidak valid, sistem mengirimkan pesan Error Validasi dan Data Scientist diminta mengisi ulang konfigurasi. Apabila valid, sistem membuat record Preprocessing dengan status pending dan segera menjalankan background task, sehingga respons API langsung dikembalikan tanpa menunggu proses selesai.

Pada pool ML Service, sistem menerima konfigurasi dari background task dan menentukan tipe data melalui sebuah gateway. Untuk dataset tabular, dijalankan fungsi `run_tabular_preprocessing()` menggunakan Column-Transformer dan OneHotEncoder. Untuk dataset gambar, dijalankan fungsi `run_image_preprocessing()` menggunakan Resize dan HOG Extraction. Untuk dataset teks, dijalankan fungsi `run_text_preprocessing()` menggunakan Cleaning, Stopwords, dan TF-IDF. Hasil preprocessing dari ketiga jalur disimpan ke output path dalam format .pkl, kemudian status dikembalikan ke System (Backend).

System (Backend) menangkap status dari ML Service melalui sebuah gateway. Apabila status failed, sistem memperbarui status menjadi gagal, menyimpan pesan kesalahan, dan mengirimkan notifikasi Preprocessing Gagal kepada Data Scientist. Apabila status completed, sistem memperbarui status menjadi completed, menulis metadata dan lineage secara otomatis, lalu mengirimkan notifikasi Preprocessing Selesai kepada Data Scientist. Proses berakhir setelah notifikasi diterima oleh Data Scientist.

3.5.3 BPMN Training Model

Diagram BPMN pada Gambar 3.11 menggambarkan alur proses pelatihan model machine learning yang melibatkan tiga pool, yaitu ML Engineer, System (Backend), dan ML Service.



Gambar 3.11: BPMN Training Model

Proses dimulai dari pool ML Engineer ketika pengguna membuka halaman Training baru, melihat daftar hasil preprocessing yang tersedia, memilih hasil preprocessing yang akan digunakan, mengisi nama eksperimen dan memilih algoritma beserta hyperparameter, lalu menekan tombol Mulai Training.

Pada pool System (Backend), sistem menerima request training dan menjalankan validasi konfigurasi. Apabila konfigurasi tidak valid, misalnya algoritma tidak sesuai dengan tipe data atau preprocessing belum berstatus completed sistem mengirimkan pesan Error Validasi dan ML Engineer diminta memperbaiki konfigurasi. Apabila valid, sistem membuat record eksperimen dengan status queued, mendaftarkan background task `run_training_job()`, lalu memperbarui status eksperimen menjadi running.

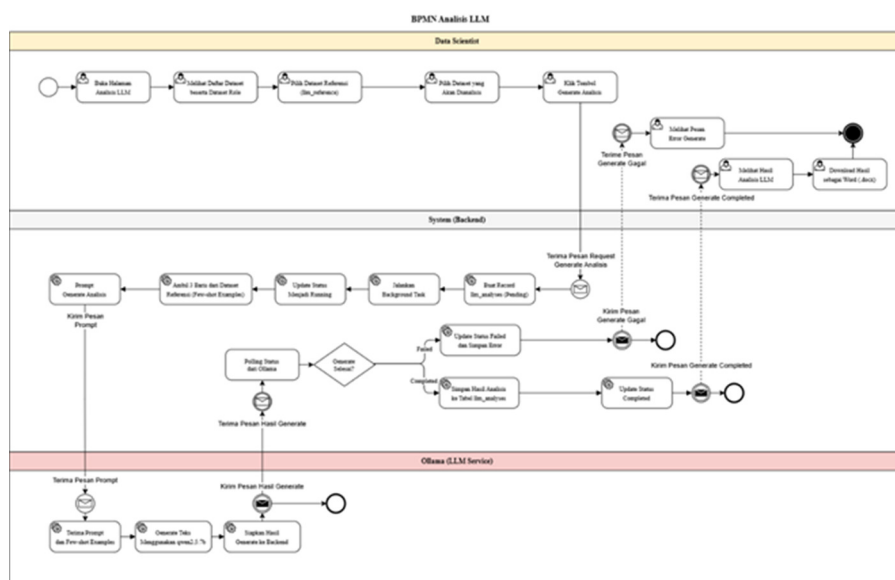
Pada pool ML Service, sistem menerima konfigurasi dari background task, memuat file `X.pkl` dan `y.pkl` dari direktori preprocessed, melatih model menggunakan Train/Test Split dan Fit Model, lalu menghitung metrik evaluasi dan menyimpan model dalam format `.pkl`. Setelah selesai, ML Service mengirimkan pesan status Completed atau Failed beserta metrik kembali ke System (Backend).

System (Backend) melakukan polling status dari ML Service melalui sebuah gateway. Apabila failed, sistem memperbarui status eksperimen menjadi gagal, menyimpan pesan kesalahan, dan mengirimkan pesan Training Gagal kepada ML Engineer. Apabila completed, sistem menyimpan metrik

dan model path ke basis data, memperbarui status menjadi completed, menulis metadata, dan mengirimkan pesan Training Berhasil. ML Engineer kemudian menerima pesan tersebut dan dapat melihat metrik evaluasi serta confusion matrix sebagai hasil akhir proses.

3.5.4 BPMN Analysis LLM

Diagram BPMN pada Gambar 3.12 menggambarkan alur proses analisis teks menggunakan LLM yang melibatkan tiga pool, yaitu Data Scientist, System (Backend), dan Ollama (LLM Service).



Gambar 3.12: BPMN Analisis LLM

Proses dimulai dari pool Data Scientist ketika pengguna membuka halaman Analisis LLM, melihat daftar dataset beserta dataset role masing-masing, memilih dataset referensi bertipe llm_reference, memilih dataset yang akan dianalisis, lalu menekan tombol Generate Analisis.

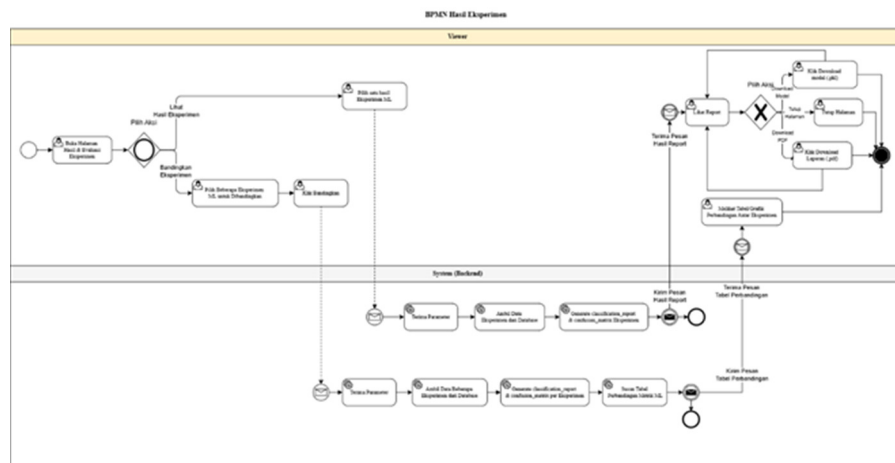
Pada pool System (Backend), sistem menerima request generate analisis dan membuat record baru pada tabel `llm_analyses` dengan status pending. Sistem kemudian menjalankan background task, memperbarui status menjadi running, mengambil tiga baris dari dataset referensi sebagai few-shot examples, dan menyusun prompt generate analisis yang dikirimkan ke Ollama.

Pada pool Ollama (LLM Service), Ollama menerima prompt beserta few-shot examples, menghasilkan teks analisis menggunakan model pilihan, menyiapkan hasil generate, dan mengirimkan respons kembali ke System (Backend).

System (Backend) menerima hasil generate dari Ollama dan memeriksa hasilnya melalui sebuah gateway. Apabila failed, sistem memperbarui status menjadi gagal, menyimpan pesan kesalahan, dan mengirimkan pesan Generate Gagal kepada Data Scientist. Apabila completed, sistem menyimpan hasil analisis ke tabel `llm_analyses`, memperbarui status menjadi completed, dan mengirimkan pesan Generate Completed. Data Scientist kemudian dapat melihat hasil analisis LLM dan mengunduhnya dalam format Word (.docx). Proses berakhir setelah Data Scientist mengunduh hasil analisis.

3.5.5 BPMN Hasil Eksperimen

Diagram BPMN pada Gambar 3.13 menggambarkan alur proses melihat dan membandingkan hasil eksperimen machine learning yang melibatkan dua pool, yaitu Viewer dan System (Backend).



Gambar 3.13: BPMN Hasil Eksperimen

Proses dimulai dari pool Viewer ketika pengguna membuka halaman Hasil & Evaluasi Eksperimen. Sistem menyajikan sebuah gateway pilihan aksi dengan dua jalur. Jalur pertama adalah Lihat Hasil Eksperimen, di mana pengguna memilih satu hasil eksperimen ML yang ingin dilihat secara detail. Jalur kedua adalah Bandingkan Eksperimen, di mana pengguna memilih be-

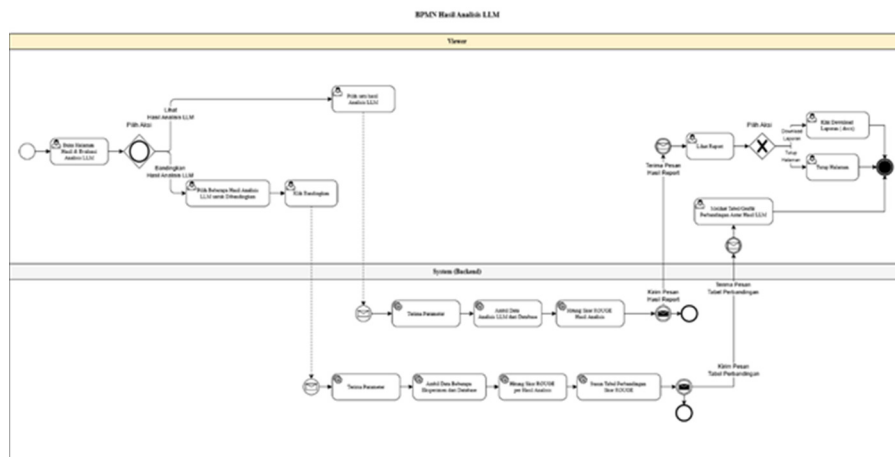
berapa eksperimen ML untuk dibandingkan lalu menekan tombol Bandingkan.

Pada pool System (Backend), kedua jalur memiliki alur yang terpisah. Untuk jalur Lihat, sistem menerima parameter, mengambil data eksperimen dari basis data, lalu generate `classification_report` dan `confusion_matrix` eksperimen tersebut, kemudian mengirimkan pesan Hasil Report ke Viewer. Untuk jalur Bandingkan, sistem menerima parameter, mengambil data beberapa eksperimen dari basis data, generate `classification_report` dan `confusion_matrix` per eksperimen, menyusun tabel perbandingan metrik ML, kemudian mengirimkan pesan Tabel Perbandingan ke Viewer.

Viewer menerima pesan yang sesuai dengan jalurnya, kemudian melihat report atau tabel/grafik perbandingan antar eksperimen. Selanjutnya terdapat gateway Pilih Aksi dengan tiga pilihan: mengunduh model (.pkl), mengunduh laporan (.pdf), atau menutup halaman. Proses berakhir setelah salah satu aksi tersebut dipilih.

3.5.6 BPMN Hasil Analisis LLM

Diagram BPMN pada Gambar 3.14 menggambarkan alur proses melihat dan membandingkan hasil analisis LLM yang melibatkan dua pool, yaitu Viewer dan System (Backend).



Gambar 3.14: BPMN Hasil Analisis LLM

Proses dimulai dari pool Viewer ketika pengguna membuka halaman Hasil & Evaluasi Analisis LLM. Sistem menyajikan sebuah gateway pilihan aksi dengan dua jalur. Jalur pertama adalah Lihat Hasil Analisis LLM, di mana

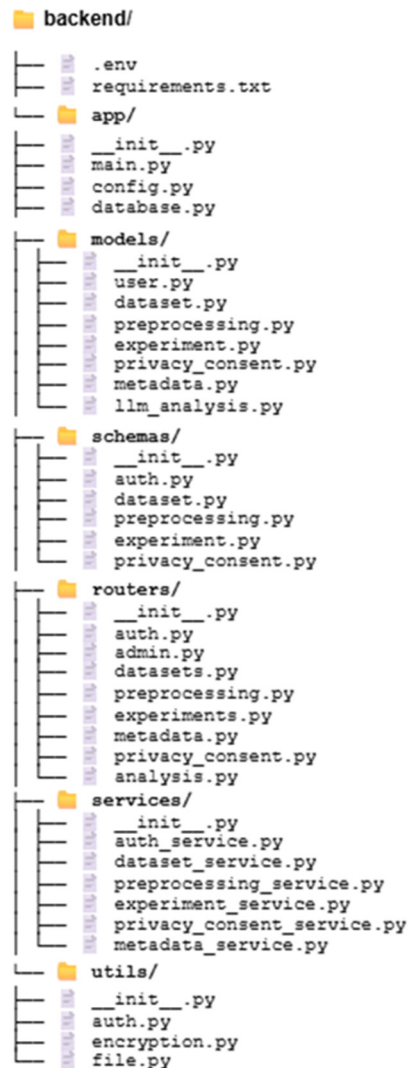
pengguna memilih satu hasil analisis LLM yang ingin dilihat secara detail. Jalur kedua adalah Bandingkan Hasil Analisis LLM, di mana pengguna memilih beberapa hasil analisis LLM untuk dibandingkan lalu menekan tombol Bandingkan.

Pada pool System (Backend), kedua jalur memiliki alur yang terpisah. Untuk jalur Lihat, sistem menerima parameter, mengambil data analisis LLM dari basis data, menghitung skor ROUGE hasil analisis tersebut, kemudian mengirimkan pesan Hasil Report ke Viewer. Untuk jalur Bandingkan, sistem menerima parameter, mengambil data beberapa eksperimen analisis dari basis data, menghitung skor ROUGE per hasil analisis, menyusun tabel perbandingan skor ROUGE antar model, kemudian mengirimkan pesan Tabel Perbandingan ke Viewer.

Viewer menerima pesan yang sesuai dengan jalurnya, kemudian melihat report atau tabel/grafik perbandingan antar hasil analisis LLM dari model yang berbeda. Selanjutnya terdapat gateway Pilih Aksi dengan dua pilihan: mengunduh laporan (.docx) atau menutup halaman. Proses berakhir setelah salah satu aksi tersebut dipilih.

3.6 Struktur Rancangan Implementasi

Struktur rancangan implementasi menggambarkan organisasi seluruh berkas kode back end yang dibangun beserta peran masing-masing berkas dalam sistem. Komponen back end diorganisasikan ke dalam satu direktori utama bernama backend/ yang di dalamnya terdapat direktori aplikasi app/ sebagai inti sistem. Struktur folder lengkap back end adalah sebagai berikut.



Gambar 3.15: Struktur Rancangan Folder Aplikasi Backend

Berkas Konfigurasi Akar. Berkas `.env` menyimpan seluruh variabel lingkungan yang bersifat sensitif dan tidak diunggah ke repositori, mencakup `DATABASE_URL`, `SECRET_KEY`, `ENCRYPTION_KEY`, `UPLOAD_DIR`, `PREPROCESSED_DIR`, `MODEL_DIR`, dan `MAX_FILE_SIZE_MB`. Berkas `requirements.txt` mendaftarkan seluruh library Python beserta versinya yang dibutuhkan untuk menjalankan aplikasi.

Direktori `app/` Inti Aplikasi. Berkas `main.py` merupakan titik masuk (entry point) aplikasi FastAPI. Berkas ini mendaftarkan seluruh router ke dalam aplikasi beserta prefix URL masing-masing, mengonfigurasi middleware CORS agar front end dapat mengakses API, mendaftarkan exception handler global untuk menangani kesalahan validasi (422), kesalahan HTTP, dan

kesalahan server (500) secara seragam, serta menjalankan fungsi `init_db()` pada saat aplikasi pertama kali dijalankan untuk membuat seluruh tabel basis data yang belum ada. Berkas `config.py` membaca variabel lingkungan dari berkas `.env` menggunakan `python-dotenv` dan mengekspornya sebagai konstanta Python yang digunakan oleh seluruh modul lain, seperti `DATABASE_URL`, `SECRET_KEY`, `ALGORITHM`, `ACCESS_TOKEN_EXPIRE_MINUTES`, `UPLOAD_DIR`, `MODEL_DIR`, dan `ML_SERVICE_PATH`. Berkas `database.py` mengonfigurasi koneksi ke basis data SQLite menggunakan `SQLAlchemy`, mendefinisikan engine, `SessionLocal`, dan `Base` yang diwarisi oleh seluruh model, serta menyediakan fungsi `get_db()` sebagai dependency FastAPI untuk manajemen sesi basis data dan fungsi `init_db()` untuk inisialisasi tabel.

Direktori `models/`. Direktori ini memuat definisi skema tabel basis data menggunakan `SQLAlchemy ORM`. Setiap berkas merepresentasikan satu atau lebih tabel. Berkas `user.py` mendefinisikan tabel `users` beserta enumerasi `role`. Berkas `dataset.py` mendefinisikan tabel `datasets`. Berkas `preprocessing.py` mendefinisikan tabel `preprocessings`. Berkas `experiment.py` mendefinisikan tabel `experiments`. Berkas `privacy_consent.py` mendefinisikan dua tabel sekaligus yaitu `privacy_consents` dan `privacy_consent_subjects`. Berkas `metadata.py` mendefinisikan tabel `metadata`. Berkas `llm_analysis.py` mendefinisikan tabel `llm_analyses`.

Direktori `schemas/`. Direktori ini memuat definisi skema validasi request dan response menggunakan `Pydantic`. Berkas `auth.py` mendefinisikan skema `LoginRequest`, `CreateUserRequest`, dan `UpdateUserRequest`. Berkas `dataset.py` mendefinisikan skema respons dataset. Berkas `preprocessing.py` mendefinisikan skema `PreprocessRequest` beserta tiga kelas konfigurasi turunannya yaitu `TabularPreprocessConfig`, `ImagePreprocessConfig`, dan `TextPreprocessConfig`, serta mendefinisikan daftar algoritma yang valid (`ALGORITHMS`) untuk setiap kombinasi tipe data dan jenis tugas. Berkas `experiment.py` mendefinisikan skema `ExperimentRequest`. Berkas `privacy_consent.py` mendefinisikan skema `PrivacyConsentRequest` beserta skema data subjek.

Direktori `routers/`. Direktori ini memuat definisi endpoint API yang dikelompokkan per domain fungsional. Setiap berkas router hanya bertugas menerima request, memeriksa otorisasi melalui `dependency require_role()`, meneruskan data ke service yang sesuai, dan mengembalikan respons — tanpa memuat logika bisnis di dalamnya. Berkas `auth.py` menangani endpo-

int /auth/login dan /auth/me. Berkas admin.py menangani operasi CRUD pengguna yang hanya dapat diakses oleh role admin. Berkas datasets.py menangani unggah dataset tiga tipe, pengambilan daftar dan detail dataset, analisis kualitas data, pratinjau gambar dan dokumen, perubahan status, serta penghapusan dataset. Berkas preprocessing.py menangani pembuatan pekerjaan preprocessing dan pengambilan statusnya. Berkas experiments.py menangani pembuatan eksperimen, pemantauan status, pengambilan hasil, dan pengunduhan model. Berkas metadata.py menangani pengambilan metadata per entitas. Berkas privacy_consent.py menangani unggah PDF bukti consent dan pengajuan data consent subjek. Berkas analysis.py menangani pengambilan daftar model Ollama yang tersedia, pembuatan pekerjaan analisis LLM, pengambilan daftar dan detail analisis, serta pemantauan status analisis.

Direktori services/. Direktori ini memuat seluruh logika bisnis sistem yang dipanggil oleh router. Berkas auth_service.py memuat logika verifikasi kredensial login, pembuatan token JWT, serta operasi CRUD pengguna oleh admin. Berkas dataset_service.py memuat logika penyimpanan berkas dataset ke direktori server, deteksi dataset role secara otomatis berdasarkan nama kolom, pembacaan metadata dan kualitas dataset, serta penghapusan berkas fisik. Berkas preprocessing_service.py memuat logika pembuatan pekerjaan preprocessing, validasi status dataset, pengecekan privacy consent, pemanggilan fungsi preprocessing dari ML Service sebagai background task, serta pembaruan status dan output path setelah proses selesai. Berkas experiment_service.py memuat logika pembuatan eksperimen, validasi algoritma terhadap tipe data, pemanggilan fungsi training dari ML Service sebagai background task, serta pembaruan status, metrik, dan model path setelah pelatihan selesai. Berkas privacy_consent_service.py memuat logika enkripsi NIK menggunakan Fernet, penentuan status dataset (uploaded atau locked) berdasarkan hasil persetujuan seluruh subjek, serta penghapusan file fisik dataset apabila persetujuan penyimpanan ditolak. Berkas metadata_service.py memuat logika penulisan metadata dan data lineage secara otomatis setiap kali terjadi operasi upload dataset, penyelesaian preprocessing, maupun penyelesaian pelatihan eksperimen.

Direktori utils/. Direktori ini memuat fungsi-fungsi pendukung yang digunakan lintas modul. Berkas auth.py memuat fungsi hash_password() untuk menghasilkan hash bcrypt, verify_password() untuk memverifikasi kata

sandi, `create_access_token()` untuk membuat token JWT, `get_current_user()` sebagai dependency FastAPI untuk memvalidasi token dan mengembalikan objek pengguna, serta `require_role()` sebagai dependency factory yang membungkus `get_current_user()` dengan pemeriksaan role tambahan. Berkas `encryption.py` memuat fungsi `encrypt_nik()` dan `decrypt_nik()` yang menggunakan algoritma Fernet simetris untuk mengenkripsi dan mendekripsi NIK subjek penelitian. Berkas `file.py` memuat fungsi-fungsi pengelolaan berkas yang mencakup validasi ekstensi dan ukuran file, penyimpanan file CSV dengan nama unik berbasis UUID, penyimpanan file TXT, serta penyimpanan dan ekstraksi file ZIP untuk dataset gambar beserta deteksi struktur folder kelas secara otomatis.

3.7 Pemilihan Teknologi

Pemilihan teknologi pada komponen back end sistem AutoML Platform didasarkan pada pertimbangan kesesuaian fungsi, kemudahan integrasi antar komponen, ketersediaan dokumentasi, dan kelayakan penggunaan dalam skala proyek akademik. Seluruh teknologi yang digunakan bersifat open-source dan tidak memerlukan lisensi berbayar. Berikut adalah penjelasan justifikasi pemilihan setiap teknologi yang digunakan.

Tabel 3.9: Teknologi yang Digunakan

No	Teknologi	Versi	Fungsi dalam Sistem
1	Python	3.12	Bahasa pemrograman utama back end
2	FastAPI	0.111.0	Framework REST API dengan dukungan async dan background task
3	Uvicorn	0.29.0	ASGI server untuk menjalankan aplikasi FastAPI
4	SQLAlchemy	2.0.30	ORM untuk interaksi dengan basis data SQLite
5	SQLite	—	Sistem manajemen basis data serverless
6	Pydantic	2.7.0	Validasi skema request dan response API
7	python-jose	3.3.0	Pembuatan dan verifikasi token JWT (HS256)
8	passlib & bcrypt	1.7.4 / 4.0.1	Hashing kata sandi pengguna
9	cryptography (Fernet)	—	Enkripsi NIK subjek penelitian
10	pandas	2.2.2	Pemrosesan dan analisis dataset tabular (CSV)
11	numpy	1.26.4	Komputasi numerik dan operasi array
12	scikit-learn	1.4.2	Preprocessing, pelatihan model, dan evaluasi
13	scikit-image	0.22.0	Ekstraksi fitur HOG untuk dataset gambar
14	Pillow	10.3.0	Pembacaan, konversi, dan resize gambar
15	nltk	3.8.1	Penghapusan stopwords untuk dataset teks
16	joblib	1.4.0	Serialisasi model terlatih ke format .pkl
17	python-multipart	0.0.9	Penanganan request unggah berkas
18	python-dotenv	1.0.1	Pengelolaan variabel lingkungan dari .env
19	Ollama	—	Engine LLM lokal untuk analisis teks

4. HASIL DAN PEMBAHASAN

4.1 Implementasi Modul Autentikasi & Manajemen Role

Modul autentikasi dan manajemen role merupakan modul fondasi yang menjadi penjaga akses seluruh sistem. Modul ini mengimplementasikan mekanisme autentikasi stateless berbasis JWT dengan algoritma HS256, hashing kata sandi menggunakan bcrypt, serta Role-Based Access Control (RBAC) yang memastikan setiap endpoint hanya dapat diakses oleh pengguna dengan role yang sesuai. Modul ini diimplementasikan melalui berkas `app/utils/auth.py`, `app/services/auth_service.py`, `app/routers/auth.py`, dan `app/routers/admin.py`.

4.1.1 Fungsi login()

Fungsi `login()` merupakan satu-satunya pintu masuk pengguna ke dalam sistem. Fungsi ini menerima email dan kata sandi, memverifikasi kecocokannya dengan data di basis data, memeriksa status aktif akun, lalu menghasilkan token JWT yang memuat `user_id` dan role pengguna sebagai payload. Berikut Pseudocode dan source code yang ditampilkan pada gambar 4.1.

```
def login(db: Session, email: str, password: str) -> dict:
    # Cari user berdasarkan email di database
    user = db.query(User).filter(User.email == email).first()

    # Tolak jika email tidak ditemukan atau password tidak cocok
    if not user or not verify_password(password, user.password_hash):
        raise HTTPException(
            status_code=401, detail="Email atau password salah")

    # Tolak jika akun dinonaktifkan oleh admin
    if not user.is_active:
        raise HTTPException(
            status_code=403, detail="Akun tidak aktif, hubungi admin")

    # Buat JWT token dengan payload user_id dan role
    token = create_access_token({"user_id": user.id, "role": user.role})
    return {
        "access_token": token,
        "token_type": "bearer",
        "user_id": user.id,
        "username": user.username,
        "role": user.role,
    }
```

Gambar 4.1: Code Fungsi Login

Pseudocode:

FUNCTION login(db, email, password)

user ← QUERY users WHERE email = email

IF user IS NULL OR verify_password(password, user.password_hash) IS FALSE THEN

RAISE HTTPException 401 "Email atau password salah"

END IF

IF user.is_active = 0 THEN

RAISE HTTPException 403 "Akun tidak aktif, hubungi admin"

END IF

payload ← {user_id: user.id, role: user.role}

token ← create_access_token(payload)

RETURN {access_token, token_type, user_id, username, role}

END FUNCTION

4.1.2 Fungsi get_current_user()

Fungsi get_current_user() merupakan dependency FastAPI yang dijalankan secara otomatis sebelum setiap handler endpoint yang membutuhkan au-

tentikasi. Fungsi ini membaca token JWT dari header Authorization, mendekode dan memverifikasi tanda tangan serta masa berlakunya, lalu mengambil objek pengguna dari basis data berdasarkan user_id yang terdapat di dalam payload token. Berikut Pseudocode dan source code yang ditampilkan pada gambar 4.2.

```
def get_current_user(
    credentials: HTTPAuthorizationCredentials = Depends(bearer_scheme),
    db: Session = Depends(get_db),
) -> User:
    token = credentials.credentials
    try:
        # Dekode dan verifikasi token JWT beserta masa berlakunya
        payload = jwt.decode(token, SECRET_KEY, algorithms=[ALGORITHM])
        user_id = payload.get("user_id")
        if user_id is None:
            raise HTTPException(
                status_code=401, detail="Token tidak valid")
    except JWTError:
        # Token tidak valid atau sudah melewati waktu kedaluwarsa
        raise HTTPException(
            status_code=401,
            detail="Token tidak valid atau sudah expired")

    # Pastikan user masih aktif di database
    user = db.query(User).filter(
        User.id == user_id, User.is_active == 1).first()
    if not user:
        raise HTTPException(
            status_code=401,
            detail="User tidak ditemukan atau tidak aktif")
    return user
```

Gambar 4.2: Code Fungsi get_current_user()

Pseudocode:

FUNCTION get_current_user(credentials, db)

token ← credentials.credentials

TRY

payload ← jwt.decode(token, SECRET_KEY, algorithm=HS256)

user_id ← payload.get("user_id")

IF user_id IS NULL THEN

RAISE HTTPException 401 "Token tidak valid"

END IF

CATCH JWTError

RAISE HTTPException 401 "Token tidak valid atau sudah expired"

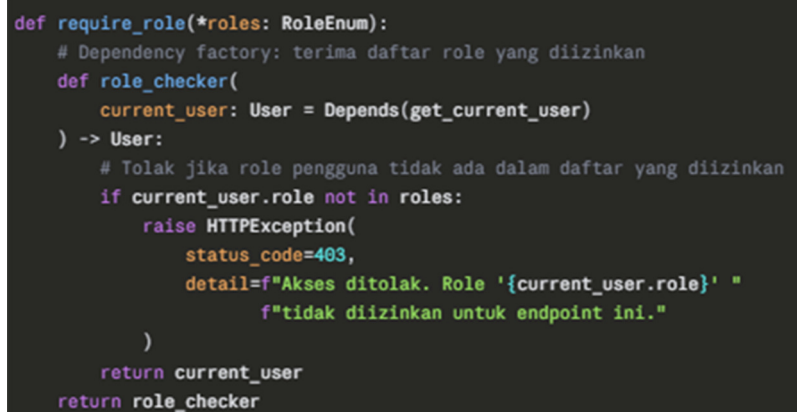
```

END TRY
user ← QUERY users WHERE id = user_id AND is_active = 1
IF user IS NULL THEN
  RAISE HTTPException 401 "User tidak ditemukan atau tidak aktif"
END IF
RETURN user
END FUNCTION

```

4.1.3 Fungsi require_role()

Fungsi `require_role()` merupakan dependency factory yang menghasilkan sebuah fungsi checker otorisasi berbasis role. Fungsi ini menerima satu atau lebih nama role yang diizinkan sebagai argumen, lalu menghasilkan fungsi checker yang memanggil `get_current_user()` dan memverifikasi apakah role pengguna yang aktif termasuk dalam daftar role yang diizinkan. Apabila tidak, sistem langsung mengembalikan respons HTTP 403 tanpa meneruskan request ke lapisan service. Berikut Pseudocode dan source code yang ditampilkan pada gambar 4.3.



```

def require_role(*roles: RoleEnum):
    # Dependency factory: terima daftar role yang diizinkan
    def role_checker(
        current_user: User = Depends(get_current_user)
    ) -> User:
        # Tolak jika role pengguna tidak ada dalam daftar yang diizinkan
        if current_user.role not in roles:
            raise HTTPException(
                status_code=403,
                detail=f"Akses ditolak. Role '{current_user.role}' "
                    f"tidak diizinkan untuk endpoint ini."
            )
        return current_user
    return role_checker

```

Gambar 4.3: Code Fungsi `require_role()`

Pseudocode:

```

FUNCTION require_role(*roles)
FUNCTION role_checker(current_user ← get_current_user())
IF current_user.role NOT IN roles THEN
  RAISE HTTPException 403 "Akses ditolak. Role tidak diizinkan"
END IF

```

```

RETURN current_user
END FUNCTION
RETURN role_checker
END FUNCTION

```

4.1.4 Fungsi create_user()

Fungsi create_user() hanya dapat dipanggil oleh pengguna dengan role admin. Fungsi ini menerima data pengguna baru, memeriksa keunikan email dan username, lalu menyimpan akun baru dengan kata sandi yang sudah di-hash menggunakan bcrypt ke basis data. Tidak ada fitur registrasi mandiri dalam system seluruh akun harus dibuat oleh admin. Berikut Pseudocode dan source code yang ditampilkan pada gambar 4.4.

```

def create_user(db: Session, payload: CreateUserRequest) -> User:
    # Periksa keunikan email dan username secara bersamaan
    existing = db.query(User).filter(
        (User.email == payload.email) |
        (User.username == payload.username)
    ).first()
    if existing:
        raise HTTPException(
            status_code=400,
            detail="Username atau email sudah digunakan")

    # Hash password sebelum disimpan, tidak pernah simpan plaintext
    user = User(
        username=payload.username,
        email=payload.email,
        password_hash=hash_password(payload.password),
        role=payload.role,
        is_active=1,
    )
    db.add(user)
    db.commit()
    db.refresh(user)
    return user

```

Gambar 4.4: Code Fungsi create_user()

Pseudocode:

```

FUNCTION create_user(db, payload)
existing ← QUERY users WHERE email = payload.email
OR username = payload.username
IF existing IS NOT NULL THEN

```

```

RAISE HTTPException 400 "Username atau email sudah digunakan"
END IF
hashed_password ← hash_password(payload.password)
user ← NEW User(
  username = payload.username,
  email = payload.email,
  password_hash = hashed_password,
  role = payload.role,
  is_active = 1
)
db.add(user)
db.commit()
db.refresh(user)
RETURN user
END FUNCTION

```

4.2 Implementasi Modul Dataset

Modul dataset bertanggung jawab atas seluruh siklus hidup dataset dalam sistem, mulai dari penerimaan dan validasi berkas yang diunggah, penyimpanan ke direktori server, deteksi dataset role secara otomatis, analisis kualitas data, hingga penghapusan dataset beserta berkas fisiknya. Modul ini diimplementasikan melalui berkas `app/utils/file.py`, `app/services/dataset_service.py`, dan `app/routers/datasets.py`. Hak akses dibagi berdasarkan role: unggah dataset hanya dapat dilakukan oleh `data_engineer`, penghapusan dan perubahan status hanya oleh `data_scientist`, sedangkan melihat daftar dan detail dataset dapat dilakukan oleh semua role yang telah terautentikasi.

4.2.1 Fungsi `_detect_dataset_role()`

Fungsi `_detect_dataset_role()` mendeteksi peran dataset secara otomatis berdasarkan nama-nama kolom yang terdapat dalam berkas CSV. Deteksi dilakukan dengan memeriksa keberadaan kata kunci verbatim, coding, dan analisis/analysis pada nama kolom yang telah dinormalisasi ke huruf kecil.

Hasil deteksi menentukan apakah dataset akan digunakan sebagai data referensi LLM, data baru yang akan dianalisis, data mentah, atau dataset machine learning umum. Berikut pseudocode dan source code yang ditampilkan pada Gambar 4.5.

```
def _detect_dataset_role(df) -> str:
    # Normalisasi nama kolom ke huruf kecil untuk pencocokan
    cols = [c.lower().strip() for c in df.columns]
    has_verbatim = any("verbatim" in c for c in cols)
    has_coding = any("coding" in c for c in cols)
    has_analisis = any("analisis" in c or "analysis" in c for c in cols)

    # Tentukan role berdasarkan kombinasi kolom yang ditemukan
    if has_verbatim and has_coding and has_analisis:
        return "llm_reference" # dataset contoh analisis lengkap
    elif has_verbatim and has_coding:
        return "llm_new" # dataset yang akan dianalisis
    elif has_verbatim:
        return "llm_raw" # dataset mentah hanya verbatim
    else:
        return "general" # dataset untuk training ML biasa
```

Gambar 4.5: Code Fungsi _detect_dataset_role()

Pseudocode:

```
FUNCTION _detect_dataset_role(df)
  cols ← [lowercase(col) FOR col IN df.columns]
  has_verbatim ← ANY col CONTAINS "verbatim" IN cols
  has_coding ← ANY col CONTAINS "coding" IN cols
  has_analisis ← ANY col CONTAINS "analisis" OR "analysis" IN cols
  IF has_verbatim AND has_coding AND has_analisis THEN
    RETURN "llm_reference"
  ELSE IF has_verbatim AND has_coding THEN
    RETURN "llm_new"
  ELSE IF has_verbatim THEN
    RETURN "llm_raw"
  ELSE
    RETURN "general"
  END IF
END FUNCTION
```

4.2.2 Fungsi `upload_tabular()`

Fungsi `upload_tabular()` menangani seluruh alur unggah dataset tabular dalam format CSV. Fungsi ini memanggil `save_tabular()` dari lapisan utilitas untuk menyimpan berkas fisik, kemudian membaca isinya menggunakan `pandas` untuk validasi lebih lanjut, mendeteksi dataset role secara otomatis, menyusun metadata ringkas, menyimpan record ke basis data, dan memanggil `write_dataset_metadata()` untuk mencatat data lineage awal. Apabila berkas tidak valid pada tahap apapun, berkas fisik yang sudah tersimpan akan dihapus sebelum pengecualian dilemparkan. Berikut pseudocode dan source code yang ditampilkan pada Gambar 4.6.

```
def upload_tabular(db: Session, user_id: int,
                  file: UploadFile, name: str) -> Dataset:
    # Simpan file fisik dan dapatkan path uniknya
    filepath, original_filename = save_tabular(file)

    try:
        # Baca CSV dengan separator auto-detect
        df = pd.read_csv(filepath, sep=None, engine="python")
    except Exception:
        os.remove(filepath)
        raise HTTPException(
            status_code=400, detail="File CSV tidak valid atau rusak")

    # Validasi isi CSV: tidak boleh kosong dan minimal 2 kolom
    if df.empty:
        os.remove(filepath)
        raise HTTPException(
            status_code=400, detail="File CSV tidak boleh kosong")
    if len(df.columns) < 2:
        os.remove(filepath)
        raise HTTPException(
            status_code=400,
            detail="File CSV harus memiliki minimal 2 kolom")

    # Deteksi role otomatis dan susun metadata ringkas
    dataset_role = _detect_dataset_role(df)
    meta = {"rows": len(df), "columns": list(df.columns)}

    dataset = Dataset(
        uploaded_by=user_id, name=name,
        data_type="tabular", filepath=filepath,
        original_filename=original_filename,
        status="uploaded", meta=meta, dataset_role=dataset_role,
    )
    db.add(dataset)
    db.commit()
    db.refresh(dataset)

    # Tulis metadata dan lineage secara otomatis setelah commit
    write_dataset_metadata(db, dataset)
    return dataset
```

Gambar 4.6: Code Fungsi `upload_tabular()`

Pseudocode:

```

FUNCTION upload_tabular(db, user_id, file, name)
  filepath, original_filename ← save_tabular(file)
  TRY
    df ← pd.read_csv(filepath)
  CATCH Exception
    DELETE file at filepath
    RAISE HTTPException 400 "File CSV tidak valid atau rusak"
  END TRY
  IF df IS EMPTY THEN
    DELETE file at filepath
    RAISE HTTPException 400 "File CSV tidak boleh kosong"
  END IF
  IF df.columns.count < 2 THEN
    DELETE file at filepath
    RAISE HTTPException 400 "File CSV harus memiliki minimal 2 kolom"
  END IF
  dataset_role ← _detect_dataset_role(df)
  meta ← {rows: len(df), columns: list(df.columns)}
  dataset ← NEW Dataset(
    uploaded_by = user_id,
    name = name,
    data_type = "tabular",
    filepath = filepath,
    original_filename = original_filename,
    status = "uploaded",
    meta = meta,
    dataset_role = dataset_role
  )
  db.add(dataset)
  db.commit()
  write_dataset_metadata(db, dataset)
  RETURN dataset
END FUNCTION

```

4.2.3 Fungsi save_image_zip()

Fungsi `save_image_zip()` menangani penyimpanan dan ekstraksi dataset gambar yang diunggah dalam format ZIP. Fungsi ini menyimpan berkas ZIP sementara ke direktori server, mengekstraknya ke folder tujuan, kemudian menghapus berkas ZIP asli untuk menghemat ruang penyimpanan. Setelah ekstraksi, fungsi memindai struktur folder hasil ekstraksi untuk mendeteksi label kelas secara otomatis, setiap subfolder dianggap sebagai satu kelas dan gambar-gambar di dalamnya dihitung. Apabila tidak ditemukan satu pun subfolder yang berisi gambar valid, folder hasil ekstraksi dihapus dan pengecualian dilemparkan. Berikut pseudocode dan source code yang ditampilkan pada Gambar 4.7.

```
def save_image_zip(file: UploadFile) -> tuple[str, dict]:
    # Validasi ekstensi dan ukuran file
    ext = os.path.splitext(file.filename)[1].lower()
    if ext not in ALLOWED_ZIP_EXT:
        raise HTTPException(status_code=400,
            detail="Manya file .zip yang diperbolehkan untuk dataset gambar")
    validate_file_size(file)

    # Simpan ZIP sementara ke direktori image_zip
    zip_path = _generate_path("image_zip", file.filename)
    with open(zip_path, "wb") as f:
        f.write(file.file.read())

    # Tentukan path ekstraksi
    extract_dir = zip_path.replace(".zip", "_extracted") \
        .replace("image_zip", "image")
    os.makedirs(extract_dir, exist_ok=True)

    try:
        with zipfile.ZipFile(zip_path, "r") as z:
            z.extractall(extract_dir)
    except zipfile.BadZipFile:
        os.remove(zip_path)
        shutil.rmtree(extract_dir, ignore_errors=True)
        raise HTTPException(status_code=400,
            detail="File ZIP tidak valid atau rusak")

    # Hapus ZIP setelah ekstraksi berhasil untuk hemat storage
    os.remove(zip_path)

    # Scan subfolder untuk deteksi label kelas secara otomatis
    classes, class_counts, total_images = [], {}, 0
    for item in sorted(os.listdir(extract_dir)):
        item_path = os.path.join(extract_dir, item)
        if os.path.isdir(item_path):
            images = [f for f in os.listdir(item_path)
                if os.path.splitext(f)[1].lower()
                in ALLOWED_IMAGE_EXT]
            if images:
                classes.append(item)
                class_counts[item] = len(images)
                total_images += len(images)

    # Tolak jika tidak ada subfolder kelas yang valid
    if not classes:
        shutil.rmtree(extract_dir, ignore_errors=True)
        raise HTTPException(status_code=400,
            detail="ZIP tidak mengandung struktur folder yang valid.")

    meta = {"classes": classes, "total_images": total_images,
        "class_counts": class_counts}
    return extract_dir, file.filename, meta
```

Gambar 4.7: Code Fungsi `save_image_zip()`

Pseudocode:

```

FUNCTION save_image_zip(file)
  IF file.extension NOT IN {".zip"} THEN
    RAISE HTTPException 400 "Hanya file .zip yang diperbolehkan"
  END IF
  validate_file_size(file)
  zip_path ← generate_unique_path("image_zip", file.filename)
  WRITE file.content TO zip_path
  extract_dir ← derive_extract_path(zip_path)
  TRY
    EXTRACT zip_path TO extract_dir
  CATCH BadZipFile
    DELETE zip_path
    DELETE extract_dir
    RAISE HTTPException 400 "File ZIP tidak valid atau rusak"
  END TRY
  DELETE zip_path
  classes ← []
  class_counts ← {}
  total_images ← 0
  FOR EACH folder IN extract_dir
    IF folder IS DIRECTORY THEN
      images ← [f FOR f IN folder IF f.extension IN ALLOWED_IMAGE_EXT]
      IF images IS NOT EMPTY THEN
        classes.append(folder.name)
        class_counts[folder.name] ← len(images)
        total_images += len(images)
      END IF
    END IF
  END FOR
  IF classes IS EMPTY THEN
    DELETE extract_dir
    RAISE HTTPException 400 "ZIP tidak mengandung struktur folder yang
valid"
  END IF
  meta ← {classes, total_images, class_counts}

```

RETURN extract_dir, file.filename, meta
END FUNCTION

4.2.4 Fungsi get_dataset_quality()

Fungsi get_dataset_quality() menganalisis kualitas dataset tabular dan menghasilkan laporan yang mencakup jumlah missing values per kolom, jumlah baris duplikat, tipe data setiap kolom, serta status keseluruhan dataset. Status per kolom ditentukan berdasarkan persentase missing values: ok apabila tidak ada, warning apabila di bawah 10%, dan error apabila melebihi 10%. Fungsi ini hanya tersedia untuk dataset bertipe tabular. Berikut pseudocode dan source code yang ditampilkan pada Gambar 4.8.

```
def get_dataset_quality(db: Session, dataset_id: int) -> dict:
    dataset = db.query(Dataset).filter(Dataset.id == dataset_id).first()
    if not dataset:
        raise HTTPException(
            status_code=404, detail="Dataset tidak ditemukan")

    # Fungsi ini hanya berlaku untuk dataset tabular
    if dataset.data_type != "tabular":
        raise HTTPException(status_code=400,
            detail="Analisis kualitas hanya tersedia untuk dataset tabular")

    df = pd.read_csv(dataset.filepath, sep=None, engine="python")

    total_rows = len(df)
    duplicate_rows = int(df.duplicated().sum())
    total_missing = int(df.isnull().sum().sum())

    columns_info = []
    for col in df.columns:
        missing_count = int(df[col].isnull().sum())
        missing_pct = round((missing_count / total_rows) * 100, 1) \
            if total_rows > 0 else 0.0

        # Klasifikasi tipe kolom
        if pd.api.types.is_numeric_dtype(df[col].dtype):
            col_type = "numerik"
        elif pd.api.types.is_datetime64_any_dtype(df[col].dtype):
            col_type = "datetime"
        else:
            col_type = "kategorikal"
```

Gambar 4.8: Code Fungsi get_dataset_quality()

Pseudocode:

```
FUNCTION get_dataset_quality(db, dataset_id)
    dataset ← QUERY datasets WHERE id = dataset_id
    IF dataset IS NULL THEN
        RAISE HTTPException 404 "Dataset tidak ditemukan"
    END IF
    IF dataset.data_type != "tabular" THEN
        RAISE HTTPException 400 "Hanya untuk dataset tabular"
```

```

END IF
df ← pd.read_csv(dataset.filepath)
total_rows ← len(df)
duplicate_rows ← df.duplicated().sum()
total_missing ← df.isnull().sum().sum()
columns_info ← []
FOR EACH col IN df.columns
missing_count ← df[col].isnull().sum()
missing_pct ← (missing_count / total_rows) * 100
IF missing_pct = 0 THEN
status ← "ok"
ELSE IF missing_pct ≤ 10 THEN
status ← "warning"
ELSE
status ← "error"
END IF
columns_info.append({name, type, missing_count, missing_pct, status})
END FOR
IF total_missing = 0 AND duplicate_rows = 0 THEN
overall_status ← "ok"
ELSE
overall_status ← "warning"
END IF
RETURN {total_rows, duplicate_rows, total_missing,
overall_status, columns_info}
END FUNCTION

```

4.3 Implementasi Modul Privacy Consent

Modul privacy consent mengimplementasikan mekanisme persetujuan privasi yang mengacu pada Undang-Undang Nomor 27 Tahun 2022 tentang Pelindungan Data Pribadi. Modul ini memastikan bahwa setiap dataset yang mengandung data subjek penelitian hanya dapat digunakan apabila seluruh subjek telah memberikan persetujuan secara tertulis. Data identitas subjek berupa NIK dienkripsi menggunakan algoritma

Fernet sebelum disimpan ke basis data sehingga tidak pernah tersimpan dalam bentuk teks biasa. Modul ini diimplementasikan melalui berkas `app/utils/encryption.py`, `app/services/privacy_consent_service.py`, dan `app/routers/privacy_consent.py`, dan hanya dapat diakses oleh pengguna dengan role `data_engineer`.

4.3.1 Fungsi `encrypt_nik()`

Fungsi `encrypt_nik()` mengenkripsi NIK subjek penelitian menggunakan algoritma Fernet sebelum disimpan ke basis data. Fernet merupakan enkripsi simetris berbasis AES-128-CBC dengan HMAC-SHA256 untuk verifikasi integritas, sehingga token hasil enkripsi tidak dapat dibaca tanpa kunci enkripsi yang benar. Kunci enkripsi dibaca dari variabel lingkungan `ENCRYPTION_KEY` yang tersimpan di berkas `.env` dan tidak pernah ditulis di dalam kode program. Berikut pseudocode dan source code yang ditampilkan pada Gambar 4.9.

```
def _get_fernet() -> Fernet:
    # Pastikan kunci enkripsi sudah dikonfigurasi di .env
    if not ENCRYPTION_KEY:
        raise ValueError("ENCRYPTION_KEY belum dikonfigurasi di .env")
    return Fernet(ENCRYPTION_KEY.encode())

def encrypt_nik(nik: str) -> str:
    # Enkripsi NIK menggunakan Fernet sebelum disimpan ke database
    f = _get_fernet()
    return f.encrypt(nik.encode()).decode()

def decrypt_nik(encrypted_nik: str) -> str:
    # Dekripsi NIK hanya untuk keperluan yang sah (misal: verifikasi admin)
    f = _get_fernet()
    return f.decrypt(encrypted_nik.encode()).decode()
```

Gambar 4.9: Code Fungsi `encrypt_nik()`

Pseudocode:

FUNCTION `encrypt_nik(nik)`

IF `ENCRYPTION_KEY` IS NULL THEN

RAISE `ValueError "ENCRYPTION_KEY belum dikonfigurasi"`

END IF

`fernet` ← `Fernet(ENCRYPTION_KEY)`

```

token ← fernet.encrypt(nik.encode())
RETURN token.decode()
END FUNCTION

```

4.3.2 Fungsi upload_consent_pdf()

Fungsi upload_consent_pdf() menangani unggah berkas PDF bukti tanda tangan persetujuan fisik dari setiap subjek penelitian. Berkas PDF disimpan dengan nama unik berbasis UUID4 untuk mencegah collision nama berkas. Fungsi ini dipisahkan dari fungsi pengiriman data consent agar front end dapat mengunggah PDF terlebih dahulu secara independen, kemudian menyertakan nama berkas yang dihasilkan saat mengirimkan data consent subjek. Berikut pseudocode dan source code yang ditampilkan pada Gambar 4.10.

```

@router.post("/upload-pdf")
async def upload_consent_pdf(
    file: UploadFile = File(...),
    current_user: User = Depends(require_role("data_engineer")),
):
    # Validasi ekstensi file harus PDF
    if not file.filename.lower().endswith(".pdf"):
        raise HTTPException(
            status_code=400, detail="Hanya file PDF yang diperbolehkan")

    # Beri nama unik berbasis UUID4 untuk hindari collision
    unique_name = f"{uuid.uuid4().hex}.pdf"
    filepath = os.path.join(PDF_UPLOAD_DIR, unique_name)

    content = await file.read()

    # Validasi ukuran maksimal 10MB
    if len(content) > 10 * 1024 * 1024:
        raise HTTPException(
            status_code=400, detail="Ukuran PDF maksimal 10MB")

    with open(filepath, "wb") as f:
        f.write(content)

    return {
        "status": "success",
        "data": {
            "pdf_filename": unique_name,
            "original_name": file.filename,
        }
    }

```

Gambar 4.10: Code Fungsi upload_consent_pdf()

Pseudocode:

```

FUNCTION upload_consent_pdf(file, current_user)
IF file.extension != ".pdf" THEN
RAISE HTTPException 400 "Hanya file PDF yang diperbolehkan"
END IF
content ← await file.read()
IF len(content) > 10MB THEN
RAISE HTTPException 400 "Ukuran PDF maksimal 10MB"
END IF
unique_name ← uuid4().hex + ".pdf"
filepath ← join(PDF_UPLOAD_DIR, unique_name)
WRITE content TO filepath
RETURN {pdf_filename: unique_name, original_name: file.filename}
END FUNCTION

```

4.3.3 Fungsi submit_consent()

Fungsi submit_consent() merupakan fungsi utama modul privacy consent yang menentukan nasib sebuah dataset berdasarkan hasil persetujuan seluruh subjek. Fungsi ini menerapkan tiga lapis pemeriksaan secara berurutan: pertama memeriksa keberadaan PDF bukti persetujuan, kedua memeriksa persetujuan penyimpanan data, dan ketiga memeriksa persetujuan pemrosesan data. Apabila salah satu dari dua pemeriksaan pertama gagal, dataset beserta berkas fisiknya dihapus permanen dari sistem. Apabila persetujuan pemrosesan tidak terpenuhi, dataset tetap disimpan namun statusnya dikunci (locked) sehingga tidak dapat digunakan untuk preprocessing maupun analisis. NIK setiap subjek dienkripsi menggunakan encrypt_nik() sebelum disimpan ke tabel privacy_consent_subjects. Berikut pseudocode dan source code yang ditampilkan pada Gambar 4.11.

```

def submit_consent(
    db: Session, user_id: int,
    dataset_id: int, subjects: list,
) -> dict:
    dataset = db.query(Dataset).filter(Dataset.id == dataset_id).first()
    if not dataset:
        raise HTTPException(
            status_code=404, detail="Dataset tidak ditemukan")

    # Periksa kondisi persetujuan dari seluruh subjek
    all_have_pdf = all(s.pdf_filename for s in subjects)
    all_agree_store = all(s.agree_store for s in subjects)
    all_agree_process = all(s.agree_process for s in subjects)

    def _delete_dataset():
        # Hapus file fisik dan record database secara bersamaan
        try:
            if dataset.data_type == "image":
                if os.path.isdir(dataset.filepath):
                    shutil.rmtree(dataset.filepath)
            else:
                if os.path.isfile(dataset.filepath):
                    os.remove(dataset.filepath)
        except Exception:
            pass
        db.delete(dataset)
        db.commit()

    # Lapis 1: tolak jika ada subjek tanpa bukti PDF
    if not all_have_pdf:
        _delete_dataset()
        return {
            "result": "rejected_store",
            "message": "Dataset tidak dapat disimpan karena ada subjek "
                        "yang tidak mengupload bukti PDF persetujuan.",
        }

```

Gambar 4.11: Code Fungsi submit_consent()

Pseudocode:

```

FUNCTION submit_consent(db, user_id, dataset_id, subjects)
dataset ← QUERY datasets WHERE id = dataset_id
IF dataset IS NULL THEN
    RAISE HTTPException 404 "Dataset tidak ditemukan"
END IF
all_have_pdf ← ALL subjects HAVE pdf_filename
all_agree_store ← ALL subjects HAVE agree_store = TRUE
all_agree_process ← ALL subjects HAVE agree_process = TRUE
// Lapis 1: periksa kelengkapan PDF
IF all_have_pdf IS FALSE THEN

```

```

DELETE dataset file AND record
RETURN {result: "rejected_store", message: "..."}
END IF
// Lapis 2: periksa persetujuan penyimpanan
IF all_agree_store IS FALSE THEN
DELETE dataset file AND record
RETURN {result: "rejected_store", message: "..."}
END IF
// Lapis 3: periksa persetujuan pemrosesan
IF all_agree_process THEN
dataset.status ← "uploaded"
consent_status_process ← "approved"
ELSE
dataset.status ← "locked"
consent_status_process ← "rejected"
END IF
db.commit()
consent ← NEW PrivacyConsent(
dataset_id = dataset_id,
submitted_by = user_id,
status_store = "approved",
status_process = consent_status_process
)
db.add(consent)
db.commit()
FOR EACH subject IN subjects
subject_record ← NEW PrivacyConsentSubject(
consent_id = consent.id,
nik_encrypted = encrypt_nik(subject.nik),
name = subject.name,
agree_store = subject.agree_store,
agree_process = subject.agree_process,
pdf_path = subject.pdf_filename
)
db.add(subject_record)
END FOR

```

```

db.commit()
IF all_agree_process THEN
RETURN {result: "approved", message: "Dataset siap digunakan"}
ELSE
RETURN {result: "locked", message: "Dataset terkunci"}
END IF
END FUNCTION

```

4.4 Implementasi Modul Preprocessing

Modul preprocessing bertanggung jawab menjalankan transformasi data-set mentah menjadi data siap latih dalam format yang dapat langsung digunakan oleh modul eksperimen. Modul ini mendukung tiga jalur preprocessing yang berbeda sesuai tipe dataset: tabular menggunakan ColumnTransformer, gambar menggunakan HOG (Histogram of Oriented Gradients), dan teks menggunakan TF-IDF. Seluruh proses preprocessing dijalankan secara asinkron sebagai background task sehingga API langsung mengembalikan respons tanpa menunggu proses selesai. Modul ini hanya dapat diakses oleh pengguna dengan role `data_scientist` dan diimplementasikan melalui berkas `app/services/preprocessing_service.py` dan `app/routers/preprocessing.py`.

4.4.1 Fungsi `create_preprocessing_job()`

Fungsi `create_preprocessing_job()` membuat record pekerjaan preprocessing baru di basis data setelah melewati serangkaian validasi. Validasi yang dilakukan mencakup pemeriksaan keberadaan dataset, pemeriksaan status privacy consent (dataset berstatus locked tidak dapat diproses), pemeriksaan kesesuaian antara tipe dataset dengan tipe konfigurasi yang dikirimkan, serta pemeriksaan bahwa dataset berstatus uploaded. Apabila seluruh validasi lolos, record preprocessing dibuat dengan status pending dan ID-nya dikembalikan untuk digunakan oleh background task. Berikut pseudocode dan source code yang ditampilkan pada Gambar 4.12.

```

def create_preprocessing_job(
    db: Session, user_id: int, dataset_id: int,
    config_type: str, name: str, config: dict
) -> Preprocessing:
    dataset = _get_dataset(db, dataset_id)

    # Dataset yang terkunci karena consent tidak boleh diproses
    if dataset.status == "locked":
        raise HTTPException(
            status_code=403,
            detail="Dataset tidak dapat diproses karena ada subjek "
                "yang tidak menyetujui pemrosesan data.")

    # Hanya dataset berstatus uploaded yang dapat diproses
    if dataset.status != "uploaded":
        raise HTTPException(
            status_code=400,
            detail=f"Dataset status '{dataset.status}' - hanya dataset "
                f"dengan status 'uploaded' yang bisa diproses")

    # Pastikan tipe config sesuai dengan tipe dataset
    if dataset.data_type != config_type:
        raise HTTPException(
            status_code=400,
            detail=f"Dataset bertipe '{dataset.data_type}' tapi "
                f"config bertipe '{config_type}'")

    # Buat record preprocessing dengan status awal pending
    prep = Preprocessing(
        dataset_id=dataset_id,
        processed_by=user_id,
        name=name,
        config={"config_type": config_type,
            **{k: v for k, v in config.items() if k != "name"}},
        status="pending",
    )
    db.add(prepare)
    db.commit()
    db.refresh(prepare)
    return prepare

```

Gambar 4.12: Code Fungsi create_preprocessing_job()

Pseudocode:

```

FUNCTION create_preprocessing_job(db, user_id, dataset_id,
    config_type, name, config)
    dataset ← QUERY datasets WHERE id = dataset_id
    IF dataset IS NULL THEN
        RAISE HTTPException 404 "Dataset tidak ditemukan"
    END IF
    IF dataset.status = "locked" THEN
        RAISE HTTPException 403 "Dataset tidak dapat diproses"
    END IF
    IF dataset.status != "uploaded" THEN
        RAISE HTTPException 400 "Dataset tidak dalam status uploaded"
    END IF

```

```

IF dataset.data_type != config_type THEN
  RAISE HTTPException 400 "Tipe dataset tidak sesuai konfigurasi"
END IF
prep ← NEW Preprocessing(
  dataset_id = dataset_id,
  processed_by = user_id,
  name = name,
  config = {config_type, ...config},
  status = "pending"
)
db.add(prepare)
db.commit()
RETURN prep
END FUNCTION

```

4.4.2 Fungsi run_preprocessing_job()

Fungsi `run_preprocessing_job()` merupakan fungsi utama yang dijalankan oleh FastAPI background task setelah record preprocessing berhasil dibuat. Fungsi ini membuka sesi basis data tersendiri karena berjalan di luar konteks request utama, memperbarui status menjadi running, lalu memanggil fungsi preprocessing yang sesuai dari komponen ML Service berdasarkan nilai `config_type`. Hasil yang dikembalikan oleh ML Service diperiksa untuk menentukan apakah proses berhasil atau gagal. Apabila berhasil, `output_path` disimpan, status dataset diperbarui menjadi `preprocessed`, dan metadata ditulis secara otomatis. Apabila gagal, pesan kesalahan disimpan dan status diperbarui menjadi `failed`. Berikut pseudocode dan source code yang ditampilkan pada Gambar 4.13.

```

def run_preprocessing_job(preprocessing_id: int):
    # Buka sesi DB baru karena fungsi ini berjalan di background
    if ML_SERVICE_PATH not in sys.path:
        sys.path.insert(0, ML_SERVICE_PATH)

    db = SessionLocal()
    try:
        prep = db.query(Preprocessing).filter(
            Preprocessing.id == preprocessing_id).first()
        if not prep:
            return

        dataset = db.query(Dataset).filter(
            Dataset.id == prep.dataset_id).first()

        # Update status menjadi running sebelum mulai proses
        prep.status = "running"
        db.commit()

        config_type = prep.config.get("config_type")
        os.makedirs(PREPROCESSED_DIR, exist_ok=True)
        output_path = os.path.join(
            PREPROCESSED_DIR, f"prep_{preprocessing_id}")

        # Panggil fungsi preprocessing dari ML Service sesuai tipe data
        if config_type == "tabular":
            from preprocessing.tabular import run_tabular_preprocessing
            result = run_tabular_preprocessing(
                dataset.filepath, prep.config, output_path)

        elif config_type == "image":
            # Kumpulkan semua path dataset gambar termasuk dataset tambahan
            dataset_paths = [dataset.filepath]
            for extra_id in prep.config.get("extra_dataset_ids", []):
                extra = db.query(Dataset).filter(
                    Dataset.id == extra_id).first()
                if extra:
                    dataset_paths.append(extra.filepath)

            from preprocessing.image import run_image_preprocessing
            result = run_image_preprocessing(
                dataset_paths, prep.config, output_path)

```

Gambar 4.13: Code Fungsi run_preprocessing_job()

Pseudocode:

FUNCTION run_preprocessing_job(preprocessing_id)

db ← NEW SessionLocal()

prep ← QUERY preprocessings WHERE id = preprocessing_id

IF prep IS NULL THEN RETURN END IF

dataset ← QUERY datasets WHERE id = prep.dataset_id

prep.status ← "running"

db.commit()

config_type ← prep.config.get("config_type")

output_path ← join(PREPROCESSED_DIR, "prep_{preprocessing_id}")

```

TRY
IF config_type = "tabular" THEN
result ← run_tabular_preprocessing(dataset.filepath,
prep.config, output_path)
ELSE IF config_type = "image" THEN
dataset_paths ← [dataset.filepath] + extra_dataset_paths
result ← run_image_preprocessing(dataset_paths,
prep.config, output_path)
ELSE IF config_type = "text" THEN
result ← run_text_preprocessing(dataset.filepath,
prep.config, output_path)
END IF
IF result.status != "completed" THEN
prep.status ← "failed"
prep.error_message ← result.error
ELSE
prep.status ← "completed"
prep.output_path ← output_path
dataset.status ← "preprocessed"
write_preprocessing_metadata(db, prep)
END IF
db.commit()
CATCH Exception AS e
prep.status ← "failed"
prep.error_message ← str(e)
db.commit()
FINALLY
db.close()
END TRY
END FUNCTION

```

4.4.3 Fungsi start_preprocessing() pada Router

Fungsi `start_preprocessing()` merupakan handler endpoint POST `/api/v2/preprocessing` yang mengintegrasikan pembuatan pekerjaan dan

pendaftaran background task dalam satu request. Setelah record preprocessing berhasil dibuat oleh `create_preprocessing_job()`, fungsi ini segera mendaftarkan `run_preprocessing_job()` sebagai background task menggunakan mekanisme `BackgroundTasks` dari FastAPI, kemudian langsung mengembalikan respons kepada klien tanpa menunggu proses preprocessing selesai. Berikut pseudocode dan source code yang ditampilkan pada Gambar 4.14.

```
@router.post("", status_code=201)
def start_preprocessing(
    payload: PreprocessRequest,
    background_tasks: BackgroundTasks,
    db: Session = Depends(get_db),
    current_user: User = Depends(require_role("data_scientist")),
):
    # Buat record preprocessing dan validasi semua syarat
    prep = create_preprocessing_job(
        db, current_user.id,
        payload.dataset_id,
        payload.config_type,
        payload.name,
        payload.config,
    )

    # Daftarkan background task - API langsung return tanpa tunggu selesai
    background_tasks.add_task(run_preprocessing_job, prep.id)

    return {
        "status": "success",
        "message": "Preprocessing dimulai",
        "data": {
            "preprocessing_id": prep.id,
            "dataset_id": prep.dataset_id,
            "status": prep.status,
        }
    }
```

Gambar 4.14: Code Fungsi `start_preprocessing()` pada Router

Pseudocode:

FUNCTION `start_preprocessing(payload, background_tasks, db, current_user)`

```
prep ← create_preprocessing_job(
    db, current_user.id,
    payload.dataset_id,
    payload.config_type,
    payload.name,
    payload.config
)
```

// Daftarkan background task tanpa menunggu selesai

```

background_tasks.add_task(run_preprocessing_job, prep.id)
RETURN {
    status: "success",
    message: "Preprocessing dimulai",
    preprocessing_id: prep.id,
    dataset_id: prep.dataset_id,
    status: prep.status
}
END FUNCTION

```

4.5 Implementasi Modul Eksperimen & Training

Modul eksperimen dan training bertanggung jawab mengelola seluruh siklus pelatihan model machine learning, mulai dari pembuatan konfigurasi eksperimen, validasi algoritma, eksekusi pelatihan sebagai background task, hingga penyimpanan metrik evaluasi dan berkas model terlatih. Sistem mendukung pelatihan untuk tiga tipe dataset dengan pilihan algoritma yang berbeda: dataset tabular mendukung classification dengan algoritma Decision Tree, Random Forest, SVM, KNN, Logistic Regression dan regression dengan algoritma Linear Regression, Decision Tree, Random Forest, SVR. Dataset gambar dan teks hanya mendukung classification dengan pilihan algoritma masing-masing. Modul ini hanya dapat diakses oleh pengguna dengan role `ml_engineer` dan diimplementasikan melalui berkas `app/services/experiment_service.py` dan `app/routers/experiments.py`.

4.5.1 Fungsi `create_experiment()`

Fungsi `create_experiment()` membuat record eksperimen baru setelah melewati serangkaian validasi. Validasi mencakup pemeriksaan keberadaan dan status preprocessing harus completed, penentuan `task_type` secara otomatis untuk dataset gambar dan teks selalu classification, serta validasi algoritma yang dipilih terhadap daftar algoritma yang diizinkan untuk kombinasi `data_type` dan `task_type` yang bersangkutan. Daftar algoritma yang valid didefinisikan dalam konstanta `ALGORITHMS` pada berkas `schemas/experiment.py`. Berikut pseudocode dan source code yang ditampilkan pada Gambar 4.15.

```

def create_experiment(db: Session, user_id: int, payload) -> Experiment:
    prep = db.query(Preprocessing).filter(
        Preprocessing.id == payload.preprocessing_id).first()
    if not prep:
        raise HTTPException(
            status_code=404, detail="Preprocessing tidak ditemukan")

    # Hanya preprocessing berstatus completed yang dapat dilatih
    if prep.status != "completed":
        raise HTTPException(
            status_code=400,
            detail=f"Preprocessing status '{prep.status}' - hanya yang "
                f"'completed' yang bisa ditraining")

    dataset = db.query(Dataset).filter(
        Dataset.id == prep.dataset_id).first()
    data_type = dataset.data_type

    # Dataset gambar dan teks selalu classification
    task_type = payload.task_type
    if data_type in ("image", "text"):
        task_type = "classification"
    elif not task_type:
        raise HTTPException(
            status_code=400,
            detail="task_type wajib diisi untuk dataset tabular")

    # Validasi algoritma terhadap daftar yang diizinkan
    allowed = ALGORITHMS.get(data_type, {}).get(task_type, [])
    if payload.algorithm not in allowed:
        raise HTTPException(
            status_code=400,
            detail=f"Algoritma '{payload.algorithm}' tidak valid untuk "
                f"'{data_type}/{task_type}'. Pilihan: {allowed}")

    # Buat record eksperimen dengan status awal queued
    exp = Experiment(
        preprocessing_id=payload.preprocessing_id,
        dataset_id=dataset.id,
        created_by=user_id,
        name=payload.name,
        data_type=data_type,
        task_type=task_type,
        algorithm=payload.algorithm,
        hyperparameters=payload.hyperparameters or {},
        status="queued",
        progress=0,
    )
    db.add(exp)
    db.commit()
    db.refresh(exp)
    return exp

```

Gambar 4.15: Code Fungsi create_experiment()

Pseudocode:

FUNCTION create_experiment(db, user_id, payload)

prep ← QUERY preprocessings WHERE id = payload.preprocessing_id

IF prep IS NULL THEN

RAISE HTTPException 404 "Preprocessing tidak ditemukan"

END IF

IF prep.status != "completed" THEN

RAISE HTTPException 400 "Preprocessing belum selesai"

END IF

dataset ← QUERY datasets WHERE id = prep.dataset_id

data_type ← dataset.data_type

// Auto-set task_type untuk image dan text

```

IF data_type IN ("image", "text") THEN
task_type ← "classification"
ELSE IF payload.task_type IS NULL THEN
RAISE HTTPException 400 "task_type wajib diisi untuk dataset tabular"
ELSE
task_type ← payload.task_type
END IF
// Validasi algoritma terhadap daftar yang diizinkan
allowed ← ALGORITHMS[data_type][task_type]
IF payload.algorithm NOT IN allowed THEN
RAISE HTTPException 400 "Algoritma tidak valid"
END IF
exp ← NEW Experiment(
preprocessing_id = payload.preprocessing_id,
dataset_id = dataset.id,
created_by = user_id,
name = payload.name,
data_type = data_type,
task_type = task_type,
algorithm = payload.algorithm,
hyperparameters = payload.hyperparameters,
status = "queued",
progress = 0
)
db.add(exp)
db.commit()
RETURN exp
END FUNCTION

```

4.5.2 Fungsi run_training_job()

Fungsi `run_training_job()` merupakan fungsi utama yang dijalankan oleh FastAPI background task untuk menjalankan pelatihan model. Fungsi ini membuka sesi basis data tersendiri, memperbarui status dan progres eksperimen secara bertahap selama proses berlangsung, menyusun konfigurasi pelatihan, lalu memanggil fungsi `run_training()` dari komponen ML Service.

Progres diperbarui secara bertahap untuk memberikan informasi kemajuan yang dapat dipantau melalui endpoint status. Apabila pelatihan berhasil, metrik evaluasi dan path model disimpan ke basis data dan metadata ditulis secara otomatis. Berikut pseudocode dan source code yang ditampilkan pada Gambar 4.16.

```

def run_training_job(experiment_id: int):
    if ML_SERVICE_PATH not in sys.path:
        sys.path.insert(0, ML_SERVICE_PATH)

    db = SessionLocal()
    try:
        exp = db.query(Experiment).filter(
            Experiment.id == experiment_id).first()
        if not exp:
            return

        prep = db.query(Preprocessing).filter(
            Preprocessing.id == exp.preprocessing_id).first()

        # Update status menjadi running dan set progres awal
        exp.status = "running"
        exp.progress = 10
        db.commit()

        os.makedirs(MODEL_DIR, exist_ok=True)
        model_save_path = os.path.join(
            MODEL_DIR, f"exp_{experiment_id}.pkl")

        # Susun konfigurasi yang dikirim ke ML Service
        config = {
            "preprocessed_path": prep.output_path,
            "data_type": exp.data_type,
            "task_type": exp.task_type,
            "algorithm": exp.algorithm,
            "hyperparameters": exp.hyperparameters or {},
            "model_save_path": model_save_path,
        }

        exp.progress = 55
        db.commit()

        # Panggil fungsi training dari ML Service
        from training.pipeline import run_training
        result = run_training(config)

        exp.progress = 90
        db.commit()

        # Periksa hasil training dari ML Service
        if not result or result.get("status") != "completed":
            exp.status = "failed"
            exp.error_message = (result or {}).get("error", "Training gagal")
        else:
            exp.status = "completed"
            exp.progress = 100
            exp.metrics = result.get("metrics")
            exp.model_path = result.get("model_path")
            # Tulis metadata dan lineage otomatis setelah training selesai
            write_experiment_metadata(db, exp)

        db.commit()

    except Exception as e:
        # Tangkap semua exception dan simpan pesan error ke database
        exp = db.query(Experiment).filter(
            Experiment.id == experiment_id).first()
        if exp:
            exp.status = "failed"
            exp.error_message = str(e)
            db.commit()
    finally:
        db.close()

```

Gambar 4.16: Code Fungsi run_training_job()

Pseudocode:

FUNCTION run_training_job(experiment_id)

db ← NEW SessionLocal()

exp ← QUERY experiments WHERE id = experiment_id

```

IF exp IS NULL THEN RETURN END IF
prep ← QUERY preprocessings WHERE id = exp.preprocessing_id
// Update status dan progres awal
exp.status ← "running"
exp.progress ← 10
db.commit()
model_save_path ← join(MODEL_DIR, "exp_{experiment_id}.pkl")
// Susun konfigurasi training
config ← {
  preprocessed_path: prep.output_path,
  data_type: exp.data_type,
  task_type: exp.task_type,
  algorithm: exp.algorithm,
  hyperparameters: exp.hyperparameters,
  model_save_path: model_save_path
}
exp.progress ← 55
db.commit()
// Panggil ML Service
result ← run_training(config)
exp.progress ← 90
db.commit()
IF result IS NULL OR result.status != "completed" THEN
  exp.status ← "failed"
  exp.error_message ← result.error
ELSE
  exp.status ← "completed"
  exp.progress ← 100
  exp.metrics ← result.metrics
  exp.model_path ← result.model_path
  write_experiment_metadata(db, exp)
END IF
db.commit()
CATCH Exception AS e
  exp.status ← "failed"
  exp.error_message ← str(e)

```

```

db.commit()
FINALLY
db.close()
END FUNCTION

```

4.5.3 Fungsi download_model()

Fungsi download_model() memungkinkan pengguna dengan role viewer, ml_engineer, atau admin untuk mengunduh berkas model terlatih dalam format .pkl langsung dari server. Fungsi ini terlebih dahulu memverifikasi bahwa eksperimen berstatus completed dan berkas model masih tersedia di direktori server sebelum mengembalikan respons berupa file stream. Berikut pseudocode dan source code yang ditampilkan pada Gambar 4.17.

```

@router.get("/{exp_id}/model/download")
def download_model(
    exp_id: int,
    db: Session = Depends(get_db),
    current_user: User = Depends(
        require_role("viewer", "ml_engineer", "admin")),
):
    e = get_experiment_by_id(db, exp_id)

    # Model hanya bisa diunduh jika training sudah selesai
    if e.status != "completed":
        raise HTTPException(
            status_code=400, detail="Model belum tersedia")

    # Pastikan file model masih ada di server
    if not e.model_path or not os.path.exists(e.model_path):
        raise HTTPException(
            status_code=404,
            detail="File model tidak ditemukan di server")

    # Kembalikan file sebagai stream unduhan
    return FileResponse(
        path=e.model_path,
        filename=f"{e.name}.pkl",
        media_type="application/octet-stream",
    )

```

Gambar 4.17: Code Fungsi download_model()

Pseudocode:

```

FUNCTION download_model(exp_id, db, current_user)
exp ← get_experiment_by_id(db, exp_id)
IF exp.status != "completed" THEN
    RAISE HTTPException 400 "Model belum tersedia"

```

```

END IF
IF exp.model_path IS NULL OR NOT file_exists(exp.model_path) THEN
RAISE HTTPException 404 "File model tidak ditemukan di server"
END IF
RETURN FileResponse(
    path = exp.model_path,
    filename = "{exp.name}.pkl",
    mediatype = "application/octet-stream"
)
END FUNCTION

```

4.6 Implementasi Modul Analisis LLM

Modul analisis LLM menyediakan fitur analisis teks menggunakan model bahasa besar yang berjalan secara lokal melalui Ollama. Studi kasus yang diimplementasikan adalah analisis verbatim dan coding psikologi klinis, di mana sistem menggunakan dataset referensi yang sudah memiliki contoh analisis sebagai panduan gaya penulisan (few-shot prompting) untuk menghasilkan analisis pada data baru. Sistem mendukung tiga model LLM yang dapat dipilih pengguna, yaitu mistral:7b, llama3.2:3b, dan qwen2.5:7b. Seluruh proses generate dijalankan secara asinkron sebagai background task dan hasil analisis dievaluasi menggunakan skor ROUGE. Modul ini diimplementasikan seluruhnya dalam berkas `app/routers/analysis.py` dan hanya dapat diakses oleh pengguna dengan role `data_scientist`.

4.6.1 Fungsi `_build_prompt()`

Fungsi `_build_prompt()` menyusun prompt yang akan dikirimkan ke Ollama menggunakan teknik few-shot prompting. Fungsi ini mengambil tiga baris pertama dari dataset referensi sebagai contoh analisis, lalu menyusunnya bersama data baru yang akan dianalisis dalam satu teks prompt yang terstruktur. Format prompt berbeda tergantung pada dataset role data baru: apabila bertipe `llm_raw` hanya memiliki verbatim tanpa coding, model diminta menghasilkan coding sekaligus analisis; apabila bertipe `llm_new` sudah memiliki verbatim dan coding, model hanya diminta menghasilkan ana-

lisis. Berikut pseudocode dan source code yang ditampilkan pada Gambar 4.18.

```
def _build_prompt(reference_rows: list, new_rows: list,
                  dataset_role: str) -> str:
    # Ambil 3 baris pertama sebagai few-shot examples
    examples = reference_rows[:3]

    prompt = ""Kamu adalah psikolog klinis profesional yang berpengalaman \
dalam analisis kualitatif.\
Tugasmu adalah membuat analisis psikologis berdasarkan verbatim wawancara \
dan coding temanya.\
Tulis analisis dalam Bahasa Indonesia yang formal dan akademis.\n\n""

    prompt += "Berikut contoh analisis yang sudah ada sebagai referensi " \
    "gaya penulisan:\n\n"

    # Sisipkan contoh analisis dari dataset referensi
    for i, ex in enumerate(examples, 1):
        prompt += f"CONTOH {i}:\n"
        prompt += f"Verbatim : {ex.get('verbatim', '')}\n"
        prompt += f"Coding   : {ex.get('coding', '')}\n"
        prompt += f"Analisis  : {ex.get('analisis', '')}\n"

    prompt += "-" * 50 + "\n\n"

    # Format prompt berbeda tergantung dataset_role data baru
    if dataset_role == "llm_raw":
        # Data mentah: model harus generate coding + analisis
        prompt += "Sekarang buatlah CODING dan ANALISIS untuk setiap " \
        "segmen berikut:\n\n"
        for i, row in enumerate(new_rows, 1):
            prompt += f"SEGMENT {i}:\n" \
            f"Verbatim : {row.get('verbatim', '')}\n\n"
            prompt += f"Format output:\nSEGMENT 1:\nCODING: [coding]\n" \
            "ANALISIS: [analisis]\n\n"
        else:
            # Data baru: model hanya perlu generate analisis
            prompt += "Sekarang buatlah ANALISIS untuk setiap segmen berikut:\n\n"
            for i, row in enumerate(new_rows, 1):
                prompt += f"SEGMENT {i}:\n" \
                f"Verbatim : {row.get('verbatim', '')}\n" \
                f"Coding   : {row.get('coding', '')}\n\n"
                prompt += f"Format output:\nSEGMENT 1:\nANALISIS: [analisis]\n\n" \
                "SEGMENT 2:\nANALISIS: [analisis]\n\n"

    return prompt
```

Gambar 4.18: Code Fungsi _build_prompt()

Pseudocode:

```
FUNCTION _build_prompt(reference_rows, new_rows, dataset_role)
examples ← reference_rows[:3]
prompt ← "Kamu adalah psikolog klinis profesional..."
prompt += "Berikut contoh analisis sebagai referensi:\n"
FOR i, ex IN enumerate(examples)
prompt += "CONTOH {i}:\n"
```

```

prompt += "Verbatim : {ex.verbatim}\n"
prompt += "Coding : {ex.coding}\n"
prompt += "Analisis : {ex.analisis}\n\n"
END FOR
IF dataset_role = "llm_raw" THEN
prompt += "Buatkan CODING dan ANALISIS untuk setiap segmen:\n"
FOR i, row IN enumerate(new_rows)
prompt += "SEGMENT {i}:\nVerbatim: {row.verbatim}\n"
END FOR
prompt += "Format: SEGMENT 1:\nCODING: [...]\nANALISIS: [...]\n"
ELSE
prompt += "Buatkan ANALISIS untuk setiap segmen:\n"
FOR i, row IN enumerate(new_rows)
prompt += "SEGMENT {i}:\nVerbatim: {row.verbatim}\nCoding:
{row.coding}\n"
END FOR
prompt += "Format: SEGMENT 1:\nANALISIS: [...]\n"
END IF
RETURN prompt
END FUNCTION

```

4.6.2 Fungsi _run_analysis()

Fungsi _run_analysis() merupakan fungsi background task utama yang mengorkestrasi seluruh alur analisis LLM secara asinkron. Fungsi ini membaca kedua dataset referensi dan data baru, menyusun prompt menggunakan _build_prompt(), mengirimkan prompt ke Ollama melalui HTTP request asinkron menggunakan httpx, mengukur waktu generate, mem-parse output mentah Ollama menjadi struktur data terstruktur, menghitung skor ROUGE dan rata-rata panjang analisis, lalu menyimpan seluruh hasil ke basis data. Berikut pseudocode dan source code yang ditampilkan pada Gambar 4.19.

```

async def _run_analysis(db, analysis_id: int):
    analysis = db.query(LLMAnalysis).filter(
        LLMAnalysis.id == analysis_id).first()
    if not analysis:
        return

    try:
        analysis.status = "running"
        db.commit()

        # Ambil kedua dataset dari database
        ref_dataset = db.query(Dataset).filter(
            Dataset.id == analysis.dataset_reference_id).first()
        new_dataset = db.query(Dataset).filter(
            Dataset.id == analysis.dataset_new_id).first()

        if not ref_dataset or not new_dataset:
            raise Exception("Dataset tidak ditemukan")

        # Baca isi kedua dataset menjadi list of dict
        ref_rows = _read_dataset_rows(
            ref_dataset.filepath, ref_dataset.data_type, "llm_reference")
        new_rows = _read_dataset_rows(
            new_dataset.filepath, new_dataset.data_type,
            new_dataset.dataset_role)

        if not ref_rows:
            raise Exception("Dataset referensi tidak dapat dibaca")
        if not new_rows:
            raise Exception("Dataset baru tidak dapat dibaca")

        # Susun prompt dengan teknik few-shot prompting
        prompt = _build_prompt(ref_rows, new_rows, new_dataset.dataset_role)

```

Gambar 4.19: Code Fungsi _run_analysis()

Pseudocode:

```

ASYNC FUNCTION _run_analysis(db, analysis_id)
analysis ← QUERY llm_analyses WHERE id = analysis_id
IF analysis IS NULL THEN RETURN END IF
TRY
analysis.status ← "running"
db.commit()
ref_dataset ← QUERY datasets WHERE id = analysis.dataset_reference_id
new_dataset ← QUERY datasets WHERE id = analysis.dataset_new_id
IF ref_dataset IS NULL OR new_dataset IS NULL THEN
RAISE Exception "Dataset tidak ditemukan"
END IF
ref_rows ← _read_dataset_rows(ref_dataset.filepath,
ref_dataset.data_type, "llm_reference")

```

```

new_rows ← _read_dataset_rows(new_dataset.filepath,
new_dataset.data_type,
new_dataset.dataset_role)
IF ref_rows IS EMPTY THEN
RAISE Exception "Dataset referensi tidak dapat dibaca"
END IF
IF new_rows IS EMPTY THEN
RAISE Exception "Dataset baru tidak dapat dibaca"
END IF
prompt          ←          _build_prompt(ref_rows,          new_rows,
new_dataset.dataset_role)
start_time ← time.now()
// Kirim prompt ke Ollama secara asinkron
response ← AWAIT httpx.post(OLLAMA_URL, {
model: analysis.model_name,
prompt: prompt,
stream: False
}, timeout=300)
raw_output ← response.json().response
generate_time ← time.now() - start_time
results      ←          _parse_output(raw_output,          new_rows,
new_dataset.dataset_role)
rouge_scores ← _calculate_rouge(results, ref_rows)
avg_length ← _calculate_avg_length(results)
analysis.results ← json.dumps(results)
analysis.rouge_scores ← json.dumps(rouge_scores)
analysis.generate_time ← generate_time
analysis.avg_length ← avg_length
analysis.status ← "completed"
CATCH Exception AS e
analysis.status ← "failed"
analysis.error_message ← str(e)
FINALLY
db.commit()
END TRY
END FUNCTION

```

4.6.3 Fungsi `_calculate_rouge()`

Fungsi `_calculate_rouge()` menghitung skor evaluasi ROUGE (Recall-Oriented Understudy for Gisting Evaluation) antara teks analisis yang dihasilkan model dengan teks analisis dari dataset referensi. Skor yang dihitung mencakup ROUGE-1 (kecocokan unigram), ROUGE-2 (kecocokan bigram), dan ROUGE-L (kecocokan longest common subsequence). Seluruh teks analisis dari hasil generate dan referensi digabungkan terlebih dahulu sebelum dihitung skornya secara global. Berikut pseudocode dan source code yang ditampilkan pada Gambar 4.20.

```
def _calculate_rouge(results: list, reference_rows: list) -> dict:
    try:
        from rouge_score import rouge_scorer
        scorer = rouge_scorer.RougeScorer(
            ["rouge1", "rouge2", "rougeL"], use_stemmer=False)

        # Kumpulkan teks analisis dari referensi dan hasil generate
        ref_texts = [r.get("analisis", "")
                     for r in reference_rows if r.get("analisis")]
        hyp_texts = [r.get("analisis", "")
                     for r in results if r.get("analisis")]

        if not ref_texts or not hyp_texts:
            return {}

        # Gabungkan semua teks untuk perhitungan skor global
        ref_combined = " ".join(ref_texts)
        hyp_combined = " ".join(hyp_texts)

        scores = scorer.score(ref_combined, hyp_combined)
        return {
            "rouge1": round(scores["rouge1"].fmeasure, 4),
            "rouge2": round(scores["rouge2"].fmeasure, 4),
            "rougeL": round(scores["rougeL"].fmeasure, 4),
        }
    except Exception as e:
        print(f"Error hitung ROUGE: {e}")
        return {}
```

Gambar 4.20: Code Fungsi `_calculate_rouge()`

Pseudocode:

FUNCTION `_calculate_rouge(results, reference_rows)`

TRY

scorer ← `RougeScorer(["rouge1", "rouge2", "rougeL"])`

ref_texts ← [r.analisis FOR r IN reference_rows IF r.analisis]

hyp_texts ← [r.analisis FOR r IN results IF r.analisis]

IF ref_texts IS EMPTY OR hyp_texts IS EMPTY THEN

RETURN {}

```

END IF
ref_combined ← join(ref_texts, " ")
hyp_combined ← join(hyp_texts, " ")
scores ← scorer.score(ref_combined, hyp_combined)
RETURN {
  rouge1: round(scores.rouge1.fmeasure, 4),
  rouge2: round(scores.rouge2.fmeasure, 4),
  rougeL: round(scores.rougeL.fmeasure, 4)
}
CATCH Exception
RETURN {}
END TRY
END FUNCTION

```

4.6.4 Fungsi generate_analysis() pada Router

Fungsi generate_analysis() merupakan handler endpoint POST /api/v2/analysis/generate yang memvalidasi semua prasyarat sebelum memulai proses analisis. Validasi mencakup pemeriksaan keberadaan dan role dataset referensi, pemeriksaan role dataset baru, pemeriksaan status consent dataset baru yang tidak boleh berstatus locked, serta pemeriksaan ketersediaan model Ollama yang dipilih. Setelah validasi lolos, record analisis dibuat dengan status pending dan _run_analysis() didaftarkan sebagai background task. Berikut pseudocode dan source code yang ditampilkan pada Gambar 4.21.

```

@router.post("/generate")
async def generate_analysis(
    payload: AnalysisRequest,
    background_tasks: BackgroundTasks,
    db: Session = Depends(get_db),
    current_user: User = Depends(require_role("data_scientist")),
):
    # Validasi dataset referensi
    ref_dataset = db.query(Dataset).filter(
        Dataset.id == payload.dataset_reference_id).first()
    if not ref_dataset:
        raise HTTPException(
            status_code=404, detail="Dataset referensi tidak ditemukan")
    if ref_dataset.dataset_role != "llm_reference":
        raise HTTPException(status_code=400,
            detail="Dataset referensi harus bertipe llm_reference")

    # Validasi dataset baru
    new_dataset = db.query(Dataset).filter(
        Dataset.id == payload.dataset_new_id).first()
    if not new_dataset:
        raise HTTPException(
            status_code=404, detail="Dataset baru tidak ditemukan")
    if new_dataset.dataset_role not in ("llm_new", "llm_raw"):
        raise HTTPException(status_code=400,
            detail="Dataset baru harus bertipe llm_new atau llm_raw")

    # Tolak jika dataset terkunci karena consent
    if new_dataset.status == "locked":
        raise HTTPException(status_code=403,
            detail="Dataset baru terkunci karena consent tidak disetujui")

    # Validasi model tersedia di Ollama
    available = _get_available_models()
    if payload.model_name not in available:
        raise HTTPException(status_code=400,
            detail=f"Model '{payload.model_name}' tidak tersedia di Ollama")

    # Buat record analisis dengan status awal pending
    analysis = LLMAnalysis(
        dataset_reference_id=payload.dataset_reference_id,
        dataset_new_id=payload.dataset_new_id,
        created_by=current_user.id,
        model_name=payload.model_name,
        status="pending",
    )
    db.add(analysis)
    db.commit()
    db.refresh(analysis)

    # Daftarkan background task - API langsung return tanpa tunggu selesai
    background_tasks.add_task(_run_analysis, db, analysis.id)

    return {
        "status": "success",
        "message": f"Analisis dimulai dengan model {payload.model_name}",
        "data": {
            "analysis_id": analysis.id,
            "model_name": analysis.model_name,
            "status": analysis.status,
        }
    }

```

Gambar 4.21: Code Fungsi generate_analysis() pada Router

Pseudocode:

ASYNC FUNCTION generate_analysis(payload, background_tasks, db,
current_user)
ref_dataset ← QUERY datasets WHERE id = paylo-

```

ad.dataset_reference_id
  IF ref_dataset IS NULL THEN
    RAISE HTTPException 404 "Dataset referensi tidak ditemukan"
  END IF
  IF ref_dataset.dataset_role != "llm_reference" THEN
    RAISE HTTPException 400 "Dataset referensi harus bertipe
llm_reference"
  END IF
  new_dataset ← QUERY datasets WHERE id = payload.dataset_new_id
  IF new_dataset IS NULL THEN
    RAISE HTTPException 404 "Dataset baru tidak ditemukan"
  END IF
  IF new_dataset.dataset_role NOT IN ("llm_new", "llm_raw") THEN
    RAISE HTTPException 400 "Dataset baru harus bertipe llm_new atau
llm_raw"
  END IF
  IF new_dataset.status = "locked" THEN
    RAISE HTTPException 403 "Dataset terkunci karena consent tidak dise-
tujui"
  END IF
  available ← _get_available_models()
  IF payload.model_name NOT IN available THEN
    RAISE HTTPException 400 "Model tidak tersedia di Ollama"
  END IF
  analysis ← NEW LLMAnalysis(
    dataset_reference_id = payload.dataset_reference_id,
    dataset_new_id = payload.dataset_new_id,
    created_by = current_user.id,
    model_name = payload.model_name,
    status = "pending"
  )
  db.add(analysis)
  db.commit()
  background_tasks.add_task(_run_analysis, db, analysis.id)
  RETURN {analysis_id, model_name, status: "pending"}
END FUNCTION

```

4.7 Pengujian

Pengujian sistem dilakukan menggunakan metode white box testing, yaitu pengujian yang dilakukan dari sisi developer dengan cara memverifikasi bahwa setiap fungsi utama sistem bekerja sesuai dengan logika internal yang telah dirancang. Pengujian tidak hanya memeriksa respons API yang dikembalikan kepada klien, tetapi juga memverifikasi kondisi internal sistem seperti record yang tersimpan di basis data, berkas fisik yang tersimpan di server, serta perubahan status entitas setelah operasi dijalankan. Pengujian dilakukan pada seluruh modul yang diimplementasikan, mencakup modul autentikasi, dataset, privacy consent, preprocessing, eksperimen & training, dan analisis LLM.

4.7.1 Pengujian Modul Autentikasi dan Manajemen Role

Pengujian whitebox testing pada modul autentikasi dan manajemen role ditunjukkan pada tabel 4.1 berikut

Tabel 4.1: Pengujian Modul Autentikasi

Fungsi yang Diuji	Skenario	Data Input	Hasil yang Diharapkan	Hasil Pengujian
login()	Login dengan kredensial valid	email: admin@test.com, password: admin123	Respons HTTP 200, token JWT dikembalikan, token memuat user_id dan role yang benar	Sesuai
login()	Login dengan email tidak terdaftar	email: tidak-da@test.com, password: apapun	Respons HTTP 401, pesan "Email atau password salah"	Sesuai

login()	Login dengan password salah	email: admin@test.com, password: salah	Respons HTTP 401, pesan "Email atau password salah"	Sesuai
login()	Login dengan akun nonaktif	email: nonaktif@test.com, password: benar	Respons HTTP 403, pesan "Akun tidak aktif"	Sesuai
get_current_user()	Akses endpoint dengan token valid	Header Authorization: Bearer <token_valid>	Respons HTTP 200, objek user dikembalikan dengan user_id dan role yang sesuai	Sesuai
get_current_user()	Akses endpoint tanpa token	Tidak ada header Authorization	Respons HTTP 403, akses ditolak	Sesuai
get_current_user()	Akses endpoint dengan token kedaluwarsa	Header Authorization: Bearer <token_expired>	Respons HTTP 401, pesan "Token tidak valid atau sudah expired"	Sesuai
require_role()	Akses endpoint admin oleh role data_engineer	Token dengan role data_engineer	Respons HTTP 403, pesan "Akses ditolak"	Sesuai

create_user()	Pembuatan akun baru oleh admin	username: user_baru, email: baru@test.com, role: data_engineer	Respons HTTP 201, record baru tersimpan di tabel users, password_hash bukan plaintext, is_active = 1	Sesuai
create_user()	Pembuatan akun dengan email duplikat	email yang sudah digunakan	Respons HTTP 400, pesan "Username atau email sudah digunakan", tidak ada record baru di tabel users	Sesuai

4.7.2 Pengujian Modul Dataset

Pengujian whitebox testing pada modul dataset ditunjukkan pada tabel 4.2 berikut

Tabel 4.2: Pengujian Modul Dataset

Fungsi yang Diuji	Skenario	Data Input	Hasil yang Diharapkan	Hasil Pengujian
-------------------	----------	------------	-----------------------	-----------------

upload_ tabular()	Upload CSV valid bertipe general	File CSV dengan kolom nama, usia, label	Respons HTTP 201, record tersimpan di tabel datasets dengan data_type = "tabular" , dataset_role = "general" , status = "uploaded" , file fisik tersimpan di direktori uploads/tabular/	Sesuai
upload_ tabular()	Upload CSV d engan kolom verbatim , coding, analisis	File CSV dengan kolom verbatim, coding, analisis	dataset_role terdeteksi otomatis sebagai "lm_reference"	Sesuai
upload_ tabular()	Upload CSV dengan kolom verbatim dan coding	File CSV dengan kolom verbatim, coding	dataset_role terdeteksi otomatis sebagai "lm_new"	Sesuai

upload_ tabular()	Upload CSV dengan kolom verbatim saja	File CSV dengan kolom verbatim	dataset_role terdeteksi otomatis sebagai "llm_raw"	Sesuai
upload_ tabular()	Upload file bukan CSV	File .xlsx	Respons HTTP 400, pesan "Hanya file .csv yang diperbolehkan" , tidak ada file fisik tersimpan	Sesuai
upload_ tabular()	Upload CSV kosong	File CSV tanpa baris data	Respons HTTP 400, pesan "File CSV tidak boleh kosong", file fisik dihapus dari server	Sesuai
save_ image_ zip()	Upload ZIP dataset gambar valid	File ZIP berisi folder kucing/ dan anjing/ masing-masing berisi gambar	Respons HTTP 201, file ZIP diekstrak ke uploads/image/, file ZIP asli dihapus, meta berisi classes , total_images, class_counts yang benar	Sesuai

save_image_zip()	Upload ZIP tanpa struktur folder kelas	File ZIP berisi gambar langsung tanpa subfolder	Respons HTTP 400, pesan "ZIP tidak mengandung struktur folder yang valid", folder ekstraksi dihapus	Sesuai
get_dataset_quality()	Analisis kualitas dataset tabular dengan missing values	File CSV dengan beberapa sel kosong	Respons HTTP 200, total_missing sesuai jumlah sel kosong aktual, kolom dengan missing >10% berstatus "error", kolom dengan missing ≤10% berstatus "warning"	Sesuai
delete_dataset	Hapus dataset tabular	dataset_id valid	Respons HTTP 200, record dihapus dari tabel datasets, file fisik CSV dihapus dari direktori server	Sesuai

4.7.3 Pengujian Modul Privacy Consent

Pengujian whitebox testing pada modul privacy concent ditunjukkan pada tabel 4.3 berikut

Tabel 4.3: Pengujian Modul Privacy Concent

Fungsi yang Diuji	Skenario	Data Input	Hasil yang Diharapkan	Hasil Pengujian
upload_consent_pdf()	Upload PDF bukti consent valid	File .pdf berukuran < 10MB	Respons HTTP 200, file PDF tersimpan di direktori consent_pdfs/ dengan nama unik berbasis UUID, nama file dikembalikan dalam respons	Sesuai
upload_consent_pdf()	Upload file bukan PDF	File .jpg	Respons HTTP 400, pesan "Hanya file PDF yang diperbolehkan", tidak ada file tersimpan	Sesuai

encrypt_nik()	Enkripsi NIK subjek	NIK: 320123456 7890001	Nilai nik_encrypted pada tabel privacy_consent_subjects bukan plaintext NIK asli, dapat didekripsi kembali menggunakan kunci yang sama	Sesuai
submit_consent()	Semua subjek setuju simpan dan proses	2 subjek dengan agree_store = true, agree_process = true, masing-masing memiliki pdf_filename	Respons HTTP 200 dengan result = "approved", dataset.status = "uploaded", record privacy_consent tersimpan dengan status_process = "approved"	Sesuai

submit_consent()	Ada subjek yang tidak setuju pemrosesan	2 subjek: 1 agree _process = true, 1 agree_process = false	Respons HTTP 200 dengan result = " locked", dataset.status = "locked", record privacy_consent tersimpan dengan status _process = "rejected"	Sesuai
submit_consent()	Ada subjek yang tidak setuju penyimpanan	1 subjek dengan agree_store = false	Respons HTTP 200 dengan result = "rejected_store", record dataset dihapus dari tabel datasets, file fisik dataset dihapus dari server	Sesuai
submit_consent()	Ada subjek tanpa bukti PDF	1 subjek dengan pdf_filename = null	Respons HTTP 200 dengan result = "rejected_store", dataset beserta file fisiknya dihapus	Sesuai

Preprocessing dataset locked	Mencoba preprocessing dataset berstatus locked	dataset_id dengan status locked	Respons HTTP 403 , pesan "Dataset tidak dapat diproses karena ada subjek yang tidak menyetujui pemrosesan data"	Sesuai
------------------------------	--	---------------------------------	---	--------

4.7.4 Pengujian Modul Preprocessing

Pengujian whitebox testing pada modul preprocessing ditunjukkan pada tabel 4.4 berikut

Tabel 4.4: Pengujian Modul Preprocessing

Fungsi yang Diuji	Skenario	Data Input	Hasil yang Diharapkan	Hasil Pengujian
create_preprocessing_job()	Preprocessing dataset tabular valid	dataset_id berstatus uploaded, config_type = "tabular", konfigurasi lengkap	Respons HTTP 201 segera dikembalikan, record preprocessings tersimpan dengan status = "pending", background task terdaftar	Sesuai

create_preprocessing_job()	Preprocessing dengan tipe config tidak sesuai	Dataset bertipe tabular tetapi config_type = "image"	Respons HTTP 400, pesan "Dataset bertipe 'tabular' tapi config bertipe 'image'"	Sesuai
run_preprocessing_job()	Preprocessing tabular berhasil	Konfigurasi valid dengan target_column, feature_columns, missing_strategy, scaling, encoding	Status berubah pending → running → completed, file X.pkl dan y.pkl tersimpan di direktori preprocessed/prep_{id}/, dataset.status berubah menjadi "preprocessed", metadata ditulis otomatis	Sesuai
run_preprocessing_job()	Preprocessing gambar berhasil	Konfigurasi valid dengan image_size = 64, grayscale = false	Status berubah hingga completed, file hasil HOG tersimpan di preprocessed/prep_{id}/, metadata ditulis otomatis	Sesuai

run_preprocessing_job()	Preprocessing teks berhasil	Konfigurasi valid dengan labels, remove_stopwords = true, language = "indonesian", max_features = 5000	Status berubah hingga completed, file hasil TF-IDF tersimpan di preprocessed/prep_{id}/, metadata ditulis otomatis	Sesuai
get_preprocessing_by_id()	Polling status preprocessing	prep_id valid saat proses berjalan	Respons HTTP 200, field status mencerminkan kondisi terkini (pending/running/completed/failed)	Sesuai

4.7.5 Pengujian Modul Eksperimen dan Training

Pengujian whitebox testing pada modul eksperimen dan training ditunjukkan pada tabel 4.5 berikut

Tabel 4.5: Pengujian Modul Eksperimen dan Training

Fungsi yang Diuji	Skenario	Data Input	Hasil yang Diharapkan	Hasil Pengujian
-------------------	----------	------------	-----------------------	-----------------

create_experiment()	Training klasifikasi dataset tabular	preprocessing_id berstatus completed, algorithm = "random_forest", task_type = "classification"	Respons HTTP 201 segera dikembalikan, record experiments tersimpan dengan status = "queued", progress = 0	Sesuai
create_experiment()	Training dengan preprocessing belum selesai	preprocessing_id berstatus running	Respons HTTP 400, pesan "Preprocessing belum selesai"	Sesuai
create_experiment()	Training dengan algoritma tidak valid	algorithm = "naive_bayes" untuk dataset tabular	Respons HTTP 400, pesan daftar algoritma yang diizinkan untuk kombinasi data_type/task_type tersebut	Sesuai
create_experiment()	Training dataset gambar	preprocessing_id dari dataset gambar, algorithm = "svm"	task_type ditetapkan otomatis sebagai "classification" meskipun tidak dikirimkan dalam request	Sesuai

run_training_job()	Training model klasifikasi berhasil	Konfigurasi eksperimen valid, file X.pkl dan y.pkl tersedia	Status berubah queued, running, completed, progres meningkat bertahap hingga 100%, file model exp_{id}.pkl tersimpan di direktori models/, field metrics terisi dengan akurasi, presisi, recall, dan F1-score	Sesuai
run_training_job()	Training model regresi berhasil	Konfigurasi dengan task_type = "regression", algorithm = "linear_regression"	Status completed, field metrics terisi dengan MSE, RMSE, MAE, dan R ² -score	Sesuai

download_model()	Unduh model terlatih	exp_id dengan status completed	Respons HTTP 200 berupa file stream .pkl, file dapat diunduh dengan nama {nama_eksperimen}.pkl	Sesuai
download_model()	Unduh model yang belum selesai	exp_id dengan status running	Respons HTTP 400 , pesan "Model belum tersedia"	Sesuai

4.7.6 Pengujian Modul Analisis LLM

Pengujian whitebox testing pada modul analisis LLM ditunjukkan pada tabel 4.6 berikut

Tabel 4.6: Pengujian Modul Analisis LLM

Fungsi yang Diuji	Skenario	Data Input	Hasil yang Diharapkan	Hasil Pengujian
_get_available_models()	Ambil daftar model Ollama	Ollama berjalan dengan model qwen2.5:7b, mistral:7b, llama3.2:3b terinstall	Respons HTTP 200, daftar nama model dikembalikan sesuai model yang terinstall di Ollama	Sesuai

generate_analysis()	Generate analisis dengan dataset referensi dan dataset baru valid	dataset_reference_id bertipe llm_reference, dataset_new_id bertipe llm_new, model_name = "qwen2.5:7b"	Respons HTTP 200 segera dikembalikan, record llm_analyses tersimpan dengan status = "pending", background task terdaftar	Sesuai
generate_analysis()	Generate analisis dengan dataset referensi bertipe salah	dataset_reference_id bertipe general	Respons HTTP 400, pesan "Dataset referensi harus bertipe llm_reference"	Sesuai
generate_analysis()	Generate analisis dengan dataset baru terkunci	dataset_new_id berstatus locked	Respons HTTP 403, pesan "Dataset baru terkunci karena consent tidak disetujui"	Sesuai
generate_analysis()	Generate analisis dengan model tidak tersedia	model_name = "gpt4"	Respons HTTP 400, pesan "Model tidak tersedia di Ollama"	Sesuai

_run _analysis()	Proses generate analisis berhasil	Dataset referens i dan data baru valid, Ollama berjalan	Status berubah pending, running, completed, field results berisi daftar objek {verbatim, coding, analisis}, field rouge_scores berisi nilai ROUGE-1, ROUGE-2, ROUGE-L, field generate_ time terisi waktu aktual dalam detik	Sesuai
_build _prompt()	Build prompt untuk dataset llm_raw	Dataset baru bertipe llm_raw (hanya kolom verbatim)	Prompt yang dihasilkan meminta model untuk menghasilkan CODING sekaligus ANALISIS untuk setiap segmen	Sesuai

<code>_build_prompt()</code>	Build prompt untuk dataset <code>llm_new</code>	Dataset baru bertipe <code>llm_new</code> (kolom verbatim dan coding)	Prompt yang dihasilkan hanya meminta model untuk menghasilkan ANALISIS berdasarkan verbatim dan coding yang sudah ada	Sesuai
<code>_calculate_rouge()</code>	Hitung skor ROUGE hasil analisis	Teks hasil generate dan teks referensi tersedia	Nilai ROUGE-1, ROUGE-2, dan ROUGE-L dikembalikan dalam rentang 0.0–1.0, tersimpan di field <code>rouge_scores</code> tabel <code>llm_analyses</code>	Sesuai
<code>get_analysis_status()</code>	Polling status analisis	<code>analysis_id</code> valid saat proses berjalan	Respons HTTP 200 , field status mencerminkan kondisi terkini (pending/running/completed/failed)	Sesuai

5. PENUTUP

5.1 Kesimpulan

Berdasarkan hasil perancangan, implementasi, dan evaluasi sistem yang telah dilakukan, maka dapat ditarik beberapa kesimpulan yang menjawab rumusan masalah penelitian sebagai berikut:

1. Komponen back end platform Automated Machine Learning berbasis web berhasil dirancang dan dibangun menggunakan framework FastAPI dengan arsitektur berlapis (layered architecture) yang terdiri dari Router Layer, Service Layer, Model Layer, dan Utility Layer. Sistem yang dihasilkan terdiri dari delapan modul endpoint API yang saling terintegrasi, yaitu modul autentikasi, admin, dataset, privacy consent, preprocessing, eksperimen, metadata, dan analisis LLM. Seluruh fungsi utama sistem berhasil diverifikasi melalui pengujian white box testing dan menghasilkan perilaku yang sesuai dengan rancangan yang ditetapkan, termasuk alur asinkron pada modul preprocessing dan eksperimen yang menggunakan mekanisme background task untuk memastikan API tetap responsif selama proses panjang berlangsung.
2. Fitur analisis teks berbasis Large Language Model lokal melalui Ollama berhasil diintegrasikan ke dalam sistem back end. Sistem mendukung tiga model LLM (mistral:7b, llama3.2:3b, dan qwen2.5:7b) dan mengimplementasikan pendekatan few-shot prompting menggunakan dataset referensi sebagai panduan gaya penulisan analisis. Kualitas hasil analisis dievaluasi secara objektif menggunakan metrik ROUGE-1, ROUGE-2, dan ROUGE-L, serta dicatat waktu generate dan rata-rata panjang analisis yang dihasilkan, sehingga pengguna dapat membandingkan performa antar model secara terukur.

3. Mekanisme privacy consent management berhasil diimplementasikan pada sistem back end dengan mengacu pada ketentuan Undang-Undang Nomor 27 Tahun 2022 tentang Pelindungan Data Pribadi. Sistem menerapkan persetujuan granular yang memisahkan izin penyimpanan data dari izin pemrosesan data, disertai bukti persetujuan tertulis dalam bentuk dokumen PDF yang diunggah sebagai pemenuhan syarat persetujuan terekam. Data identitas subjek berupa NIK dienkripsi menggunakan algoritma Fernet sebelum disimpan ke basis data. Dataset yang memiliki subjek yang tidak menyetujui pemrosesan data secara otomatis dikunci dan tidak dapat digunakan untuk preprocessing maupun analisis lebih lanjut.

5.2 Saran

Berdasarkan kesimpulan dan keterbatasan yang ditemukan selama penelitian, berikut adalah beberapa saran yang dapat menjadi acuan untuk pengembangan selanjutnya:

1. Platform ini telah mendukung algoritma machine learning klasik berbasis Scikit-learn yang sesuai dengan karakteristik data penelitian. Pengembangan lanjutan dapat mempertimbangkan penambahan dukungan terhadap algoritma berbasis PyTorch atau TensorFlow untuk memperluas opsi pemodelan bagi pengguna yang memiliki kebutuhan analisis yang lebih kompleks.
2. Platform saat ini mendukung tiga tipe data yang paling umum digunakan dalam penelitian. Pengembangan selanjutnya dapat memperluas dukungan tipe data dengan menambahkan format lain seperti data audio atau data time-series untuk menjangkau domain penelitian yang lebih luas.
3. Penggunaan Model LLM saat ini menggunakan Ollama, dapat dikembangkan lebih lanjut dengan menambahkan dukungan model LLM baru yang lebih mutakhir seiring dengan perkembangan ekosistem Ollama.
4. Pengujian white box testing yang telah berhasil memverifikasi kebenaran logika seluruh modul sistem dapat dilengkapi dengan pengujian

beban (load testing) pada penelitian lanjutan untuk mengkarakterisasi kapasitas sistem dalam skenario penggunaan yang lebih intensif.

DAFTAR PUSTAKA

Barros, C. F., Azevedo, B. B., Graciano Neto, V. V., Kassab, M., Kalinowski, M., Nascimento, H. A. D., & Bandeira, M. C. G. S. P. (2025). Large Language Model for Qualitative Research — A Systematic Mapping Study. arXiv. Diakses dari <https://doi.org/10.48550/arXiv.2411.14473> Giraldo, L., & Laso, S. (2025). Democratizing Machine Learning: A Practical Comparison of Low-Code and No-Code Platforms. *Machine Learning and Knowledge Extraction*, 7(4), 141. Diakses dari <https://doi.org/10.3390/make7040141> Nurmiati, E., & Kusuma, F. T. (2026). Pelindungan Data Pelanggan pada Pengembangan Sistem Informasi Transaksi Bisnis dan Pemasaran Digital. *Jurnal Bisnis dan Teknologi Informasi*, 1(3), 131–138. Republik Indonesia. (2022). Undang-Undang Republik Indonesia Nomor 27 Tahun 2022 tentang Pelindungan Data Pribadi. Jakarta: Sekretariat Negara.

Peppers, K., Tuunanen, T., Rothenberger, M. A., & Chatterjee, S. (2007). A Design Science Research Methodology for Information Systems Research. *Journal of Management Information Systems*, 24(3), 45–77. Diakses dari <https://doi.org/10.2753/MIS0742-1222240302>

Inan, D. I. (2023). Kapan Evaluasi Sebuah Artefak Sistem Informasi Dianggap Cukup? Perspektif Design Science Research. *Jurnal Sistem Informasi, Teknologi Informasi, dan Edukasi Sistem Informasi (JUST-SI)*, 4(1), 11–18.

Khan, R. A. H., Gopal, P. R., Yogesh, P. B., Nitin, P. V., Sahebrao, K. S., Sharad, P. N., & Santosh, P. V. (2024). The Waterfall Model-Software Engineering. *International Journal of Research Publication and Reviews*, 5(4), 7825–7830.

Tjahjanto, Arista, A., & Ermatita. (2022). Application of the Waterfall Method in Information System for State-owned Inventories Management Development. *Sinkron: Jurnal dan Penelitian Teknik Informatika*, 6(4), 2182–2192. Diakses dari <https://doi.org/10.33395/sinkron.v7i4.11678>

Feng, Y., Yao, Y., & Sadeh, N. (2021). A Design Space for Privacy Choices: Towards Meaningful Privacy Control in the Internet of Things. CHI Conference on Human Factors in Computing Systems (CHI '21). Diakses dari <https://doi.org/10.1145/3411764.3445148>

European Parliament and Council of the European Union. (2016). Regulation (EU) 2016/679 (General Data Protection Regulation). Official Journal of the European Union, L119/1.

Aghili, S., Mohammadi, S., & Aghili, M. (2024). Understanding Web Application Workloads and Their Applications. (lengkapi detail volume/halaman sesuai file web_Agili.pdf di project)

Saia, R., Carta, S., Recupero, D. R., & Fenu, G. (2022). (lengkapi detail lengkap sesuai file web_Saia.pdf di project)

Nyabuto, G. O. (2024). Client-server Architecture, a Review. (lengkapi detail lengkap sesuai file web_Nyabuto.pdf di project)

Castro, R., dkk. (2023). (lengkapi detail lengkap sesuai file python_castro.pdf di project)

Lo, R., Yunanto, A. E., Movia, R. N., Soehardjianto, L. A., Wangsa, F., Lidjaja, N. A., & Ningsih, R. Y. (2023). Penggunaan Bahasa Pemrograman Python dalam Menganalisis Hubungan Kualitas Kopi dengan Lokasi Pertanian Kopi. *Jurnal Publikasi Teknik Informatika (JUPTI)*, 2(2), 100–109.

Wardhana, R. G., Wang, G., & Sibuea, F. (2023). Penerapan Machine Learning dalam Prediksi Tingkat Kasus Penyakit di Indonesia. *Journal of Information System Management (JOISM)*, 5(1).

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, E. (2011). Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.

Buitinck, L., Louppe, G., Blondel, M., Pedregosa, F., Mueller, A., Grisel, O., Niculae, V., Prettenhofer, P., Gramfort, A., Grobler, J., Layton, R., Vanderplas, J., Joly, A., Holt, B., & Varoquaux, G. (2013). API Design for Machine Learning Software: Experiences from the Scikit-learn Project. *ECML PKDD Workshop: Languages for Data Mining and Machine Learning*, 108–122.

Fahmi, dkk. (2023). (lengkapi detail lengkap sesuai file ml_fahmi.pdf di project)

FastAPI. (2024). FastAPI framework, high performance, easy to learn, fast

to code, ready for production. Diakses dari <https://fastapi.tiangolo.com/>

Azhari, M. (2022). (lengkapi detail lengkap sesuai file FastAPI_Azari.pdf di project)

Alimi, dkk. (2024). (lengkapi detail lengkap sesuai file FastAPI_alimi.pdf di project)

Maratkar, A., & Adkar, S. (2021). React JS: A Modern Approach to Web Development. *IRE Journals*, 4(12), 98–102.

Juan, K., & Budi, S. (2023). Pengembangan Menu Digital Menggunakan ReactJs Implementasi Hasil Belajar Studi Independen di Frontend Engineering Program Ruanguru CAMP. *Jurnal Strategi*, 5(1), 130–138.

Jonathan, J., & Supriyadi. (2023). (lengkapi detail lengkap sesuai file reactjs_jonathan.pdf di project)

Muharom, dkk. (2025). (lengkapi detail lengkap sesuai file RestAPI_muharom.pdf di project)

Gowda, dkk. (2024). (lengkapi detail lengkap sesuai file RestAPI_Growda.pdf di project)

Edy, Ferdiansyah, Pramusinto, W., & Waluyo, S. (2019). Pengamanan Restful API menggunakan JWT untuk Aplikasi Sales Order. *Jurnal RESTI (Rekayasa Sistem dan Teknologi Informasi)*, 3(2), 106–112.

Bharadwaj, A., dkk. (2019). (lengkapi detail lengkap sesuai file sqllite_baradwaj.pdf di project)

Nugraha, dkk. (2023). (lengkapi detail lengkap sesuai file sqllite_nugraha.pdf di project)

Herawan, M. H. S., & Ma'sum, H. (2024). Implementasi Aplikasi E-Commerce Untuk Flamboyant Bakery Dengan Menggunakan Python Flask. *JTSI*, 5(1), 12–23.

Nashikhuddin, A. Y., Karaman, J., & Litanianda, Y. (2023). Implementasi API RESTful dengan JSON Web Token (JWT) pada Aplikasi E-Commerce Thrifty Shop untuk Otentikasi dan Otorisasi Pengguna. *METHOMIKA: Jurnal Manajemen Informatika & Komputerisasi Akuntansi*, 7(2), 239–246. Diakses dari <https://doi.org/10.46880/jmika.Vol7No2.pp239-246>

Cahyadi, R., & Akbi, D. R. (2026). Implementasi RESTful API Menggunakan JSON Web Token pada Website Manajemen Infrastruktur. *JUPI (Jurnal Ilmiah Penelitian dan Pembelajaran Informatika)*, 11(2), 996–1009. Diakses dari <https://doi.org/10.29100/jupi.v11i2.8202>

Mahesh, B. (2020). Machine Learning Algorithms — A Review. Interna-

tional Journal of Science and Research, 9(1), 381–386.

Suyal, M., & Goyal, P. (2022). A Review on Analysis of K-Nearest Neighbor Classification Machine Learning Algorithms based on Supervised Learning. *International Journal of Engineering Trends and Technology*, 70(7), 43–48. Diakses dari <https://doi.org/10.14445/22315381/IJETT-V70I7P205>

Nurhalizah, R. S., Ardianto, R., & Purwono. (2024). Analisis Supervised dan Unsupervised Learning pada Machine Learning: Systematic Literature Review. *Jurnal Ilmu Komputer dan Informatika (JIKI)*, 4(1), 61–72. Diakses dari <https://doi.org/10.54082/jiki.168>

Charbuty, B., & Abdulazeez, A. (2021). Classification Based on Decision Tree Algorithm for Machine Learning. *Journal of Applied Science and Technology Trends*, 2(01), 20–28. Diakses dari <https://doi.org/10.38094/jastt20165>

Mahmuda, S. (2024). (lengkapi detail lengkap sesuai file `algoritmaml_mahmuda.pdf` di project)

Ovirianti, dkk. (2022). (lengkapi detail lengkap sesuai file `algoritmaml_ovirianti.pdf` di project)

Hidayat, dkk., & Wahyudi. (2024). (lengkapi detail lengkap sesuai file `algoritmaml_hidayat.pdf` di project)

Fahmuddin, dkk. (2023). (lengkapi detail lengkap sesuai file `algoritmaml_fahmuddin.pdf` di project)

Miller, dkk. (2024). (lengkapi detail lengkap sesuai file `evaluasiml_miller.pdf` di project)

Kurniawati, dkk. (2025). (lengkapi detail lengkap sesuai file `evaluasiml_kurniawati.pdf` di project)

Widodo, dkk. (2022). (lengkapi detail lengkap sesuai file `evaluasiml_widodo.pdf` di project)

Zöller, M. A., & Huber, M. F. (2021). Benchmark and Survey of Automated Machine Learning Frameworks. *Journal of Artificial Intelligence Research*, 70, 409–472.

Rahman, dkk. (2023). (lengkapi detail lengkap sesuai file `automl_rahman.pdf` di project)

Nwokonkwo, dkk. (2022). (lengkapi detail lengkap sesuai file `automl_Nwokonkwo.pdf` di project)

Aprianto, A. N., Girsang, A. S., Nugroho, Y., & Putra, W. K. (2024). Analisis Kinerja Streaming RabbitMQ dan Nats untuk Komunikasi di Layanan

Mikro. TEKNOLOGI: Jurnal Ilmiah Sistem Informasi, 14(1), 37–47. Diakses dari <https://doi.org/10.26594/teknologi.v14i1.4498>

Ramadhani, A. F., & Nurhidayat, A. K. (2026). Implementasi LLM Ollama untuk Sistem Retrieval-Augmented Generation (RAG) Secara Lokal dalam Analisis Dokumen PDF Berbentuk Chatbot. JMI, 15(1), 287–294.

Riza, F., Al Din, S. J., Al Afghani, D. Y., Setiabudi, R., & Wibisono. (2025). LLM-Based Self-Related Local AI Agent Design through N8N Orchestration for Conversational Memory on RAG. Journal of Information Technology and Computer Science (INTECOMS), 8(3), 939–946.

Albert, G. D., & Voutama, A. (2025). Pengembangan Chatbot Berbasis PDF Menggunakan Local Retrieval-Augmented Generation (RAG) dan Ollama. JITET (Jurnal Informatika dan Teknik Elektro Terapan), 13(2). Diakses dari <https://doi.org/10.23960/jitet.v13i2.6361>

Ahmad, F., Rahman, H., & Sutabri, T. (2025). Integrasi Model LLM Ollama dan OpenStreetMap API pada BI untuk Rekomendasi Lokasi. Saturnus: Jurnal Teknologi dan Sistem Informasi, 3(4), 167–179. Diakses dari <https://doi.org/10.61132/saturnus.v3i4.1138>

DAFTAR PUSTAKA

LAMPIRAN

Lampiran 1 *Listing* Program

Config.py

```
import os
```

Logic.py

```
import os
```

Model.py

```
import os
```

Lampiran 2 *Output* Program

Tampilan *Chatbot*



gambar/Login.png